

# Machine Learning

Minh Pham Van

January, 2026

# Contents

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>Lời nói đầu</b>                                                  | <b>1</b>  |
| <b>1 Introduction to Machine Learning</b>                           | <b>3</b>  |
| 1.1 Overview and Definitions . . . . .                              | 3         |
| 1.1.1 Basic Concepts . . . . .                                      | 3         |
| 1.1.2 Arthur Samuel Definition (1959) . . . . .                     | 3         |
| 1.2 Formal Definition (Tom Mitchell - 1998) . . . . .               | 3         |
| 1.2.1 Well-posed Learning Problem . . . . .                         | 3         |
| 1.2.2 Example: Spam Email Filter . . . . .                          | 4         |
| 1.3 Classification of Machine Learning Algorithms . . . . .         | 4         |
| 1.3.1 Main Branches . . . . .                                       | 4         |
| 1.3.2 Deep Dive: Supervised Learning (SL) . . . . .                 | 4         |
| 1.4 Supervised Learning Example: Housing Price Prediction . . . . . | 5         |
| 1.4.1 Problem Description . . . . .                                 | 5         |
| 1.4.2 Characteristics of Supervised Learning . . . . .              | 6         |
| 1.5 Unsupervised Learning (USL) . . . . .                           | 6         |
| 1.5.1 Characteristics . . . . .                                     | 6         |
| <b>2 Linear Regression</b>                                          | <b>7</b>  |
| 2.1 Problem Definition . . . . .                                    | 7         |
| 2.1.1 Data and Goal . . . . .                                       | 7         |
| 2.1.2 Model Representation . . . . .                                | 8         |
| 2.2 Loss Function (Cost Function) . . . . .                         | 8         |
| 2.2.1 Distance Metrics . . . . .                                    | 8         |
| 2.2.2 Objective . . . . .                                           | 8         |
| 2.3 Optimization Derivatives . . . . .                              | 8         |
| 2.3.1 Concept of Derivative . . . . .                               | 8         |
| 2.4 Gradient Descent . . . . .                                      | 9         |
| 2.4.1 Update Rule . . . . .                                         | 9         |
| 2.4.2 Applying to Linear Regression . . . . .                       | 9         |
| 2.5 Chain Rule Backpropagation . . . . .                            | 10        |
| 2.5.1 Computing Derivatives . . . . .                               | 10        |
| <b>3 Gradient Descent Derivation</b>                                | <b>11</b> |
| 3.1 Giới thiệu và Hàm Giả Thuyết (Hypothesis Function) . . . . .    | 11        |
| 3.1.1 Tập dữ liệu huấn luyện (Training Set) . . . . .               | 11        |
| 3.2 Hàm Mất Mất (MSE Cost Function) . . . . .                       | 11        |
| 3.3 Thuật toán Gradient Descent . . . . .                           | 12        |

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| 3.3.1    | Khái niệm và Quy tắc cập nhật . . . . .                     | 12        |
| 3.3.2    | Mô phỏng quá trình hội tụ . . . . .                         | 12        |
| 3.4      | Phân tích Trực quan (Intuition) . . . . .                   | 13        |
| 3.4.1    | Độ lớn và Dấu của Đạo hàm . . . . .                         | 13        |
| 3.4.2    | Vai trò của Learning Rate ( $\alpha$ ) . . . . .            | 13        |
| 3.4.3    | Tiêu chuẩn dừng . . . . .                                   | 14        |
| 3.5      | Đạo hàm chi tiết cho Hồi quy Tuyến tính . . . . .           | 14        |
| 3.5.1    | Các bước chứng minh (Derivation Steps) . . . . .            | 14        |
| <b>4</b> | <b>Logistic Regression (Classification)</b>                 | <b>15</b> |
| 4.1      | Giới thiệu về Bài toán Phân loại (Classification) . . . . . | 15        |
| 4.1.1    | Tại sao không dùng Linear Regression? . . . . .             | 15        |
| 4.2      | Mô hình Logistic Regression . . . . .                       | 16        |
| 4.2.1    | Hàm Sigmoid . . . . .                                       | 16        |
| 4.2.2    | Ý nghĩa của đầu ra (Interpretation) . . . . .               | 17        |
| 4.3      | Ranh giới Quyết định (Decision Boundary) . . . . .          | 17        |
| 4.3.1    | Ví dụ: Linear Decision Boundary . . . . .                   | 17        |
| 4.3.2    | Ví dụ: Non-linear Decision Boundary . . . . .               | 18        |
| 4.4      | Hàm Mất Mát (Cost Function) . . . . .                       | 18        |
| 4.4.1    | Tại sao không dùng MSE? . . . . .                           | 18        |
| 4.4.2    | Logistic Regression Cost Function (Log Loss) . . . . .      | 19        |
| 4.4.3    | Công thức thu gọn (Simplified Cost Function) . . . . .      | 19        |
| 4.5      | Gradient Descent cho Logistic Regression . . . . .          | 20        |
| 4.6      | Phân loại Đa lớp (Multiclass Classification) . . . . .      | 20        |
| 4.6.1    | Phương pháp One-vs-All (One-vs-Rest) . . . . .              | 20        |
| <b>5</b> | <b>Regularization (Điều chuẩn)</b>                          | <b>22</b> |
| 5.1      | Vấn đề Overfitting và Underfitting . . . . .                | 22        |
| 5.1.1    | Định nghĩa các trạng thái của mô hình . . . . .             | 22        |
| 5.2      | Giải pháp khắc phục Overfitting . . . . .                   | 23        |
| 5.3      | Hàm Chi phí (Cost Function) có Regularization . . . . .     | 23        |
| 5.3.1    | Trực giác (Intuition) . . . . .                             | 23        |
| 5.3.2    | Công thức tổng quát . . . . .                               | 23        |
| 5.4      | Vai trò của tham số $\lambda$ . . . . .                     | 24        |
| 5.5      | Regularized Gradient Descent . . . . .                      | 24        |
| 5.5.1    | Cập nhật $\theta_0$ . . . . .                               | 24        |
| 5.5.2    | Cập nhật $\theta_j$ (với $j = 1, 2, \dots, n$ ) . . . . .   | 24        |
| <b>6</b> | <b>Non-linear Hypotheses &amp; Neural Networks</b>          | <b>25</b> |
| 6.1      | Non-linear Classification (Phân loại phi tuyến) . . . . .   | 25        |
| 6.1.1    | Hạn chế của Linear/Logistic Regression . . . . .            | 25        |
| 6.1.2    | Ví dụ: Computer Vision (Thị giác máy tính) . . . . .        | 25        |
| 6.2      | Neural Networks (Mạng Nơ-ron) . . . . .                     | 26        |
| 6.2.1    | Lịch sử và Nguồn gốc . . . . .                              | 26        |
| 6.2.2    | Giả thuyết "One Learning Algorithm" . . . . .               | 26        |
| 6.3      | Mô hình Neural Network (Model Representation) . . . . .     | 26        |
| 6.3.1    | Mô phỏng Nơ-ron sinh học . . . . .                          | 26        |
| 6.3.2    | Nơ-ron nhân tạo (Artificial Neuron) . . . . .               | 27        |

|          |                                                                                |           |
|----------|--------------------------------------------------------------------------------|-----------|
| 6.3.3    | Kiến trúc Mạng Nơ-ron . . . . .                                                | 28        |
| 6.4      | Forward Propagation (Lan truyền xuôi) . . . . .                                | 28        |
| 6.4.1    | Tính toán từng bước (Vectorized Implementation) . . . . .                      | 30        |
| 6.5      | Trực giác về Neural Network (Logic Gates) . . . . .                            | 31        |
| 6.5.1    | Ví dụ đơn giản: Cổng AND . . . . .                                             | 31        |
| 6.5.2    | Bài toán XNOR (Non-linear) . . . . .                                           | 31        |
| 6.6      | Multi-class Classification (Phân loại đa lớp) . . . . .                        | 34        |
| <b>7</b> | <b>Neural Network: Cost Function &amp; Propagation</b>                         | <b>36</b> |
| 7.1      | Tổng quan và Ký hiệu . . . . .                                                 | 36        |
| 7.1.1    | Ký hiệu kiến trúc mạng . . . . .                                               | 36        |
| 7.1.2    | Phân loại (Classification Types) . . . . .                                     | 36        |
| 7.2      | Hàm Chi phí (Cost Function) . . . . .                                          | 37        |
| 7.3      | Forward Propagation (Lan truyền xuôi) . . . . .                                | 37        |
| 7.3.1    | Các bước tính toán . . . . .                                                   | 37        |
| 7.4      | Backward Propagation (Lan truyền ngược) . . . . .                              | 38        |
| 7.4.1    | Đạo hàm tại lớp đầu ra . . . . .                                               | 38        |
| 7.4.2    | Gradient của trọng số lớp Output ( $W^{(2)}$ ) . . . . .                       | 38        |
| 7.4.3    | Gradient của trọng số lớp Input/Hidden ( $W^{(1)}$ ) . . . . .                 | 39        |
| 7.5      | Tổng kết Công thức Cập nhật . . . . .                                          | 39        |
| <b>8</b> | <b>Advice for Applying Machine Learning</b>                                    | <b>40</b> |
| 8.1      | Debugging a Learning Algorithm (Gỡ lỗi thuật toán học) . . . . .               | 40        |
| 8.2      | Evaluating a Hypothesis (Đánh giá giả thuyết) . . . . .                        | 40        |
| 8.2.1    | Training / Test Split (Chia tập Huấn luyện / Kiểm tra) . . . . .               | 40        |
| 8.2.2    | Công thức tính lỗi trên tập test . . . . .                                     | 41        |
| 8.3      | Model Selection (Lựa chọn mô hình) . . . . .                                   | 41        |
| 8.3.1    | Cross Validaton . . . . .                                                      | 41        |
| 8.3.2    | Quy trình lựa chọn mô hình . . . . .                                           | 43        |
| 8.4      | Diagnosing Bias vs Variance (Chẩn đoán Độ lệch và Phương sai) . . . . .        | 43        |
| 8.4.1    | Phân tích dựa trên đồ thị lỗi . . . . .                                        | 44        |
| 8.5      | Regularization và Bias/Variance . . . . .                                      | 44        |
| 8.6      | Learning Curves (Đường cong học tập) . . . . .                                 | 45        |
| 8.6.1    | Trường hợp High Bias (Underfitting) . . . . .                                  | 45        |
| 8.6.2    | Trường hợp High Variance (Overfitting) . . . . .                               | 46        |
| 8.7      | Tổng kết giải pháp (Deciding What to Do Next) . . . . .                        | 47        |
| <b>9</b> | <b>Machine Learning System Design (Thiết kế hệ thống học máy)</b>              | <b>48</b> |
| 9.1      | Building a Spam Classifier (Xây dựng bộ phân loại thư rác) . . . . .           | 48        |
| 9.1.1    | Problem Description (Mô tả bài toán) . . . . .                                 | 48        |
| 9.1.2    | Feature Representation (Biểu diễn đặc trưng) . . . . .                         | 48        |
| 9.1.3    | Debugging a Learning Algorithm (Gỡ lỗi thuật toán học) . . . . .               | 48        |
| 9.2      | Error Analysis (Phân tích lỗi) . . . . .                                       | 49        |
| 9.2.1    | Ví dụ thực tế . . . . .                                                        | 49        |
| 9.3      | Error Metrics for Skewed Classes (Độ đo lỗi cho lớp lệch) . . . . .            | 50        |
| 9.3.1    | Vấn đề với Accuracy (Độ chính xác) . . . . .                                   | 50        |
| 9.4      | Precision and Recall (Độ chính xác và Độ phủ) . . . . .                        | 50        |
| 9.4.1    | Định nghĩa . . . . .                                                           | 50        |
| 9.5      | Trading off Precision and Recall (Đánh đổi giữa Precision và Recall) . . . . . | 50        |



|           |                                                              |           |
|-----------|--------------------------------------------------------------|-----------|
| 9.6       | F1 Score (Điểm F1)                                           | 51        |
| 9.6.1     | Tổng quát: $F_\beta$ Score                                   | 51        |
| <b>10</b> | <b>Clustering (Phân cụm)</b>                                 | <b>52</b> |
| 10.1      | Giới thiệu về Clustering                                     | 52        |
| 10.1.1    | Các ứng dụng của Clustering                                  | 53        |
| 10.2      | Thuật toán K-means                                           | 53        |
| 10.2.1    | Quy trình thuật toán                                         | 53        |
| 10.3      | Hàm Mục tiêu (Optimization Objective)                        | 55        |
| 10.4      | <b>Chứng minh hàm tối ưu:</b>                                | 56        |
| 10.5      | Khởi tạo ngẫu nhiên (Random Initialization)                  | 56        |
| 10.6      | Lựa chọn số cụm K (Choosing the Value of K)                  | 57        |
| 10.6.1    | Phương pháp Elbow (Khuỷu tay)                                | 57        |
| 10.6.2    | Dựa trên mục đích sử dụng (Downstream Purpose)               | 57        |
| 10.6.3    | Khi nào thì sử dụng K-Means ?                                | 58        |
| <b>11</b> | <b>Dimensionality Reduction (Giảm chiều dữ liệu)</b>         | <b>59</b> |
| 11.1      | Giới thiệu về Dimensionality Reduction                       | 59        |
| 11.1.1    | Động lực (Motivation)                                        | 59        |
| 11.1.2    | Ví dụ minh họa                                               | 60        |
| 11.2      | Principal Component Analysis (PCA)                           | 60        |
| 11.2.1    | Định nghĩa bài toán                                          | 60        |
| 11.2.2    | Các bước thực hiện PCA                                       | 62        |
| 11.3      | Bản chất PCA                                                 | 63        |
| 11.4      | Lựa chọn số chiều K (Choosing K)                             | 64        |
| 11.5      | Lời khuyên khi áp dụng PCA                                   | 65        |
| 11.5.1    | Khi nào nên dùng PCA?                                        | 65        |
| 11.5.2    | Sai lầm thường gặp (Bad use of PCA)                          | 65        |
| <b>12</b> | <b>Anomaly Detection (Phát hiện bất thường)</b>              | <b>66</b> |
| 12.1      | Giới thiệu về Anomaly Detection                              | 66        |
| 12.1.1    | Ví dụ minh họa: Động cơ máy bay                              | 66        |
| 12.1.2    | Phương pháp Density Estimation (Ước lượng mật độ)            | 67        |
| 12.2      | Các ứng dụng phổ biến                                        | 67        |
| 12.3      | Gaussian (Normal) Distribution                               | 68        |
| 12.3.1    | Hàm mật độ xác suất (PDF)                                    | 68        |
| 12.3.2    | Gaussian Distribution Example)                               | 68        |
| 12.3.3    | Ước lượng tham số (Parameter Estimation)                     | 69        |
| 12.4      | Thuật toán Anomaly Detection                                 | 70        |
| 12.5      | Anomaly Detection vs Supervised Learning                     | 70        |
| 12.6      | Xử lý đặc trưng (Feature Engineering)                        | 70        |
| 12.6.1    | Non-Gaussian Features                                        | 70        |
| 12.6.2    | Error Analysis                                               | 71        |
| 12.7      | Multivariate Gaussian Distribution (Phân phối chuẩn đa biến) | 72        |
| 12.7.1    | Hạn chế của mô hình gốc                                      | 72        |
| 12.7.2    | Mô hình đa biến                                              | 72        |
| 12.7.3    | So sánh mô hình                                              | 72        |

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>13 Recommender Systems (Hệ thống gợi ý)</b>                          | <b>73</b> |
| 13.1 Problem Definition (Định nghĩa bài toán)                           | 73        |
| 13.1.1 Ví dụ: Dự đoán đánh giá phim (Predicting movie ratings)          | 73        |
| 13.2 Content-based Recommendations (Gợi ý dựa trên nội dung)            | 74        |
| 13.2.1 Mô hình hóa                                                      | 74        |
| 13.2.2 Hàm tối ưu (Optimization Objective)                              | 74        |
| 13.3 Collaborative Filtering (Lọc cộng tác)                             | 74        |
| 13.3.1 Ý tưởng cốt lõi                                                  | 74        |
| 13.3.2 Hàm tối ưu đồng thời (Simultaneous Optimization)                 | 75        |
| 13.3.3 Thuật toán (Collaborative Filtering Algorithm)                   | 75        |
| 13.4 Low Rank Matrix Factorization (Phân rã ma trận hạng thấp)          | 75        |
| <b>14 Support Vector Machines (SVM)</b>                                 | <b>76</b> |
| 14.1 Optimization Objective (Mục tiêu tối ưu hóa)                       | 76        |
| 14.1.1 Từ Logistic Regression đến SVM                                   | 76        |
| 14.1.2 Hàm chi phí của SVM                                              | 77        |
| 14.2 Large Margin Intuition (Trực giác về Lề lớn)                       | 77        |
| 14.2.1 Điều kiện an toàn                                                | 77        |
| 14.2.2 Decision Boundary (Ranh giới quyết định)                         | 78        |
| 14.3 Mathematics Behind Large Margin Classification (Toán học đằng sau) | 78        |
| 14.3.1 Inner Product                                                    | 78        |
| 14.3.2 Tối ưu hóa Margin                                                | 78        |
| 14.4 Kernels (Hạt nhân)                                                 | 79        |
| 14.4.1 Ý tưởng Landmarks (Điểm mốc)                                     | 79        |
| 14.4.2 Gaussian Kernel (RBF Kernel)                                     | 79        |
| 14.4.3 Ảnh hưởng của $\sigma^2$                                         | 80        |
| 14.4.4 Feature Scaling cho SVM                                          | 80        |
| 14.5 SVM trong thực tế (Using an SVM)                                   | 81        |
| 14.5.1 Các bước thực hiện                                               | 81        |
| 14.5.2 So sánh Logistic Regression và SVM                               | 81        |
| <b>A Appendix: Summary &amp; Cheat Sheet (Phụ lục và Tóm tắt)</b>       | <b>82</b> |
| A.1 Bảng chọn lựa thuật toán (Algorithm Selection Guide)                | 82        |
| A.2 Bảng ký hiệu toán học (Mathematical Notation)                       | 82        |
| A.3 Các bước xây dựng hệ thống ML (Pipeline Checklist)                  | 82        |
| A.4 Thư viện Python tham khảo                                           | 83        |

# Lời nói đầu

Chào mừng bạn đọc đến với thế giới của **Machine Learning** (Học máy).

Trong kỷ nguyên số hóa hiện nay, Trí tuệ nhân tạo (AI) và Học máy không còn là những khái niệm viễn tưởng mà đã hiện hữu trong từng góc ngách của đời sống và công nghệ. Từ những hệ thống gợi ý phim ảnh, xe tự hành, đến các công cụ chẩn đoán y khoa, Machine Learning đang thay đổi cách chúng ta giải quyết các vấn đề phức tạp.

## Mục đích của cuốn sách

Cuốn sách này được mình biên soạn dựa trên các ghi chú và bài giảng nền tảng từ khóa học ML của Stanford, với mục tiêu trở thành **tấm bản đồ nhập môn** cho những ai mới bắt đầu bước chân vào lĩnh vực rộng lớn này. Chúng tôi tập trung hệ thống hóa các khái niệm cốt lõi, từ các thuật toán cơ bản như *Linear Regression*, *Logistic Regression* đến các kiến thức phức tạp hơn như *Neural Networks*, *Support Vector Machines (SVM)* hay *Dimensionality Reduction*.

Mục tiêu của mình không phải là bao quát toàn bộ tri thức nhân loại về AI, mà là giúp bạn xây dựng một **tư duy toán học vững chắc** và hiểu rõ **bản chất bên dưới** của các dòng lệnh.

## Phạm vi kiến thức và Lời khuyên

Điều quan trọng cần nhấn mạnh: **Những gì trình bày trong cuốn sách này chỉ là những kiến thức rất cơ bản.**

Lĩnh vực Machine Learning phát triển với tốc độ chóng mặt. Những thuật toán "state-of-the-art" (tiên tiến nhất) hôm nay có thể trở nên lỗi thời vào ngày mai. Do đó, cuốn sách này không thể thay thế cho quá trình tự rèn luyện liên tục. Để thực sự làm chủ công nghệ này, bạn đọc cần:

- Chủ động đào sâu:** Hãy coi các công thức và khái niệm ở đây là từ khóa. Bạn cần chủ động tìm đọc thêm các tài liệu chuyên sâu, các bài báo khoa học (papers) và sách giáo trình nâng cao để hiểu tường tận các biến thể và cải tiến của chúng.
- Thực hành không ngừng:** Lý thuyết chỉ là màu xám. Hãy bắt tay vào viết code (sử dụng Python, NumPy, PyTorch, TensorFlow...) để hiện thực hóa các thuật toán này. Chỉ khi tự tay gỡ lỗi (debug) một mô hình, bạn mới thực sự hiểu nó hoạt động ra sao.
- Tư duy phản biện:** Đừng chỉ áp dụng thuật toán một cách máy móc. Hãy luôn đặt câu hỏi: "*Tại sao lại dùng thuật toán này?*", "*Dữ liệu này có phù hợp không?*", "*Làm sao để cải thiện độ chính xác?*".

Mình hy vọng cuốn sách này sẽ là viên gạch đầu tiên vững chắc, giúp bạn tự tin xây dựng nên những công trình tri thức đồ sộ hơn trong tương lai.

Chúc các bạn kiên trì và tìm thấy niềm vui trong hành trình chinh phục tri thức.

**Minh Phạm Văn**  
*(Ngày 3 tháng 1 năm 2026)*

# Chapter 1

## Introduction to Machine Learning

### 1.1 Overview and Definitions

#### 1.1.1 Basic Concepts

Giới thiệu sơ lược về bối cảnh và ứng dụng của Machine Learning (ML).

- **Ứng dụng của ML:** Lọc email spam, database mining, và các ứng dụng không thể lập trình thủ công (Application can't program by hand).
- **Nguồn gốc:** ML phát triển từ AI (Artificial Intelligence). ML là một tập con của AI.
- **Cấu thành của AI:** AI bao gồm FS (Fuzzy Systems?), ML (Machine Learning), ES (Expert Systems).

#### 1.1.2 Arthur Samuel Definition (1959)

Định nghĩa cổ điển về Machine Learning:

"Machine Learning: Field of study that gives computers the ability to learn without being explicitly programmed."

Ghi chú:

- **Explicitly:** Lập trình tường minh.
- **Implicitly:** Lập trình không tường minh (ML hướng tới điều này).

### 1.2 Formal Definition (Tom Mitchell - 1998)

#### 1.2.1 Well-posed Learning Problem

Tom Mitchell đưa ra định nghĩa hiện đại và chặt chẽ hơn về ML:

"A computer program is said to learn from experience  $E$  with respect to some task  $T$  and some performance measure  $P$ , if its performance on  $T$ , as measured by  $P$ , improves with experience  $E$ ."

Quy trình lặp lại (Loop):

Experience ( $E$ )  $\rightarrow$  Task ( $T$ )  $\rightarrow$  Performance ( $P$ )  $\rightarrow E \dots$

( $E$  giải quyết task  $T$  đánh giá bởi  $P$  và cải thiện  $E \dots$ )

### 1.2.2 Example: Spam Email Filter

Phân tích các thành phần  $T, P, E$  trong bài toán lọc thư rác (Spam Filter):

- **Context:** Chương trình email theo dõi xem bạn đánh dấu email nào là spam hoặc không phải spam, và dựa vào đó học cách lọc spam tốt hơn.
- **Task ( $T$ ):** Classifying emails as spam or not spam (Phân loại email).
- **Experience ( $E$ ):** Watching you label emails as spam or not spam (Quan sát người dùng gán nhãn).
- **Performance ( $P$ ):** The number (or fraction) of emails correctly classified as spam/not spam (Tỷ lệ phân loại đúng).

## 1.3 Classification of Machine Learning Algorithms

### 1.3.1 Main Branches

Dựa vào cách học và dữ liệu đầu vào, ML được chia thành các nhánh chính sau:

1. **Supervised Learning (SL) - Học có giám sát**
2. **Unsupervised Learning (USL) - Học không giám sát**
3. **Reinforcement Learning (RL) - Học tăng cường**

Ngoài ra còn có Recommender Systems, Random Forest (RF), và các thuật toán lai (hybrid) ngày nay.

### 1.3.2 Deep Dive: Supervised Learning (SL)

Trong Supervised Learning, chúng ta chia nhỏ thành:

- **Regression (Hồi quy):** Dùng cho dữ liệu đầu ra liên tục (*continuous*).
- **Classification (Phân loại):** Dùng cho dữ liệu đầu ra rời rạc (*discrete*).

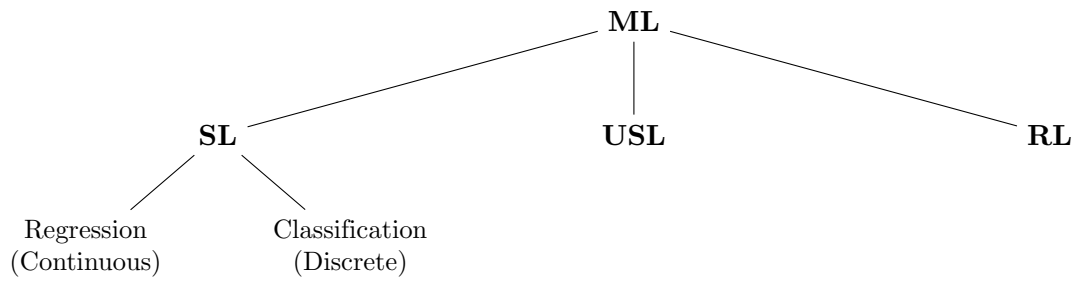


Figure 1.1: Sơ đồ phân loại các thuật toán Machine Learning

## 1.4 Supervised Learning Example: Housing Price Prediction

### 1.4.1 Problem Description

Ví dụ về dự đoán giá nhà dựa trên diện tích (Size in feet).

- **Trục tung (Y):** Price (\$ in 1000's) - Giá trị liên tục.
- **Trục hoành (X):** Size in feet (500, 1000, 1500, ...).
- **Dataset:** Các điểm đánh dấu X (crosses) trên biểu đồ là thông tin thu thập được từ thực tế.

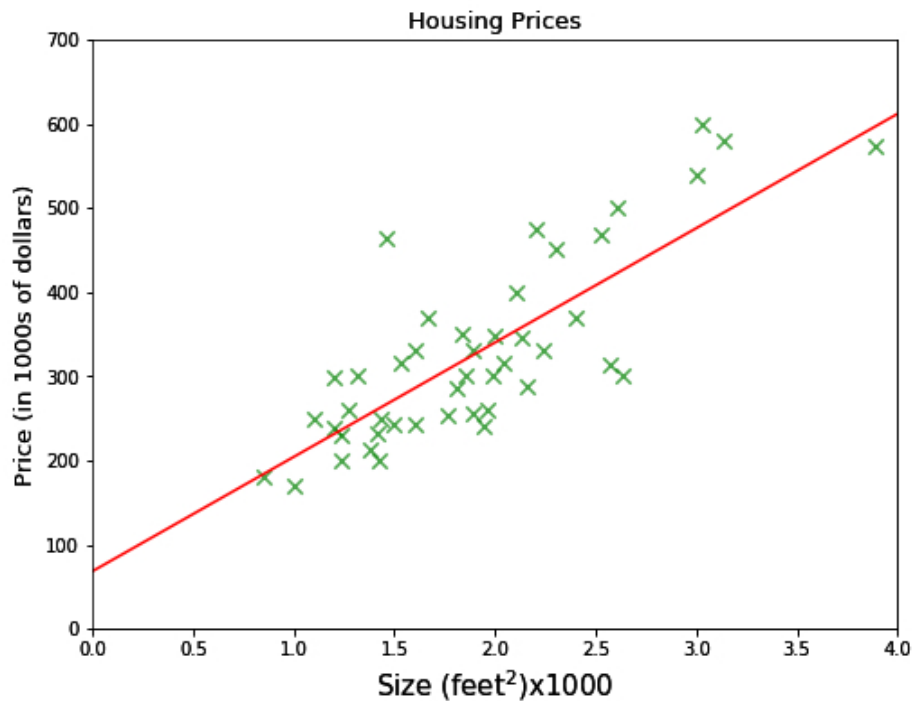


Figure 1.2: Housing Price Prediction Plot (Regression Problem)

## 1.4.2 Characteristics of Supervised Learning

Dựa trên ví dụ này, ta rút ra đặc điểm của SL:

- Với **Supervised Learning**, các đáp án đúng (output/label) đã có sẵn trong dataset.
- Mục tiêu là tìm ra mối quan hệ (đường cong hoặc đường thẳng) khớp với các điểm dữ liệu này để dự đoán giá trị mới (ví dụ: nhà 750 feet có giá bao nhiêu?).

## 1.5 Unsupervised Learning (USL)

### 1.5.1 Characteristics

Khác với SL, Unsupervised Learning (USP/USL):

- Chỉ dựa vào các đặc trưng (features) và phân phối của các điểm dữ liệu đầu vào.
- **Cơ chế:** Learn from input data  $\rightarrow$  find hidden patterns  $\rightarrow$  output.
- Không có nhãn (labels) hay đáp án đúng trước.

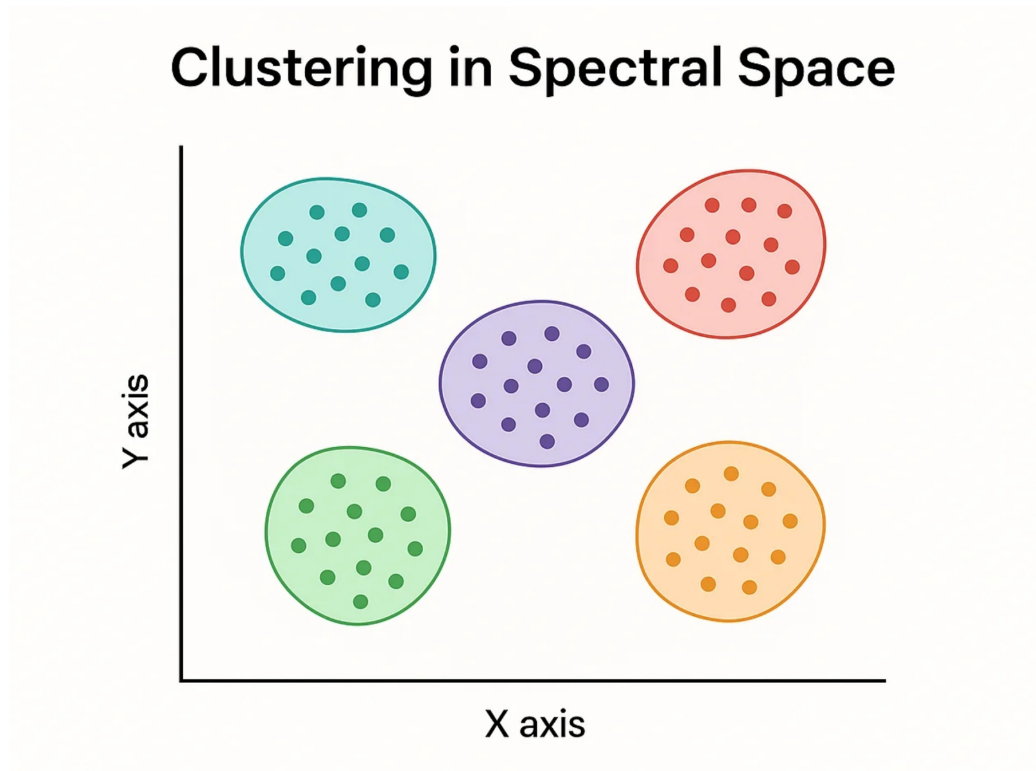


Figure 1.3: Minh họa Unsupervised Learning (Phân cụm dữ liệu)



## Chapter 2

# Linear Regression

### 2.1 Problem Definition

#### 2.1.1 Data and Goal

Bài toán Linear Regression (Hồi quy tuyến tính) bắt đầu với dữ liệu đầu vào (Features) và giá trị cần dự đoán (Label).

- **Input (Features):** Dữ liệu đầu vào (ví dụ: Diện tích nhà - Area).
- **Output (Label):** Giá trị thực tế (ví dụ: Giá nhà - Price).
- **Goal:** Tìm hàm  $f(x)$  sao cho "fit" (khớp) nhất với dữ liệu đã cho.

| Features (Area) | Label (Price) |
|-----------------|---------------|
| 6.7             | 9.1           |
| 4.6             | 5.9           |
| 3.5             | 4.6           |
| 5.5             | 6.7           |

Table 2.1: Bảng dữ liệu ví dụ (Area vs Price)

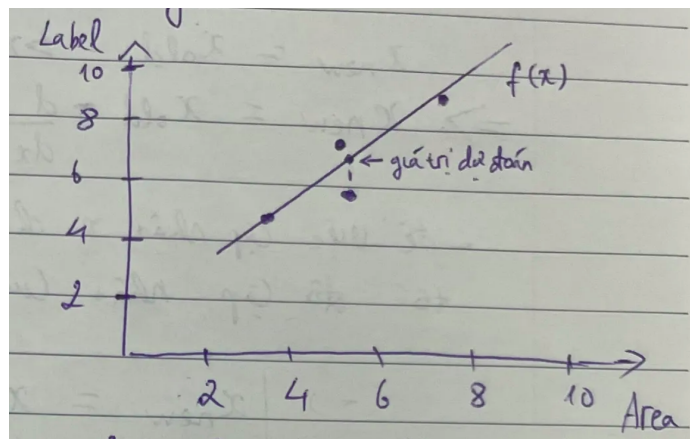


Figure 2.1: Minh họa mô hình dự đoán

### 2.1.2 Model Representation

Hàm tuyến tính cần tìm có dạng:

$$\hat{y} = f(x) = ax + b$$

Trong đó:

- $\hat{y}$ : Giá trị dự đoán.
- $a, b$ : Các tham số cần tìm dựa trên dữ liệu.

## 2.2 Loss Function (Cost Function)

### 2.2.1 Distance Metrics

Để đánh giá độ sai lệch giữa giá trị dự đoán ( $\hat{y}$ ) và giá trị thực ( $y$ ), ta dùng hàm khoảng cách (Distance):

1. Mean Absolute Error (MAE):  $|\hat{y} - y|$
2. Mean Squared Error (MSE):  $(\hat{y} - y)^2$

**Lựa chọn:** Người ta thường chọn cách (2)  $(\hat{y} - y)^2$  vì việc tính đạo hàm dễ dàng hơn phục vụ cho tối ưu hóa.

### 2.2.2 Objective

Mục tiêu của bài toán là tìm cực tiểu (minimum) của hàm khoảng cách:

$$L = (\hat{y} - y)^2 \rightarrow \min$$

## 2.3 Optimization Derivatives

### 2.3.1 Concept of Derivative

Để tìm cực tiểu, ta sử dụng đạo hàm. Đạo hàm thể hiện sự biến thiên của hàm số. Tại điểm cực trị (cực tiểu hoặc cực đại), đạo hàm bằng 0.

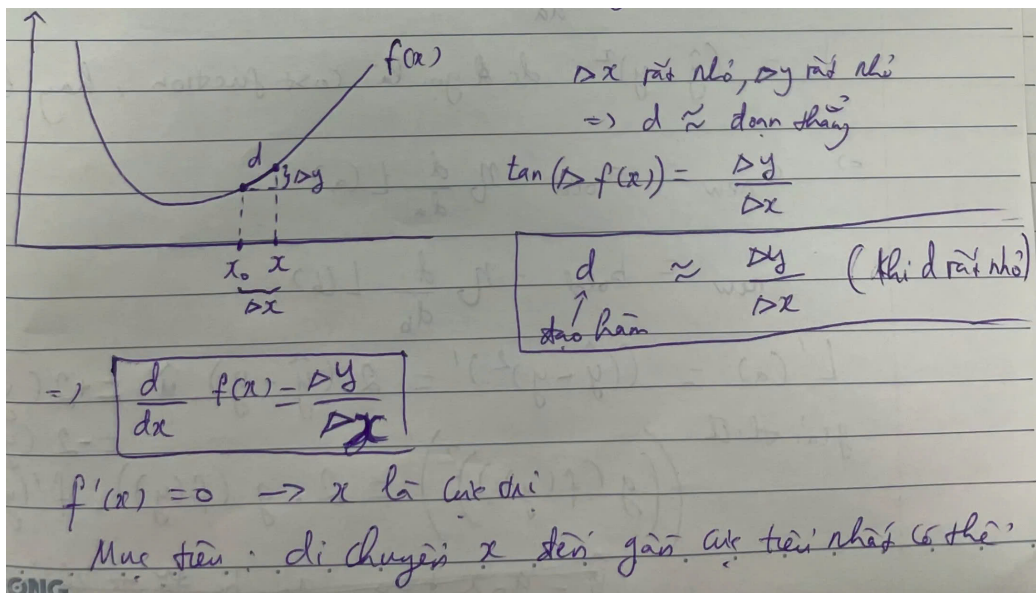


Figure 2.2: Minh họa tìm cực tiểu bằng đạo hàm

$$\frac{dy}{dx} = f'(x) = 0 \rightarrow \text{Điểm cực trị}$$

Mục tiêu là di chuyển các tham số sao cho giá trị hàm số đi dần về điểm cực tiểu.

## 2.4 Gradient Descent

### 2.4.1 Update Rule

Để cập nhật tham số nhằm giảm thiểu sai số, ta sử dụng thuật toán Gradient Descent. Quy tắc cập nhật tổng quát:

$$x_{new} = x_{old} - \eta \cdot \frac{d}{dx} f(x)$$

Trong đó  $\eta$  (eta) là **Learning rate** (tốc độ học). Cần điều chỉnh  $\eta$  phù hợp để việc cập nhật chính xác hơn.

### 2.4.2 Applying to Linear Regression

Áp dụng cho các tham số  $a$  và  $b$  của hàm  $f(x) = ax + b$  với hàm mất mát  $L = (\hat{y} - y)^2$ :

$$a_{new} = a_{old} - \eta \frac{dL}{da} \quad (2.1)$$

$$b_{new} = b_{old} - \eta \frac{dL}{db} \quad (2.2)$$

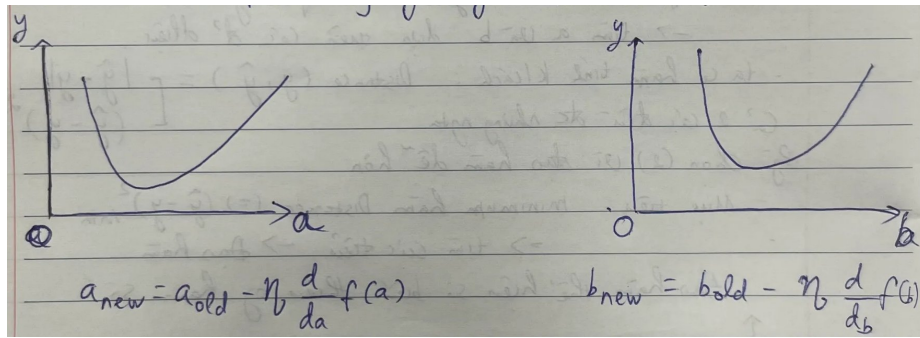


Figure 2.3: Minh họa cập nhật trọng số cho  $a$  và  $b$

## 2.5 Chain Rule Backpropagation

### 2.5.1 Computing Derivatives

Sử dụng Chain Rule (quy tắc chuỗi) để tính đạo hàm của hàm  $L$  theo  $a$  và  $b$ .  
Sơ đồ tính toán:

$$x, a, b \xrightarrow{\text{hàm } f} \hat{y} \xrightarrow{\text{hàm } L} \text{Loss}$$

**Đạo hàm chi tiết:** Ta có  $L = (\hat{y} - y)^2$ . Đạo hàm của  $L$  theo  $\hat{y}$ :

$$\frac{dL}{d\hat{y}} = 2(\hat{y} - y)$$

1. Đạo hàm theo  $a$ :

$$\frac{dL}{da} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{da} = 2(\hat{y} - y) \cdot x$$

2. Đạo hàm theo  $b$ :

$$\frac{dL}{db} = \frac{dL}{d\hat{y}} \cdot \frac{d\hat{y}}{db} = 2(\hat{y} - y) \cdot 1 = 2(\hat{y} - y)$$

**Kết luận:**

- $a$  và  $b$  là các tham số cần cập nhật.
- $x$  và  $y$  là dữ liệu thực tế (hằng số trong quá trình đạo hàm).
- $(\hat{y} - y)$  là thành phần đạo hàm theo  $y$  (gradient).

## Chapter 3

# Gradient Descent Derivation

### 3.1 Giới thiệu và Hàm Giả Thuyết (Hypothesis Function)

Trong bài toán Hồi quy tuyến tính (Linear Regression), chúng ta khởi đầu với một hàm giả thuyết tuyến tính (Linear Hypothesis Function). Đây là hàm số thực hiện việc ánh xạ dữ liệu đầu vào sang đầu ra dự đoán.

Hàm giả thuyết được định nghĩa như sau:

$$h(x) = \theta_0 + \theta_1 x \quad (3.1)$$

**Lưu ý thuật ngữ:**

- *Linear*: Mô hình dùng cho dữ liệu có tính chất biến đổi tuyến tính.
- *Regression*: Âm chỉ bài toán có đầu ra là các giá trị liên tục (continuous).

Mục tiêu cốt lõi là tìm kiếm các giá trị của tham số  $\theta_0$  và  $\theta_1$  sao cho hàm giả thuyết khớp ("fit") tốt nhất với tập dữ liệu huấn luyện (training set).

#### 3.1.1 Tập dữ liệu huấn luyện (Training Set)

Để huấn luyện mô hình, ta cần tập dữ liệu bao gồm các cặp  $(x, y)$ , trong đó  $x$  là đầu vào và  $y$  là đầu ra thực tế.

Ký hiệu cho mẫu huấn luyện thứ  $i$  là  $(x^{(i)}, y^{(i)})$ .

**Cảnh báo quan trọng:** Ký hiệu  $(i)$  ở đây chỉ là chỉ số thứ tự của mẫu huấn luyện, tuyệt đối không được nhầm lẫn là số mũ.

### 3.2 Hàm Mất Mát (MSE Cost Function)

Để đánh giá độ lệch giữa giá trị dự đoán bởi hàm giả thuyết và giá trị thực tế, ta sử dụng Hàm chi phí (Cost Function), cụ thể là Sai số Bình phương Trung bình (Mean Squared Error - MSE).

Công thức của hàm chi phí  $J(\theta)$ :

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \quad (3.2)$$

**Giải thích các biến số:**

- $m$ : Số lượng mẫu dữ liệu huấn luyện.
- $x^{(i)}$ : Vector đầu vào của mẫu thứ  $i$ .
- $y^{(i)}$ : Nhãn lớp (class label) của mẫu thứ  $i$ .
- $\theta$ : Các tham số trọng số (weights)  $(\theta_0, \theta_1, \dots)$ .
- $h_\theta(x^{(i)})$ : Giá trị dự đoán của thuật toán cho mẫu thứ  $i$ .

Nhiệm vụ trong hầu hết các bài toán học máy là tìm  $\theta$  để cực tiểu hóa  $J(\theta)$ , biểu diễn toán học là  $\min_\theta J(\theta)$ .

### 3.3 Thuật toán Gradient Descent

Làm thế nào để chọn đúng giá trị  $\theta$  giúp giảm thiểu hàm mất mát? Nếu chọn ngẫu nhiên, ta không thể biết khi nào mới tìm được giá trị tối ưu. Giải pháp là sử dụng thuật toán **Gradient Descent**.

#### 3.3.1 Khái niệm và Quy tắc cập nhật

Xét trường hợp đơn giản với hàm chi phí  $J(\theta) = \theta^2$ .

- Tại  $\theta = 0$ ,  $J(\theta)$  đạt cực tiểu.
- Giả sử khởi tạo  $\theta = 3$ . Làm sao thuật toán biết cần di chuyển  $\theta$  về hướng nào và bao nhiêu?

Gradient Descent là thuật toán lặp. Tại mỗi bước, ta áp dụng quy tắc cập nhật (với  $:=$  là phép gán):

$$\theta := \theta - \alpha \frac{d}{d\theta} J(\theta) \quad (3.3)$$

Trong đó  $\alpha$  là *learning rate* (tốc độ học). Ví dụ, gán  $\alpha = 0.1$ . Đạo hàm của  $J(\theta) = \theta^2$  là  $2\theta$ .

#### 3.3.2 Mô phỏng quá trình hội tụ

Hình dưới đây minh họa hàm  $J(\theta)$  và giá trị của  $\theta$  qua 10 vòng lặp. Các dấu "x" thể hiện quá trình tham số di chuyển dần về đáy của parabol (điểm cực tiểu).

Dưới đây là bảng số liệu chi tiết cho thấy sai số giảm dần về 0:

| Iteration | $\theta$ (current) | Gradient $\frac{d}{d\theta} J(\theta)$ | Update Amount ( $\alpha \times 2\theta$ ) |
|-----------|--------------------|----------------------------------------|-------------------------------------------|
| 0         | 3                  | 6                                      | 0.6                                       |
| 1         | 2.4                | 4.8                                    | 0.48                                      |
| 2         | 1.92               | 3.84                                   | 0.384                                     |
| 3         | 1.536              | 3.072                                  | 0.307                                     |
| 4         | 1.229              | 2.458                                  | 0.246                                     |
| 5         | 0.983              | 1.966                                  | 0.197                                     |
| 6         | 0.786              | 1.572                                  | 0.157                                     |
| 7         | 0.629              | 1.258                                  | 0.126                                     |
| 8         | 0.503              | 1.006                                  | 0.101                                     |
| 9         | 0.403              | 0.806                                  | 0.081                                     |

Below is a plot of our function,  $J(\theta)$ , and the value of  $\theta$  over ten iterations of gradient descent.

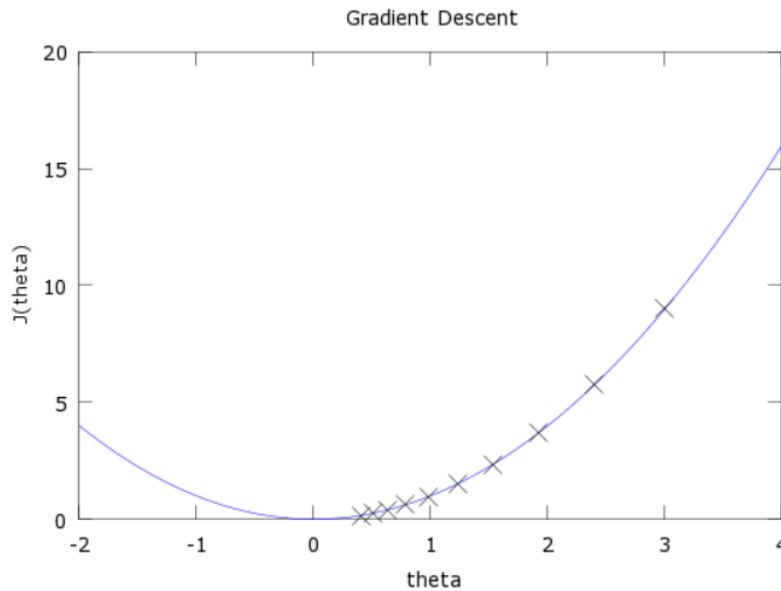


Figure 3.1: Minh họa các bước cập nhật

### 3.4 Phân tích Trực quan (Intuition)

Tại sao quy tắc cập nhật theo Gradient lại đảm bảo hàm mất mát chạy về cực tiểu?

#### 3.4.1 Độ lớn và Dấu của Đạo hàm

Đạo hàm biểu thị độ dốc (slope):

- **Độ lớn:** Đạo hàm lớn nghĩa là dốc đứng  $\rightarrow$  bước di chuyển lớn. Đạo hàm nhỏ nghĩa là hàm phẳng  $\rightarrow$  bước di chuyển nhỏ.
- **Dấu (Hướng):**
  - Đạo hàm dương: Hàm đang tăng, cần giảm  $\theta$  (đi ngược chiều dương).
  - Đạo hàm âm: Hàm đang giảm, cần tăng  $\theta$  (đi ngược chiều âm).

$\rightarrow$  **Nguyên tắc:** Luôn di chuyển  $\theta$  ngược dấu với Gradient.

#### 3.4.2 Vai trò của Learning Rate ( $\alpha$ )

Nếu chỉ cập nhật bằng giá trị Gradient, bước nhảy có thể quá lớn gây dao động. Ta dùng  $\alpha$  (hyperparameter) nhân với gradient để điều chỉnh tốc độ.

- Gradient lớn  $\rightarrow$  nên dùng  $\alpha$  nhỏ để hãm lại.
- Có thể tinh chỉnh  $\alpha$  qua từng iteration.

### 3.4.3 Tiêu chuẩn dừng

1. Chọn số vòng lặp cố định (10, 20, 100...).
2. So sánh  $\theta$  giữa các epoch: Nếu thay đổi không đáng kể thì dừng.
3. Có nhiều phương pháp dừng phức tạp khác.

## 3.5 Đạo hàm chi tiết cho Hồi quy Tuyến tính

Với hàm giả thuyết  $h(x) = \theta_0 + \theta_1 x$ , ta cần cập nhật từng tham số bằng đạo hàm riêng:

### 3.5.1 Các bước chứng minh (Derivation Steps)

Xét hàm mục tiêu:

$$J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

**Bước 1 (Tính tuyến tính):** Đạo hàm của tổng bằng tổng các đạo hàm.

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^m \frac{\partial}{\partial \theta_0} (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

**Bước 2 (Chain Rule):** Áp dụng quy tắc chuỗi  $\frac{d}{dx} f(g(x)) = f'(g(x))g'(x)$ .

$$= \frac{1}{m} \sum_{i=1}^m 2(h_{\theta}(x^{(i)}) - y^{(i)}) \cdot \frac{\partial}{\partial \theta_0} (h_{\theta}(x^{(i)}) - y^{(i)})$$

**Bước 3 (Đạo hàm hàm hợp):** Vì  $h_{\theta}(x^{(i)}) = \theta_0 + \theta_1 x^{(i)}$ , đạo hàm riêng theo  $\theta_0$  là 1.

$$\frac{\partial}{\partial \theta_0} (\dots) = 1$$

Kết quả cho  $\theta_0$ :

$$\frac{\partial J}{\partial \theta_0} = \frac{2}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \quad (3.4)$$

Tương tự với  $\theta_1$ , nhưng đạo hàm riêng của hàm hợp theo  $\theta_1$  là  $x^{(i)}$ :

$$\frac{\partial J}{\partial \theta_1} = \frac{2}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x^{(i)} \quad (3.5)$$



## Chapter 4

# Logistic Regression (Classification)

### 4.1 Giới thiệu về Bài toán Phân loại (Classification)

Trong Machine Learning, bài toán phân loại (Classification) là bài toán dự đoán đầu ra rời rạc (discrete values). Các ví dụ phổ biến bao gồm:

- **Email:** Spam (Thư rác) hay Not Spam (Không phải thư rác)?
- **Giao dịch trực tuyến:** Lừa đảo (Fraudulent - Yes) hay Không (No)?
- **Khối u (Tumor):** Ác tính (Malignant) hay Lành tính (Benign)?

Trong bài toán Phân loại Nhị phân (Binary Classification), ta quy ước:

- $y \in \{0, 1\}$
- **0: "Negative Class"** (ví dụ: khối u lành tính - benign tumor).
- **1: "Positive Class"** (ví dụ: khối u ác tính - malignant tumor).

#### 4.1.1 Tại sao không dùng Linear Regression?

Xét ví dụ dự đoán khối u dựa trên kích thước (Tumor Size). Nếu ta áp dụng Linear Regression ( $h_{\theta}(x) = \theta^T x$ ) và chọn ngưỡng (threshold) là 0.5:

- Nếu  $h_{\theta}(x) \geq 0.5 \rightarrow$  dự đoán  $y = 1$ .
- Nếu  $h_{\theta}(x) < 0.5 \rightarrow$  dự đoán  $y = 0$ .

**Vấn đề (Outlier Issue):**

- Ban đầu, Linear Regression có thể hoạt động ổn.
- Tuy nhiên, nếu xuất hiện một điểm dữ liệu có kích thước khối u rất lớn (cực phải trên đồ thị - dữ liệu chuẩn, không phải nhiễu). Đường hồi quy sẽ bị "kéo" nghiêng về phía dữ liệu đó để giảm thiểu sai số.
- Kết quả: Ngưỡng 0.5 bị dịch chuyển, khiến các điểm dữ liệu bên trái (đang được phân loại đúng) bị phân loại sai thành lành tính.
- **Kết luận:** Linear Regression không phù hợp vì nó nhạy cảm với dữ liệu ngoại lai và giá trị dự đoán có thể vượt ra ngoài khoảng  $[0, 1]$ .

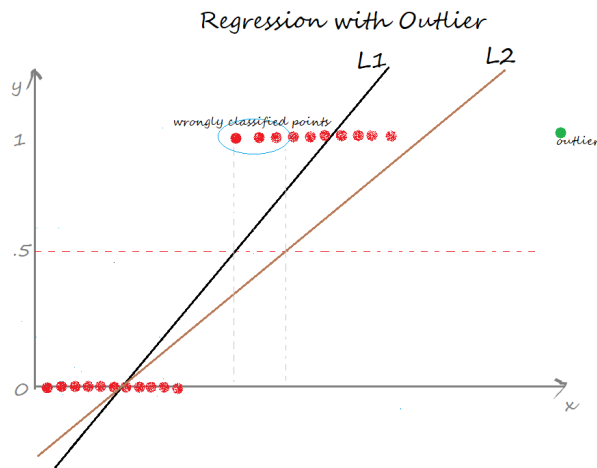


Figure 4.1: Minh họa outlier với Linear Regression

## 4.2 Mô hình Logistic Regression

Để giải quyết vấn đề trên, ta cần một hàm giả thuyết (hypothesis) luôn trả về giá trị trong khoảng  $0 \leq h_{\theta}(x) \leq 1$ . Ta sử dụng **Hàm Sigmoid** (hay Logistic function).

### 4.2.1 Hàm Sigmoid

Công thức hàm giả thuyết:

$$h_{\theta}(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}} \quad (4.1)$$

Trong đó  $g(z)$  là hàm Sigmoid:

$$g(z) = \frac{1}{1 + e^{-z}} \quad (4.2)$$

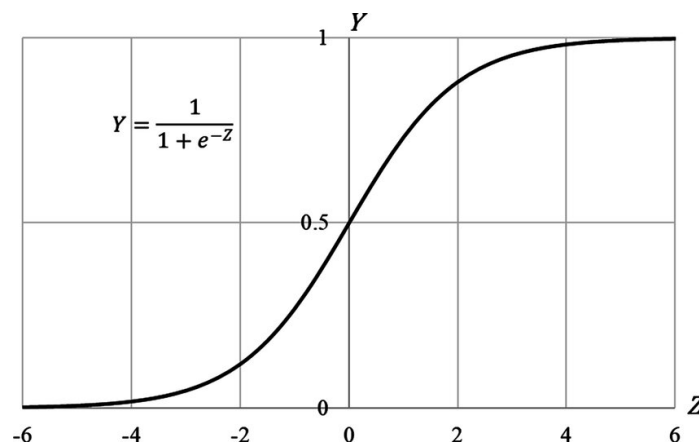


Figure 4.2: Đồ thị hàm sigmoid.

**Đặc điểm của hàm Sigmoid:**

- Biến đổi giá trị liên tục vô hạn ( $z \in [-\infty, +\infty]$ ) về khoảng  $[0, 1]$ .

- Đồ thị hình chữ S (S-shaped), rất mượt ("smooth").
- Đạo hàm dễ tính toán:  $g'(z) = g(z)(1 - g(z))$ .
- Được sử dụng như một ràng buộc (constraints) để quy về bài toán xác suất.

#### 4.2.2 Ý nghĩa của đầu ra (Interpretation)

Giá trị  $h_\theta(x)$  được hiểu là xác suất ước lượng để  $y = 1$  với đầu vào  $x$ :

$$h_\theta(x) = P(y = 1|x; \theta) \quad (4.3)$$

**Ví dụ:** Nếu  $x$  là kích thước khối u và  $h_\theta(x) = 0.7$ , điều này có nghĩa là "Bệnh nhân có 70% nguy cơ khối u là ác tính".

Vì đây là bài toán nhị phân (tổng xác suất bằng 1), ta có:

$$P(y = 0|x; \theta) = 1 - P(y = 1|x; \theta) = 1 - h_\theta(x) \quad (4.4)$$

### 4.3 Ranh giới Quyết định (Decision Boundary)

Mặc dù hàm Sigmoid trả về xác suất, ta cần một ngưỡng để đưa ra quyết định cuối cùng ( $y = 1$  hoặc  $y = 0$ ). Thông thường ta chọn ngưỡng 0.5:

- Dự đoán  $y = 1$  nếu  $h_\theta(x) \geq 0.5 \Rightarrow \theta^T x \geq 0$  (vì  $g(z) \geq 0.5$  khi  $z \geq 0$ ).
- Dự đoán  $y = 0$  nếu  $h_\theta(x) < 0.5 \Rightarrow \theta^T x < 0$ .

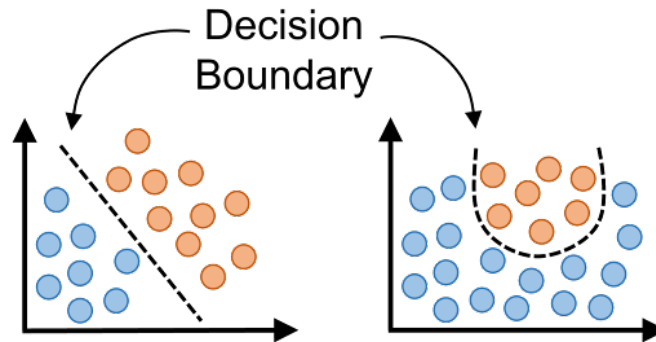


Figure 4.3: Minh họa decision boundary.

#### 4.3.1 Ví dụ: Linear Decision Boundary

Xét hàm giả thuyết với tham số  $\theta = [-3, 1, 1]^T$ :

$$h_\theta(x) = g(-3 + x_1 + x_2)$$

Ta dự đoán  $y = 1$  khi:

$$-3 + x_1 + x_2 \geq 0 \Rightarrow x_1 + x_2 \geq 3$$

Đường thẳng  $x_1 + x_2 = 3$  chính là **Decision Boundary**.

- Phần bên phải/trên đường thẳng ( $x_1 + x_2 \geq 3$ ) là vùng dự đoán  $y = 1$ .
- Phần bên trái/dưới đường thẳng ( $x_1 + x_2 < 3$ ) là vùng dự đoán  $y = 0$ .

*Lưu ý:* Trong thực tế, các tham số  $\theta$  được học thông qua thuật toán (như Gradient Descent) chứ không phải chọn cố định.

#### 4.3.2 Ví dụ: Non-linear Decision Boundary

Để phân loại dữ liệu phức tạp hơn (ví dụ phân bố hình tròn), ta có thể thêm các đặc trưng bậc cao (polynomial features). Xét giả thuyết:

$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2)$$

Giả sử ta học được  $\theta = [-1, 0, 0, 1, 1]^T$ , khi đó điều kiện dự đoán  $y = 1$  là:

$$-1 + x_1^2 + x_2^2 \geq 0 \Rightarrow x_1^2 + x_2^2 \geq 1$$

Đây là phương trình đường tròn bán kính 1.

- Bên ngoài đường tròn:  $y = 1$ .
- Bên trong đường tròn:  $y = 0$ .
- **Bản chất:** Tuy biểu thức  $\theta^T x$  là hàm tuyến tính, nhưng  $g$  là hàm phi tuyến  $\rightarrow$  biến nó thành hàm phi tuyến  $\rightarrow h$  là hàm phi tuyến.

### 4.4 Hàm Mất Mát (Cost Function)

Ta có tập dữ liệu huấn luyện  $m$  mẫu:  $\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$ . Làm thế nào để chọn tham số  $\theta$ ?

#### 4.4.1 Tại sao không dùng MSE?

Nếu dùng hàm Mean Squared Error (như Linear Regression):

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

( $1/2$  ở đây chỉ là để đạo hàm triệt tiêu, không có cũng chẳng sao)

Do  $h_{\theta}(x)$  là hàm Sigmoid (phi tuyến), hàm  $J(\theta)$  sẽ trở thành **Non-convex** (không lồi).

- **Non-convex:** Có nhiều cực trị địa phương (local optima), khiến thuật toán Gradient Descent khó tìm được cực trị toàn cục (global minimum).
- **Convex:** Hình cái bát, chỉ có một cực trị toàn cục duy nhất.

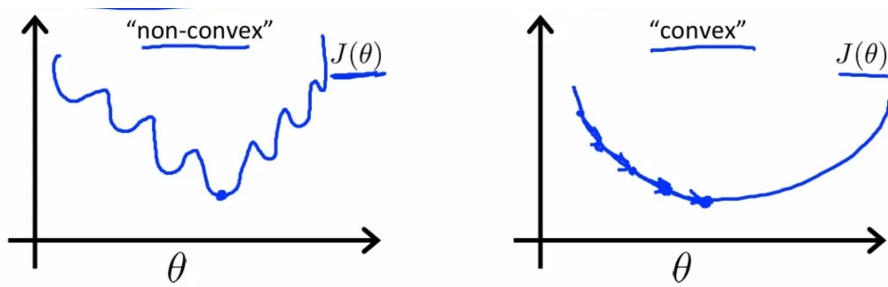


Figure 4.4: Minh họa hàm Convex/Non-convex.

#### 4.4.2 Logistic Regression Cost Function (Log Loss)

Ta xây dựng hàm Cost function mới phù hợp hơn:

$$Cost(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases} \quad (4.5)$$

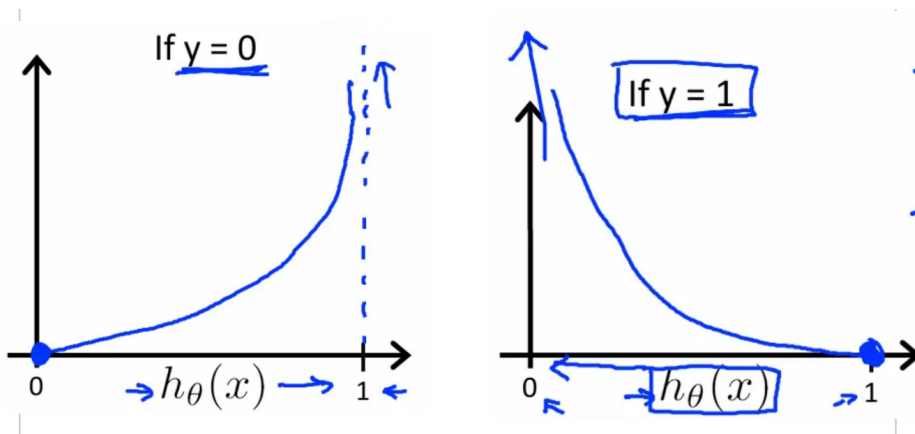


Figure 4.5: Minh họa Log Loss bằng đồ thị.

**Phân tích trực quan:**

- **Trường hợp  $y = 1$ :**
  - Nếu dự đoán  $h_{\theta}(x) \approx 1$  (đúng)  $\rightarrow Cost \approx 0$ .
  - Nếu dự đoán  $h_{\theta}(x) \rightarrow 0$  (sai)  $\rightarrow Cost \rightarrow \infty$  (Phạt cực nặng lỗi sai).
- **Trường hợp  $y = 0$ :**
  - Nếu dự đoán  $h_{\theta}(x) \approx 0$  (đúng)  $\rightarrow Cost \approx 0$ .
  - Nếu dự đoán  $h_{\theta}(x) \rightarrow 1$  (sai)  $\rightarrow Cost \rightarrow \infty$ .

#### 4.4.3 Công thức thu gọn (Simplified Cost Function)

Vì  $y$  chỉ có thể là 0 hoặc 1, ta có thể gộp 2 trường hợp trên thành một công thức duy nhất (Binary Cross Entropy):

$$Cost(h_{\theta}(x), y) = -y \log(h_{\theta}(x)) - (1 - y) \log(1 - h_{\theta}(x)) \quad (4.6)$$

Hàm mất mát toàn cục trên  $m$  mẫu dữ liệu:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[ y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] \quad (4.7)$$

Mục tiêu: Tìm  $\min_{\theta} J(\theta)$ .

## 4.5 Gradient Descent cho Logistic Regression

Để cực tiểu hóa  $J(\theta)$ , ta sử dụng thuật toán Gradient Descent. Quy tắc cập nhật (Update rule):

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \quad (4.8)$$

Sau khi tính đạo hàm riêng (có thể tự chứng minh bằng tay), ta thu được công thức cập nhật:

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} \quad (4.9)$$

(Cập nhật đồng thời cho tất cả các  $j$ ).

**Lưu ý:**

- Công thức này nhìn *giống hệt* Linear Regression.
- Tuy nhiên, sự khác biệt nằm ở  $h_{\theta}(x)$ :
  - Linear Regression:  $h_{\theta}(x) = \theta^T x$
  - Logistic Regression:  $h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$
- $\alpha$  là learning rate (tốc độ học).

## 4.6 Phân loại Đa lớp (Multiclass Classification)

Trong thực tế, ta thường gặp bài toán có nhiều hơn 2 nhãn ( $y \in \{0, 1, \dots, n\}$ ). Ví dụ:

- Phân loại Email: Công việc, Bạn bè, Gia đình, Sở thích.
- Thời tiết: Nắng, Mưa, Mây, Tuyết.
- Chẩn đoán y tế: Không bệnh, Cảm lạnh, Cúm.

### 4.6.1 Phương pháp One-vs-All (One-vs-Rest)

Ý tưởng: Biến bài toán đa lớp thành nhiều bài toán nhị phân.

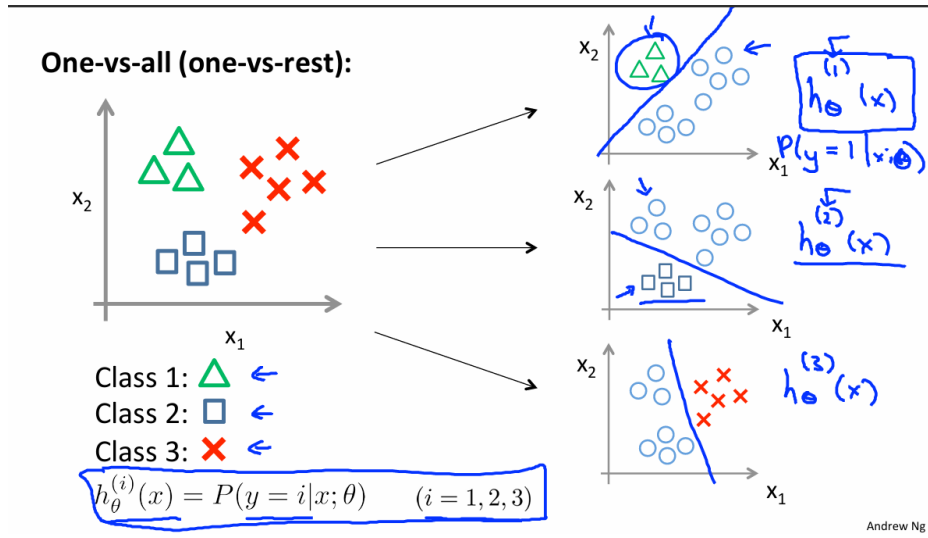


Figure 4.6: Minh họa multi-class classification.

**Quy trình:**

1. Với mỗi lớp  $i$ , ta huấn luyện một mô hình Logistic Regression  $h_{\theta}^{(i)}(x)$  để dự đoán xác suất  $y = i$  (coi lớp  $i$  là Positive, tất cả các lớp còn lại là Negative).
2. Ta sẽ có  $n$  hàm giả thuyết:

$$h_{\theta}^{(1)}(x) = P(y = 1|x; \theta), \dots, h_{\theta}^{(n)}(x) = P(y = n|x; \theta)$$

3. **Dự đoán (Prediction):** Với một đầu vào  $x$  mới, ta chọn lớp  $i$  có giá trị xác suất cao nhất:

$$\max_i h_{\theta}^{(i)}(x) \quad (4.10)$$

## Chapter 5

# Regularization (Điều chuẩn)

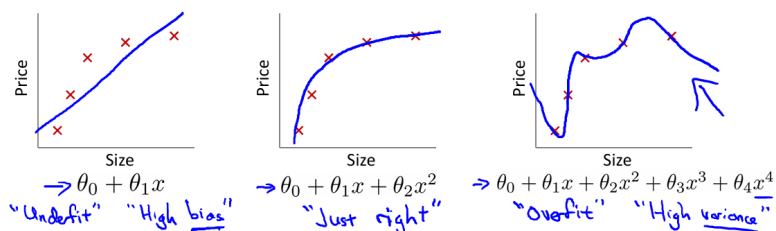
### 5.1 Vấn đề Overfitting và Underfitting

Trong các bài toán học máy (ví dụ: Linear Regression dự đoán giá nhà), việc lựa chọn các đặc trưng (features) và độ phức tạp của mô hình là rất quan trọng để đảm bảo độ chính xác.

#### 5.1.1 Định nghĩa các trạng thái của mô hình

Xét ví dụ dự đoán giá nhà dựa trên kích thước ( $x$ ). Ta có ba trường hợp điển hình:

Example: Linear regression (housing prices)



**Overfitting:** If we have too many features, the learned hypothesis may fit the training set very well ( $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \approx 0$ ), but fail to generalize to new examples (predict prices on new examples).

Figure 5.1: Minh họa mối liên hệ giữa độ phức tạp và chất lượng mô hình.

- **Underfitting (High Bias - Độ lệch cao):** Mô hình quá đơn giản, không thể nắm bắt được cấu trúc của dữ liệu.
- **Just Right (Vừa vặn):** Mô hình phù hợp với xu hướng chung của dữ liệu.
- **Overfitting (High Variance - Phương sai cao):** Mô hình quá phức tạp, cố gắng đi qua tất cả các điểm dữ liệu (bao gồm cả nhiễu).

**Hệ quả của Overfitting:** Nếu chúng ta có quá nhiều đặc trưng (features), giả thuyết học được có thể khớp rất tốt với tập huấn luyện ( $J(\theta) \approx 0$ ) nhưng lại thất bại trong việc tổng



quát hóa (generalize) với các dữ liệu mới (dữ liệu kiểm tra - test set). Nếu xây dựng model quá phức tạp, nó sẽ "học vẹt" cả các điểm nhiễu.

Tương tự, hiện tượng này cũng xảy ra với Logistic Regression khi ranh giới quyết định (decision boundary) quá phức tạp để bao quanh chính xác tuyệt đối các điểm dữ liệu huấn luyện.

## 5.2 Giải pháp khắc phục Overfitting

Có hai phương pháp chính để giải quyết vấn đề này:

### 1. Giảm số lượng đặc trưng (Reduce number of features):

- Chọn lọc thủ công các đặc trưng quan trọng cần giữ lại.
- Sử dụng thuật toán lựa chọn mô hình (Model selection algorithm).

### 2. Regularization (Điều chuẩn):

- Giữ lại tất cả các đặc trưng, nhưng giảm độ lớn (magnitude) hoặc làm triệt tiêu ảnh hưởng của các tham số  $\theta_j$ .
- Phương pháp này hoạt động hiệu quả khi chúng ta có nhiều đặc trưng và mỗi đặc trưng đóng góp một phần nhỏ vào việc dự đoán  $y$ .

## 5.3 Hàm Chi phí (Cost Function) có Regularization

### 5.3.1 Trực giác (Intuition)

Giả sử ta có hàm giả thuyết bậc 4:  $\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$ . Để tránh overfitting, ta muốn làm giảm ảnh hưởng của các thành phần bậc cao ( $\theta_3, \theta_4$ ).

Ta thay đổi hàm mất mát bằng cách cộng thêm một lượng phạt (penalize) cực lớn cho  $\theta_3$  và  $\theta_4$ :

$$\min_{\theta} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + 1000\theta_3^2 + 1000\theta_4^2 \quad (5.1)$$

Việc đặt hệ số phạt rất lớn (1000) sẽ ép các tham số  $\theta_3, \theta_4$  xấp xỉ về 0 trong quá trình tối ưu hóa, từ đó làm mô hình trở nên đơn giản hơn (về cơ bản trở thành hàm bậc 2).

### 5.3.2 Công thức tổng quát

Nguyên lý chung: Giá trị tham số  $\theta_0, \dots, \theta_n$  càng nhỏ thì giả thuyết càng đơn giản và ít bị overfitting.

Hàm chi phí Regularized Linear Regression được định nghĩa như sau:

$$J(\theta) = \frac{1}{2m} \left[ \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right] \quad (5.2)$$

**Lưu ý quan trọng:**

- $\lambda$  (Lambda) là tham số điều chuẩn (Regularization Parameter).
- $\sum_{j=1}^n \theta_j^2$  gọi là thành phần điều chuẩn (Regularization Term).

- Theo quy ước (convention), ta **không** điều chuẩn tham số  $\theta_0$  (bias unit), do đó tổng chỉ chạy từ  $j = 1$  đến  $n$ .

## 5.4 Vai trò của tham số $\lambda$

Tham số  $\lambda$  đóng vai trò kiểm soát sự cân bằng giữa hai mục tiêu:

1. Khớp tốt với tập huấn luyện (giảm MSE).
  2. Giữ cho các tham số nhỏ (đơn giản hóa mô hình).
- **Nếu  $\lambda$  quá lớn (ví dụ  $10^{10}$ ):** Mô hình bị phạt quá nặng, dẫn đến tất cả  $\theta_j \approx 0$  (trừ  $\theta_0$ ). Hàm giả thuyết trở thành đường thẳng nằm ngang  $h_\theta(x) = \theta_0$ . Điều này gây ra hiện tượng **Underfitting**.
  - **Nếu  $\lambda$  quá nhỏ (hoặc bằng 0):** Thành phần điều chuẩn không có tác dụng, mô hình quay trở lại bài toán ban đầu và có nguy cơ bị **Overfitting**.

## 5.5 Regularized Gradient Descent

Để tối ưu hóa hàm  $J(\theta)$  có chứa thành phần điều chuẩn, ta áp dụng Gradient Descent. Do  $\theta_0$  không bị điều chuẩn, ta tách quy tắc cập nhật thành hai phần riêng biệt.

### 5.5.1 Cập nhật $\theta_0$

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)}) x_0^{(i)} \quad (5.3)$$

(Giữ nguyên như công thức Gradient Descent truyền thống).

### 5.5.2 Cập nhật $\theta_j$ (với $j = 1, 2, \dots, n$ )

Đạo hàm riêng của phần Regularization term là  $\frac{\partial}{\partial \theta_j} (\frac{\lambda}{2m} \theta_j^2) = \frac{\lambda}{m} \theta_j$ . Khi đưa vào công thức cập nhật, ta có:

$$\theta_j := \theta_j - \alpha \left[ \frac{1}{m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} \theta_j \right] \quad (5.4)$$

Ta có thể viết lại công thức này dưới dạng thu gọn để thấy rõ bản chất "co lại" (shrinkage) của tham số:

$$\theta_j := \theta_j \left( 1 - \alpha \frac{\lambda}{m} \right) - \alpha \frac{1}{m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)} \quad (5.5)$$

**Phân tích:** Thành phần  $(1 - \alpha \frac{\lambda}{m})$  thường là một số dương nhỏ hơn 1 một chút (ví dụ 0.99). Điều này có nghĩa là ở mỗi bước lặp, thuật toán sẽ thu nhỏ giá trị  $\theta_j$  đi một chút trước khi thực hiện cập nhật theo Gradient của loss.

## Chapter 6

# Non-linear Hypotheses & Neural Networks

### 6.1 Non-linear Classification (Phân loại phi tuyến)

#### 6.1.1 Hạn chế của Linear/Logistic Regression

Trước đây, chúng ta sử dụng Logistic Regression để giải quyết bài toán phân loại. Phương pháp này hoạt động tốt khi tìm kiếm một đường **Decision Boundary** (Ranh giới quyết định) đơn giản hoặc phi tuyến tính cơ bản (như hình tròn, elip) với số lượng **Features** (Đặc trưng/Thuộc tính) ít.

Tuy nhiên, trong thực tế, các bài toán thường có rất nhiều features.

Ví dụ:

- Giả sử ta có  $n = 100$  features  $(x_1, \dots, x_{100})$ .
- Để tạo ra ranh giới phi tuyến phức tạp, ta cần các số hạng bậc cao (polynomial terms) như  $x_1^2, x_1x_2, \dots$ .
- Số lượng features bậc 2 sẽ tăng theo  $O(n^2) \approx \frac{n^2}{2}$ . Với  $n = 100$ , ta có khoảng 5000 features. Với  $n$  lớn hơn, con số này bùng nổ (ví dụ bậc 3 là  $O(n^3)$ ).

Việc có quá nhiều features khiến mô hình Logistic Regression trở nên cồng kềnh, tốn kém tài nguyên tính toán và dễ bị Overfitting.

#### 6.1.2 Ví dụ: Computer Vision (Thị giác máy tính)

Xét bài toán **Car Detection** (Phát hiện xe hơi). Máy tính nhìn nhận một bức ảnh dưới dạng ma trận các con số (cường độ điểm ảnh - pixel intensity grid).

- Với một bức ảnh nhỏ  $50 \times 50$  pixels, ta có  $n = 2500$  features (hoặc 7500 nếu là ảnh màu RGB).
- Nếu dùng Logistic Regression với các features bậc 2 (Quadratic features  $x_i x_j$ ), số lượng features sẽ lên tới khoảng 3 triệu ( $\approx \frac{2500^2}{2}$ ).

Điều này cho thấy Logistic Regression không khả thi cho các bài toán phức tạp như Computer Vision. Chúng ta cần một giải thuật khác có khả năng học được các giả thuyết phi tuyến phức tạp ( $\theta$  thay đổi) một cách hiệu quả hơn. Đó chính là Neural Networks.

What is this?

You see this:

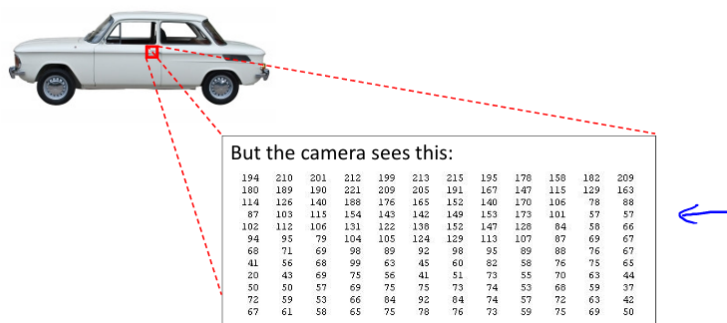


Figure 6.1: Minh họa cách máy tính nhận đầu vào trong bài toán Car Detection.

## 6.2 Neural Networks (Mạng Nơ-ron)

### 6.2.1 Lịch sử và Nguồn gốc

**Neural Networks** là các thuật toán được thiết kế để mô phỏng hoạt động của bộ não con người.

- Được sử dụng rộng rãi vào những năm 80 và đầu 90.
- Suy giảm sự phổ biến vào cuối những năm 90 (do sự nổi lên của SVM và hạn chế về phần cứng/dữ liệu).
- Hồi sinh mạnh mẽ từ khoảng 2011-2012 nhờ sự phát triển của phần cứng (GPU) và dữ liệu lớn (Big Data), trở thành kỹ thuật "State-of-the-art" (hiện đại nhất) cho nhiều ứng dụng.

### 6.2.2 Giả thuyết "One Learning Algorithm"

Các nhà khoa học nhận thấy vỏ não (cortex) có khả năng thích nghi tuyệt vời (**Neuroplasticity**).

- **Thí nghiệm trên động vật:** Cắt dây thần kinh từ tai đến vùng **Auditory Cortex** (Vỏ não thính giác) và nối dây thần kinh từ mắt vào đó. Kết quả là vùng não này học cách "nhìn" thay vì "nghe".
- Tương tự với vùng **Somatosensory Cortex** (Vỏ não xúc giác).

Điều này dẫn đến giả thuyết rằng có thể bộ não sử dụng một thuật toán học tập duy nhất ("One Learning Algorithm") để xử lý mọi loại dữ liệu (hình ảnh, âm thanh, xúc giác). Neural Network cố gắng xấp xỉ thuật toán này.

## 6.3 Mô hình Neural Network (Model Representation)

### 6.3.1 Mô phỏng Nơ-ron sinh học

Một nơ-ron sinh học bao gồm:

- **Dendrites** (Sợi nhánh): Dây nhận tín hiệu đầu vào (Input wires).
- **Cell body** (Thân tế bào): Xử lý thông tin (Nucleus).
- **Axon** (Sợi trục): Dây truyền tín hiệu đầu ra (Output wire).

### Neuron in the brain

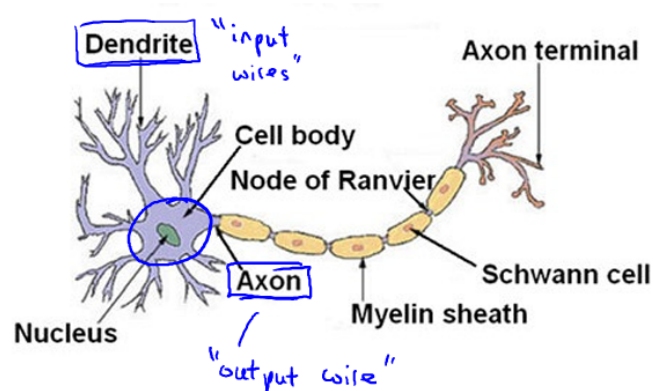
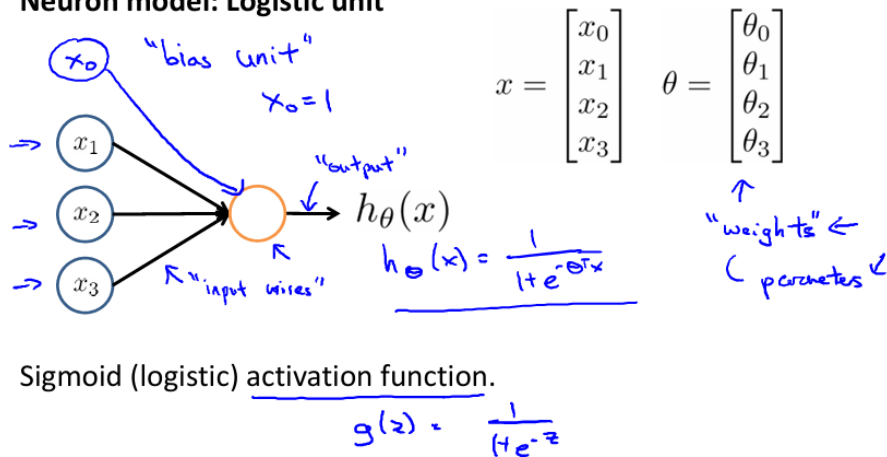


Figure 6.2: Minh họa nơ ron sinh học của động vật.

### 6.3.2 Nơ-ron nhân tạo (Artificial Neuron)

Trong máy tính, ta mô phỏng nơ-ron dưới dạng một **Logistic Unit**:

#### Neuron model: Logistic unit



Andrew

Figure 6.3: Minh họa Logistic Regression bằng Neuron Network.

- **Input:**  $x = [x_0, x_1, x_2, \dots]^T$  (với  $x_0 = 1$  là **Bias Unit**).  
 người ta muốn mô phỏng chính xác theo bộ não con người -> trước khi gặp dữ liệu, nó đã có 1 phần phán đoán về dữ liệu -> đúng cách bộ não con người hoạt động -> cộng thêm  $x_0$ .

- **Weights** (Trọng số):  $\theta = [\theta_0, \theta_1, \theta_2, \dots]^T$ .
- **Output**:  $h_{\theta}(x) = \frac{1}{1+e^{-\theta^T x}}$ .
- Hàm kích hoạt (**Activation Function**): Ta sử dụng hàm Sigmoid (logistic function)  
 $g(z) = \frac{1}{1+e^{-z}}$ .

### 6.3.3 Kiến trúc Mạng Nơ-ron

Một Neural Network là tập hợp các nơ-ron được kết nối với nhau theo các lớp (**Layers**).

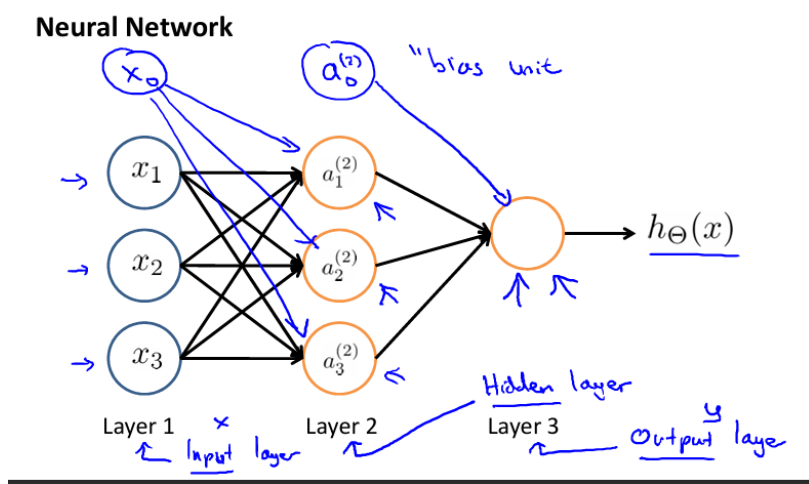


Figure 6.4: Minh họa Neuron Network cơ bản.

1. **Input Layer** (Lớp đầu vào): Chứa các features  $x$ .
2. **Hidden Layer** (Lớp ẩn): Các lớp ở giữa. Gọi là "ẩn" vì ta không trực tiếp thấy giá trị của chúng trong tập dữ liệu huấn luyện (tư tưởng Black Box).
3. **Output Layer** (Lớp đầu ra): Đưa ra giá trị dự đoán cuối cùng.

## 6.4 Forward Propagation (Lan truyền xuôi)

Để tính toán đầu ra của mạng, ta thực hiện quá trình lan truyền xuôi từ Input đến Output.  
 Ký hiệu:

- $a_i^{(j)}$ : "Activation" (giá trị kích hoạt) của đơn vị  $i$  trong lớp  $j$ .
- $\Theta^{(j)}$ : Ma trận trọng số kiểm soát ánh xạ từ lớp  $j$  sang lớp  $j + 1$ .

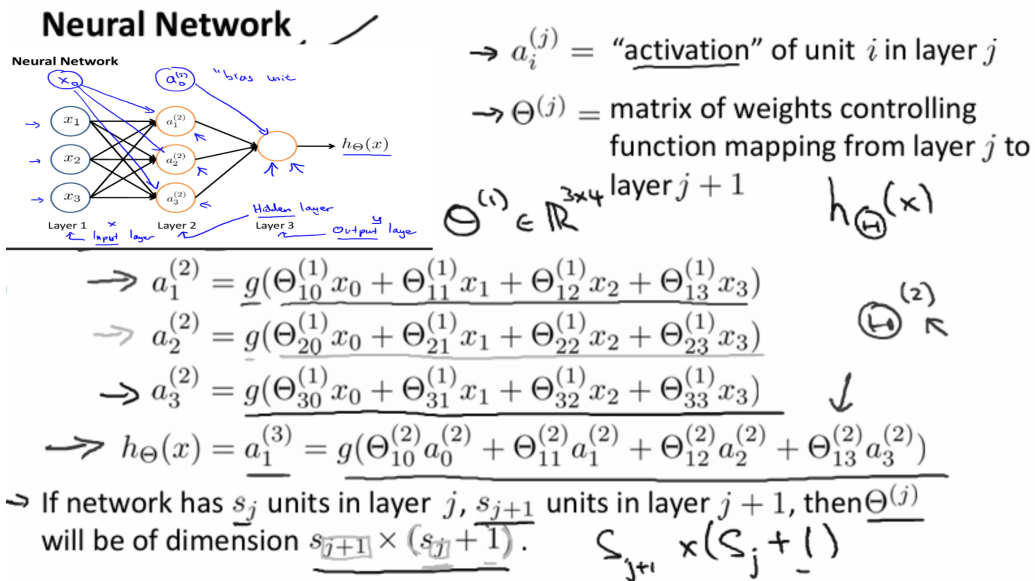


Figure 6.5: Minh họa quá trình lan truyền xuôi của Neuron Network.

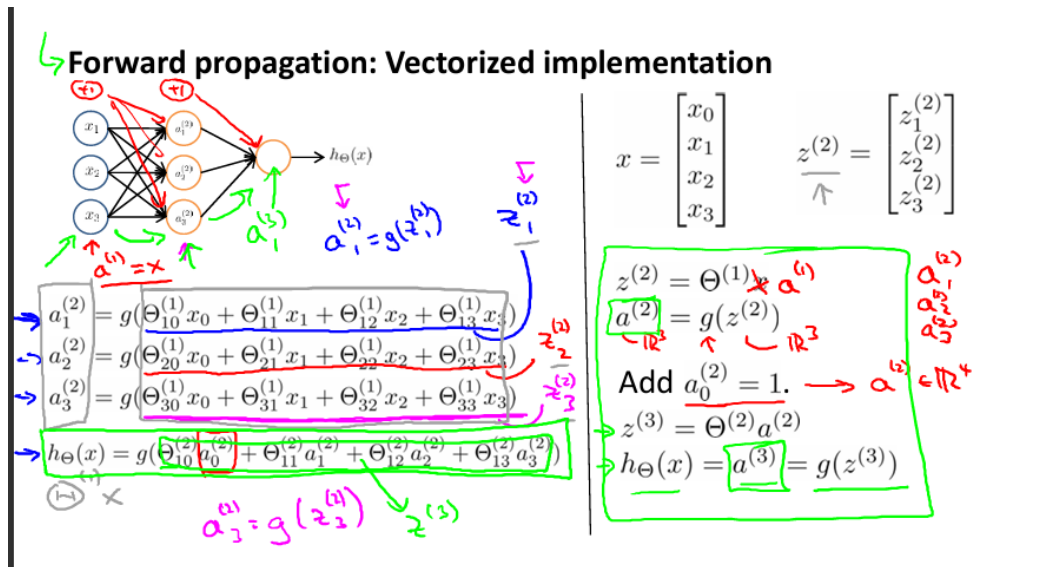


Figure 6.6: Minh họa quá trình lan truyền xuôi của Neuron Network (viết gọn hơn).

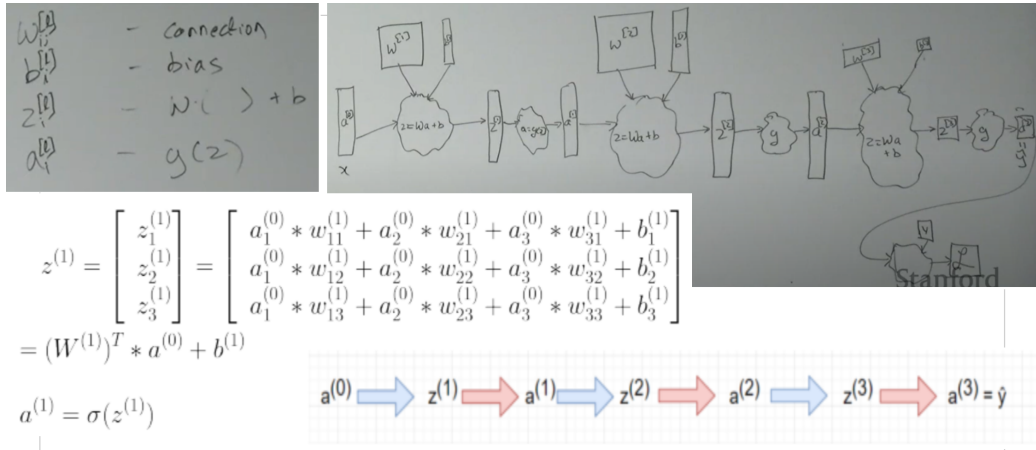


Figure 6.7: Minh họa quá trình lan truyền xuôi của Neuron Network (chi tiết).

#### 6.4.1 Tính toán từng bước (Vectorized Implementation)

Giả sử mạng có 3 lớp (Input  $x$ , 1 Hidden, Output).

1. **Tại lớp 2 (Hidden Layer):** Tính  $z^{(2)}$  và kích hoạt  $a^{(2)}$ :

$$z^{(2)} = \Theta^{(1)} a^{(1)} \quad (\text{với } a^{(1)} = x) \quad (6.1)$$

$$a^{(2)} = g(z^{(2)}) \quad (6.2)$$

(Thêm bias unit  $a_0^{(2)} = 1$  sau bước này).

2. **Tại lớp 3 (Output Layer):** Tính  $z^{(3)}$  và đầu ra  $h_{\Theta}(x)$ :

$$z^{(3)} = \Theta^{(2)} a^{(2)} \quad (6.3)$$

$$h_{\Theta}(x) = a^{(3)} = g(z^{(3)}) \quad (6.4)$$

Kiến trúc này cho phép mạng nơ-ron học các đặc trưng phức tạp hơn của dữ liệu thông qua các lớp ẩn, thay vì chỉ dựa vào input thô ban đầu.



## 6.5 Trực giác về Neural Network (Logic Gates)

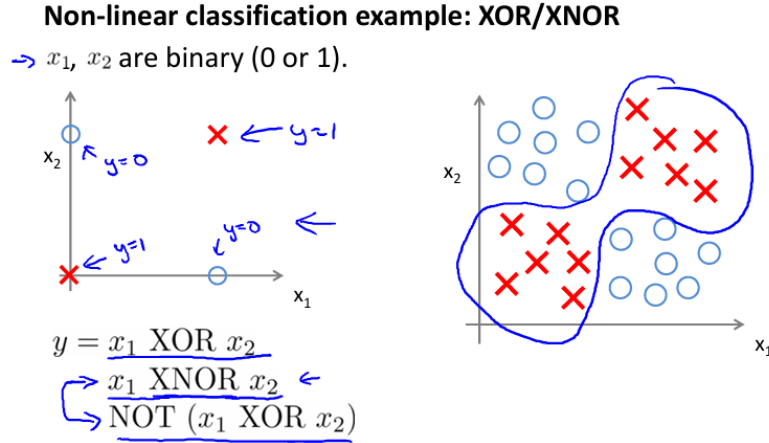


Figure 6.8: Non-linear classification sử dụng Neuron Network.

Tại sao Neural Network có thể học được các hàm phi tuyến phức tạp? Ta có thể chứng minh thông qua việc mô phỏng các cổng logic (Logic Gates) như AND, OR, NOT, XNOR.

### 6.5.1 Ví dụ đơn giản: Cổng AND

Giả sử  $x_1, x_2 \in \{0, 1\}$ . Ta muốn  $y = x_1 \text{ AND } x_2$ . Ta thiết lập một nơ-ron với trọng số:  $\theta_0 = -30, \theta_1 = 20, \theta_2 = 20$ .

Hàm giả thuyết:  $h_{\Theta}(x) = g(-30 + 20x_1 + 20x_2)$ .

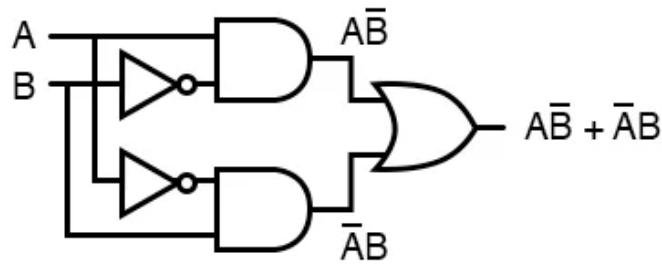
- Nếu  $x_1 = 0, x_2 = 0 \rightarrow g(-30) \approx 0$ .
- Nếu  $x_1 = 0, x_2 = 1 \rightarrow g(-10) \approx 0$ .
- Nếu  $x_1 = 1, x_2 = 0 \rightarrow g(-10) \approx 0$ .
- Nếu  $x_1 = 1, x_2 = 1 \rightarrow g(10) \approx 1$ .

→ Đây chính xác là hàm AND.

### 6.5.2 Bài toán XNOR (Non-linear)



... is equivalent to ...



$$A \oplus B = A\bar{B} + \bar{A}B$$

Figure 6.9: Cổng XOR.

Bài toán XNOR (hoặc XOR) là bài toán phi tuyến điển hình mà Logistic Regression đơn thuần không giải quyết được (không thể phân chia bằng 1 đường thẳng). XNOR là sự kết hợp của các cổng logic cơ bản:  $(x_1 \text{ AND } x_2) \text{ OR } ((\text{NOT } x_1) \text{ AND } (\text{NOT } x_2))$ .

Trong Neural Network:

- **Layer 1:** Input  $x_1, x_2$ .
- **Layer 2:** Tính toán các thành phần con (AND, NOR,...).
- **Layer 3:** Kết hợp kết quả từ Layer 2 (OR) để ra kết quả XNOR.

Việc chồng nhiều layer ("Deep" Network) cho phép mô hình học các hàm số cực kỳ phức tạp từ các hàm đơn giản hơn.

### Simple example: AND

→  $x_1, x_2 \in \{0, 1\}$   
 →  $y = x_1 \text{ AND } x_2$

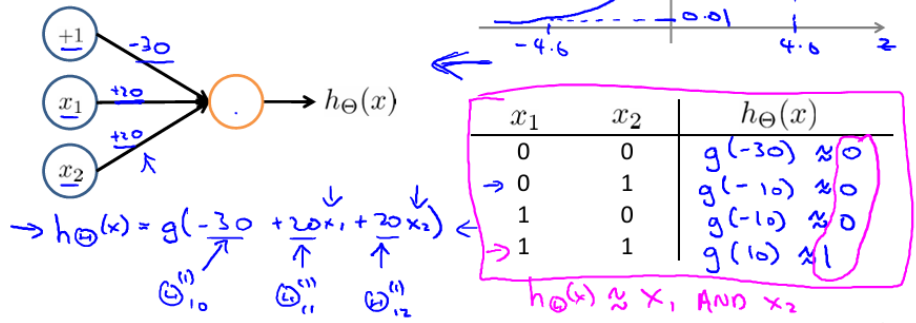


Figure 6.10: Tạo hàm logic AND sử dụng Neuron Network.

### Example: OR function

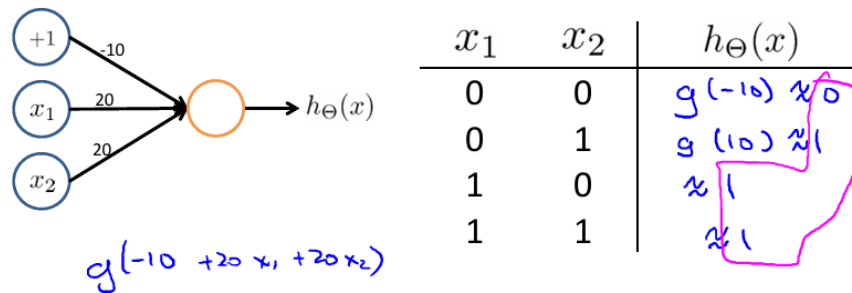


Figure 6.11: Tạo hàm logic OR sử dụng Neuron Network.

→  $x_1 \text{ AND } x_2$

→  $x_1 \text{ OR } x_2$

$\{0, 1\}$

### Negation:

NOT  $x_1$

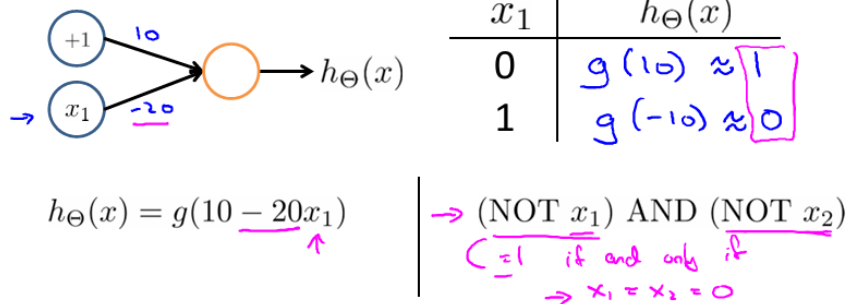


Figure 6.12: Tạo hàm logic Negation (NOT) sử dụng Neuron Network.

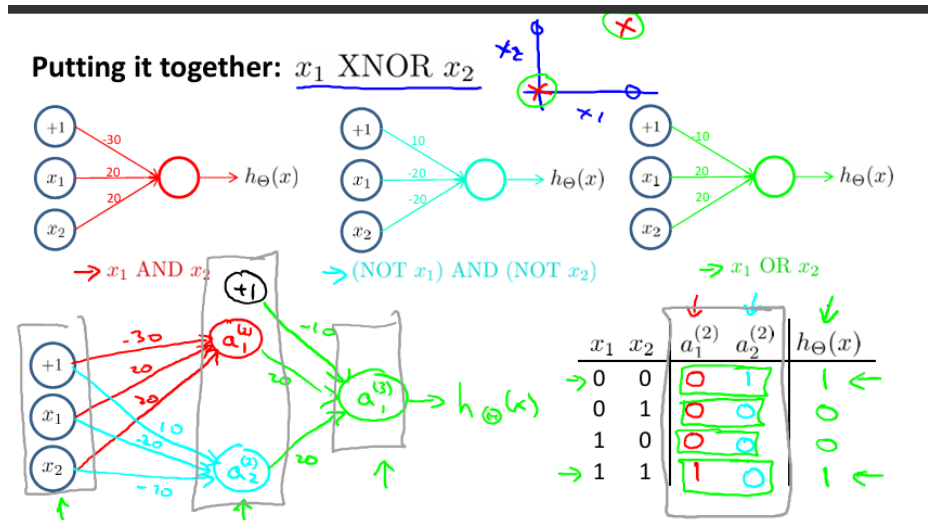


Figure 6.13: Tạo hàm logic XOR sử dụng Neuron Network.

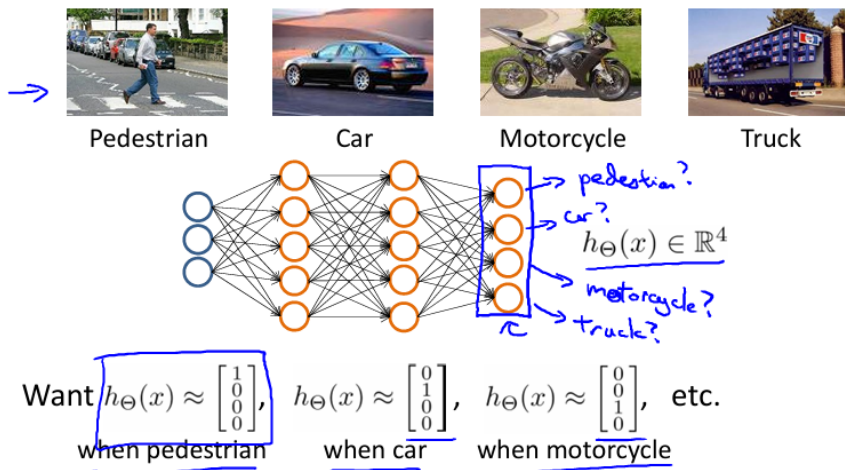
## 6.6 Multi-class Classification (Phân loại đa lớp)

Để phân loại nhiều lớp (ví dụ: Người đi bộ, Xe hơi, Xe máy, Xe tải → 4 lớp), ta sử dụng phương pháp **One-vs-All** mở rộng.

- Thay vì  $h_{\Theta}(x)$  trả về 1 số thực, nó sẽ trả về một vector  $\in R^K$  (với  $K$  là số lớp).
- Ví dụ với 4 lớp:
  - Người đi bộ  $\approx [1, 0, 0, 0]^T$
  - Xe hơi  $\approx [0, 1, 0, 0]^T$
  - Xe máy  $\approx [0, 0, 1, 0]^T$
  - Xe tải  $\approx [0, 0, 0, 1]^T$

Mỗi đơn vị ở lớp đầu ra sẽ dự đoán xác suất  $P(y = i|x)$  cho lớp  $i$  tương ứng.

### Multiple output units: One-vs-all.



### Multiple output units: One-vs-all.

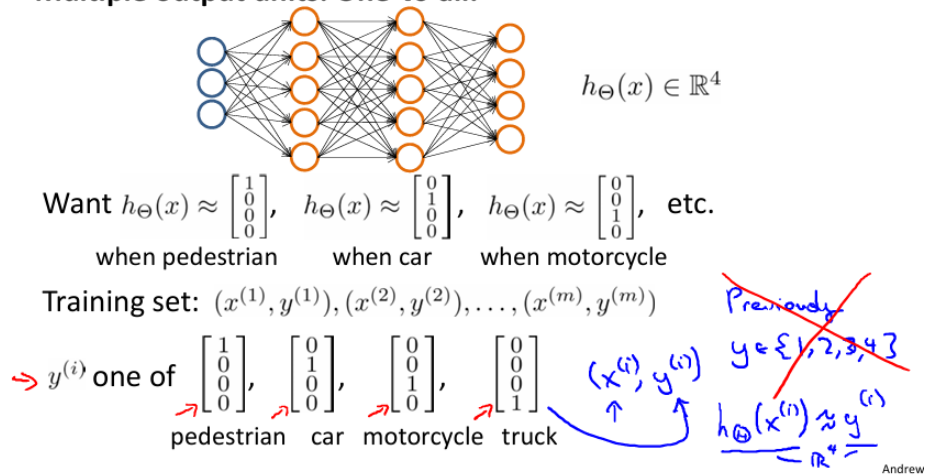


Figure 6.14: Multiple output unit: One-vs-all.

## Chapter 7

# Neural Network: Cost Function & Propagation

### 7.1 Tổng quan và Ký hiệu

Xét bài toán phân loại với tập dữ liệu huấn luyện gồm  $m$  mẫu:

$$\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$$

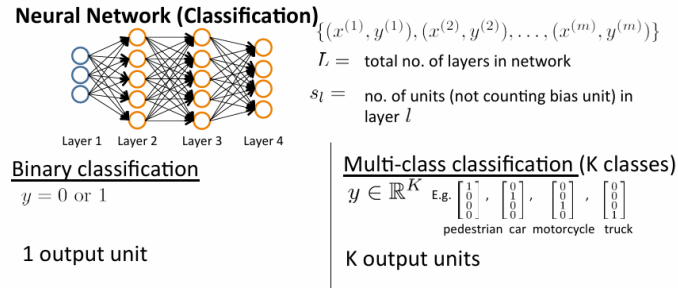


Figure 7.1: Neural Network for Classification

#### 7.1.1 Ký hiệu kiến trúc mạng

- $L$ : Tổng số lớp (layers) trong mạng.
- $s_l$ : Số lượng đơn vị (units) trong lớp  $l$  (không tính bias unit).
- $K$ : Số lượng đơn vị đầu ra (Output units), tương ứng với số lớp cần phân loại.

#### 7.1.2 Phân loại (Classification Types)

##### 1. Binary Classification (Phân loại nhị phân):

- $y = 0$  hoặc  $1$ .
- Chỉ có 1 unit đầu ra ( $s_L = 1$ ).

##### 2. Multi-class Classification (Phân loại đa lớp):

- $y \in R^K$  (Ví dụ: Người đi bộ, Xe hơi, Xe máy, Xe tải).
- Có  $K$  units đầu ra ( $s_L = K$ ).

## 7.2 Hàm Chi phí (Cost Function)

Đối với Neural Network, hàm chi phí là sự mở rộng của Logistic Regression, áp dụng cho  $K$  đầu ra và bao gồm thành phần điều chuẩn (Regularization term) cho tất cả các trọng số.

Công thức tổng quát:

$$J(\Theta) = -\frac{1}{m} \left[ \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(h_{\Theta}(x^{(i)}))_k + (1 - y_k^{(i)}) \log(1 - (h_{\Theta}(x^{(i)}))_k) \right] + \frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (\Theta_{ji}^{(l)})^2 \quad (7.1)$$

**Giải thích thành phần:**

- **Phần đầu (Loss):** Là tổng sai số Binary Cross Entropy (BCE) trên tất cả  $K$  đầu ra và  $m$  mẫu dữ liệu.
- **Phần sau (Regularization term):** Tổng bình phương của tất cả các trọng số  $\Theta_{ji}^{(l)}$  trong mạng (trừ bias unit).
- $\Theta_{ji}^{(l)}$ : Trọng số nối từ unit  $i$  của lớp  $l$  đến unit  $j$  của lớp  $l + 1$ .
- $h_{\Theta}(x) \in R$ ;  $(h_{\Theta}(x))_i = i^{th}$  output.

## 7.3 Forward Propagation (Lan truyền xuôi)

Sau khi định nghĩa hàm chi phí, ta cần tính toán giá trị đầu ra và loss. Quá trình này gọi là Forward Propagation. Để đơn giản hóa, giả sử mạng có 1 lớp ẩn (hidden layer), đầu vào  $x \in R^d$  và không tính bias term.

### 7.3.1 Các bước tính toán

**Bước 1: Tại lớp ẩn (Hidden Layer)** Tính biến trung gian  $z$  và giá trị kích hoạt  $h$ :

$$z = W^{(1)}x \quad (7.2)$$

$$h = \phi(z) \quad (7.3)$$

Trong đó:

- $W^{(1)} \in R^{h \times d}$ : Ma trận trọng số lớp ẩn.
- $\phi(z)$ : Hàm kích hoạt (Activation function), ví dụ Sigmoid hoặc ReLU.

**Bước 2: Tại lớp đầu ra (Output Layer)** Tính đầu ra  $o$ :

$$o = W^{(2)}h \quad (7.4)$$

Trong đó  $W^{(2)} \in R^{q \times h}$  là trọng số lớp đầu ra.

**Bước 3: Tính hàm mục tiêu (Objective Function)** Hàm mục tiêu  $J$  bao gồm Loss  $L$  và Regularization term  $s$ :

$$L = l(o, y) \quad (\text{Loss trên một mẫu}) \quad (7.5)$$

$$s = \frac{\lambda}{2} (\|W^{(1)}\|_F^2 + \|W^{(2)}\|_F^2) \quad (\text{Regularization term}) \quad (7.6)$$

$$J = L + s \quad (7.7)$$

*Ghi chú:* Nếu  $\lambda$  cực bé, regularization không có tác động. Nếu  $\lambda$  cực lớn, trọng số sẽ bị ép về giá trị rất nhỏ để giảm thiểu loss.

## 7.4 Backward Propagation (Lan truyền ngược)

Backward Propagation sử dụng quy tắc chuỗi (Chain Rule) để tính đạo hàm của hàm mục tiêu  $J$  theo từng trọng số  $W$ , từ đó cập nhật tham số nhằm cực tiểu hóa  $J$ .

### 7.4.1 Đạo hàm tại lớp đầu ra

Ký hiệu chain rule:

$$\frac{\partial Z}{\partial X} = \text{prod}\left(\frac{\partial Z}{\partial Y}, \frac{\partial Y}{\partial X}\right) \quad (7.8)$$

Ta có:

$$\frac{\partial J}{\partial L} = 1 \text{ and } \frac{\partial J}{\partial s} = 1 \quad (7.9)$$

Đầu tiên, ta tính đạo hàm của  $J$  theo biến đầu ra  $o$ :

$$\frac{\partial J}{\partial o} = \text{prod}\left(\frac{\partial J}{\partial L}, \frac{\partial L}{\partial o}\right) = \frac{\partial L}{\partial o} \in R^q \quad (7.10)$$

Đạo hàm của thành phần điều chuẩn  $s$  theo trọng số:

$$\frac{\partial s}{\partial W^{(l)}} = \lambda W^{(l)} \quad (7.11)$$

### 7.4.2 Gradient của trọng số lớp Output ( $W^{(2)}$ )

Áp dụng quy tắc chuỗi (chain rule):

$$\frac{\partial J}{\partial W^{(2)}} = \text{prod}\left(\frac{\partial J}{\partial o}, \frac{\partial o}{\partial W^{(2)}}\right) + \text{prod}\left(\frac{\partial J}{\partial s}, \frac{\partial s}{\partial W^{(2)}}\right) = \frac{\partial J}{\partial o} h^T + \lambda W^{(2)} \quad (7.12)$$

(Lưu ý:  $h^T$  là chuyển vị của vector kích hoạt lớp ẩn, dùng để thực hiện phép nhân ma trận).



### 7.4.3 Gradient của trọng số lớp Input/Hidden ( $W^{(1)}$ )

Để tính gradient cho  $W^{(1)}$ , ta cần lan truyền sai số ngược về lớp ẩn.

**Bước 1: Gradient tại đầu ra lớp ẩn ( $h$ )**

$$\frac{\partial J}{\partial h} = \text{prod}\left(\frac{\partial J}{\partial o}, \frac{\partial o}{\partial h}\right) = W^{(2)T} \frac{\partial J}{\partial o} \quad (7.13)$$

**Bước 2: Gradient tại biến trung gian ( $z$ )** Do hàm kích hoạt áp dụng theo từng phần tử (element-wise), ta sử dụng phép nhân Hadamard (kí hiệu  $\odot$ ):

$$\frac{\partial J}{\partial z} = \text{prod}\left(\frac{\partial J}{\partial h}, \frac{\partial h}{\partial z}\right) = \frac{\partial J}{\partial h} \odot \phi'(z) \quad (7.14)$$

- hàm kích hoạt hoạt động độc lập trên từng phần tử  $\rightarrow$  Jacobian là ma trận đường chéo  $\rightarrow$  dùng element wise.
- mỗi output phụ thuộc vào nhiều input  $\rightarrow$  Jacobian là ma trận đầy đủ  $\rightarrow$  phép nhân tuyến tính  $\rightarrow$  dùng prod (matrix multiply).

(Ví dụ: Nếu  $\phi$  là sigmoid, thì  $\phi'(z) = \phi(z)(1 - \phi(z))$ ).

**Bước 3: Gradient cuối cùng cho  $W^{(1)}$**

$$\frac{\partial J}{\partial W^{(1)}} = \text{prod}\left(\frac{\partial J}{\partial z}, \frac{\partial z}{\partial W^{(1)}}\right) + \text{prod}\left(\frac{\partial J}{\partial s}, \frac{\partial s}{\partial W^{(1)}}\right) = \frac{\partial J}{\partial z} x^T + \lambda W^{(1)} \quad (7.15)$$

## 7.5 Tổng kết Công thức Cập nhật

Quy tắc chung để tính đạo hàm Loss theo trọng số tại bất kỳ lớp nào:

**Gradient của  $W$**  = (Gradient của Loss theo biến đổi tuyến tính tại lớp đó) + (Regularization term)

Việc tính toán này cho phép ta sử dụng các thuật toán tối ưu (như Gradient Descent) để huấn luyện mạng nơ-ron sâu.

## Chapter 8

# Advice for Applying Machine Learning

### 8.1 Debugging a Learning Algorithm (Gỡ lỗi thuật toán học)

Giả sử bạn đã triển khai thuật toán **Regularized Linear Regression** (Hồi quy tuyến tính có điều chuẩn) để dự đoán giá nhà. Tuy nhiên, khi kiểm tra giả thuyết (*hypothesis*) trên dữ liệu mới, bạn nhận thấy sai số dự đoán quá lớn. Bạn nên làm gì tiếp theo?

Thông thường, chúng ta sẽ nghĩ đến các phương án sau:

- Thu thập thêm dữ liệu huấn luyện (*Get more training examples*).
- Thử giảm bớt số lượng đặc trưng (*Try smaller sets of features*).
- Thử bổ sung thêm đặc trưng mới (*Try getting additional features*).
- Thử thêm các đặc trưng đa thức (*Try adding polynomial features*) như  $x_1^2, x_2^2, x_1x_2, \dots$ .
- Thử giảm tham số điều chuẩn  $\lambda$  (*Try decreasing  $\lambda$* ).
- Thử tăng tham số điều chuẩn  $\lambda$  (*Try increasing  $\lambda$* ).

Thay vì thử sai ngẫu nhiên rất tốn thời gian, chúng ta cần sử dụng **Machine Learning Diagnostic** (Chẩn đoán học máy). Đây là phương pháp kiểm tra để hiểu rõ thuật toán đang gặp vấn đề gì (ví dụ: đang bị High Bias hay High Variance) và hướng dẫn cách cải thiện hiệu suất tốt nhất.

### 8.2 Evaluating a Hypothesis (Đánh giá giả thuyết)

Để đánh giá xem giả thuyết  $h_\theta(x)$  có hoạt động tốt hay không và tránh hiện tượng **Overfitting** (Quá khớp - khi mô hình học cả nhiễu của tập huấn luyện và thất bại trên dữ liệu mới), ta cần chia dữ liệu.

#### 8.2.1 Training / Test Split (Chia tập Huấn luyện / Kiểm tra)

Phương pháp tiêu chuẩn là chia dữ liệu thành 2 phần (thường là 70% cho huấn luyện và 30% cho kiểm tra):

- **Training Set:**  $\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$  dùng để tìm tham số  $\theta$  nhằm cực tiểu hóa hàm mất mát huấn luyện  $J_{train}(\theta)$ .
- **Test Set:**  $\{(x_{test}^{(1)}, y_{test}^{(1)}), \dots, (x_{test}^{(m_{test})}, y_{test}^{(m_{test})})\}$  dùng để đánh giá sai số tổng quát hóa.

## 8.2.2 Công thức tính lỗi trên tập test

### 1. Linear Regression:

$$J_{test}(\theta) = \frac{1}{2m_{test}} \sum_{i=1}^{m_{test}} (h_{\theta}(x_{test}^{(i)}) - y_{test}^{(i)})^2 \quad (8.1)$$

### 2. Logistic Regression:

$$J_{test}(\theta) = -\frac{1}{m_{test}} \sum_{i=1}^{m_{test}} (y_{test}^{(i)} \log(h_{\theta}(x_{test}^{(i)})) - (1 - y_{test}^{(i)}) \log(h_{\theta}(x_{test}^{(i)}))) \quad (8.2)$$

## 8.3 Model Selection (Lựa chọn mô hình)

Nếu ta dùng tập Test để lựa chọn bậc của đa thức (degree of polynomial  $d$ ), tham số  $\theta$  sẽ bị tối ưu cho tập Test đó, dẫn đến kết quả đánh giá không còn khách quan (công bằng).

Khách hàng là người quyết định vấn đề cần giải quyết, khách hàng mới biết được nhu cầu thực sự của mình -> Khách hàng sẽ là người chia train/test data, họ sẽ giữ phần test data lại, mình sẽ sử dụng phần còn lại để phát triển mô hình -> mình chỉ lấy 1 phần cho training, phần còn lại cho validation -> Nếu không tốt trên validation set, mình có thể bác bỏ các phần sai, và cải thiện mô hình.

Giải pháp là chia dữ liệu thành 3 phần:

- **Training Set (60%):** Dùng để huấn luyện tham số  $\theta$ .
- **Cross Validation Set (20%)** (Tập kiểm định chéo - CV): Dùng để lựa chọn mô hình (chọn bậc  $d$ , chọn  $\lambda$ ).
- **Test Set (20%):** Dùng để đánh giá cuối cùng.

### 8.3.1 Cross Validation

Có thể một số điểm dữ liệu có ích cho quá trình train đã bị bạn ném vào để làm validation, test và model không có cơ hội học điểm dữ liệu đó. Thậm chí, đôi khi do ít dữ liệu nên có một vài class chỉ có trong validation, test mà không có trong train (do việc chia train, val là hoàn toàn ngẫu nhiên) dẫn đến một kết quả tồi tệ khi validation và test. Và nếu chúng ta dựa ngay vào kết quả đó để đánh giá rằng model không tốt thì thật là oan uổng cho nó giống như một học sinh không được học Tiếng Anh mà phải đi thi TOEFL vậy.

Test data ở đây là validation mà ta đề cập ở trên, còn test data sẽ để riêng và dành cho bước đánh giá cuối cùng nhằm kiểm tra “phản ứng” của model khi gặp các dữ liệu unseen hoàn toàn

Phần dữ liệu Training thì sẽ được chia ngẫu nhiên thành  $K$  phần ( $K$  là một số nguyên, hay chọn là 5 hoặc 10). Sau đó train model  $K$  lần, mỗi lần train sẽ chọn 1 phần làm dữ liệu validation và  $K-1$  phần còn lại làm dữ liệu training. Kết quả đánh giá model cuối cùng sẽ

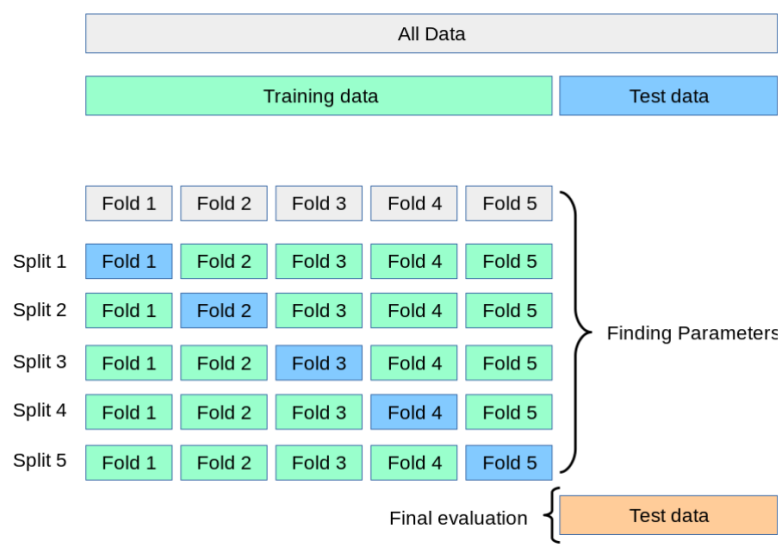


Figure 8.1: Minh họa Cross Validation.

là trung bình cộng kết quả đánh giá của K lần train. Đó chính là lý do vì sao ta đánh giá khách quan và chính xác hơn.

Sau khi đánh giá xong model và nếu cảm thấy kết quả (ví dụ accuracy trung bình) chấp nhận được thì ta có thể thực hiện một trong 2 cách sau để tạo ra model cuối cùng (để mang đi dùng predict):

- Cách một: Trong quá trình train các fold, ta lưu lại model tốt nhất và mang model đó đi dùng luôn. Cách này sẽ có ưu điểm là không cần train lại nhưng lại có nhược điểm là model sẽ không nhìn được all data và có thể không làm việc tốt với các dữ liệu trong thực tế.
- Cách hai: train model 1 lần nữa với toàn bộ dữ liệu (không chia train, val nữa) và sau đó save lại và mang đi predict với test set để xem kết quả như nào

Chú ý: Còn có một cách khác mà theo mình thấy là nó mở rộng từ K-Fold CV và hay hơn nhiều là Stratified K-Fold CV. Với phương pháp này thì nó sẽ chỉ shuffle dữ liệu một lần đầu tiên trước khi bắt đầu chia fold và nó sẽ cố gắng chia sao cho tỷ lệ các class trong các fold là tương đồng nhau.

### Train/validation/test error

Training error:

$$\rightarrow J_{train}(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \quad \mathcal{J}(\theta)$$

Cross Validation error:

$$\rightarrow J_{cv}(\theta) = \frac{1}{2m_{cv}} \sum_{i=1}^{m_{cv}} (h_{\theta}(x_{cv}^{(i)}) - y_{cv}^{(i)})^2$$

Test error:

$$\rightarrow J_{test}(\theta) = \frac{1}{2m_{test}} \sum_{i=1}^{m_{test}} (h_{\theta}(x_{test}^{(i)}) - y_{test}^{(i)})^2$$

Figure 8.2: Công thức tính loss train/valid/test.

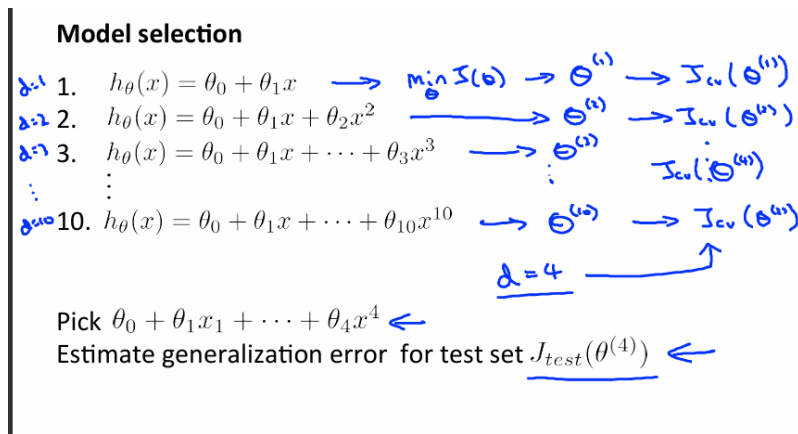


Figure 8.3: Minh họa Cross Validation model selection.

10 mô hình  $\rightarrow$  10 validation loss  $\rightarrow$  pick model có validation loss thấp nhất để test trên test data

### 8.3.2 Quy trình lựa chọn mô hình

1. Huấn luyện các mô hình với bậc khác nhau (ví dụ  $d = 1, d = 2, \dots, d = 10$ ) trên tập Training.
2. Tính sai số trên tập Cross Validation ( $J_{cv}$ ) cho từng mô hình.
3. Chọn mô hình có  $J_{cv}$  thấp nhất.
4. Đánh giá sai số tổng quát ( $J_{test}$ ) của mô hình đã chọn trên tập Test.

## 8.4 Diagnosing Bias vs Variance (Chẩn đoán Độ lệch và Phương sai)

Nếu mô hình có kết quả dự đoán kém, thường nó rơi vào một trong hai trường hợp: **High Bias** (Underfitting - Chưa khớp) hoặc **High Variance** (Overfitting - Quá khớp).

### 8.4.1 Phân tích dựa trên đồ thị lỗi

Chúng ta vẽ đồ thị của  $J_{train}(\theta)$  và  $J_{cv}(\theta)$  theo bậc của đa thức  $d$ .

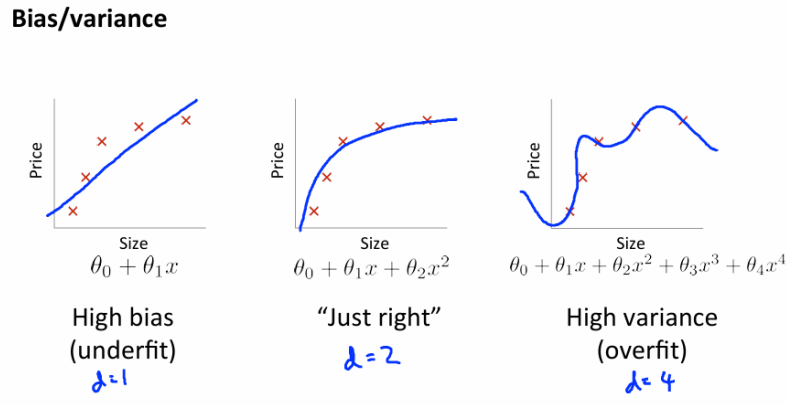


Figure 8.4: Bias and Variance Traceoff.

- **High Bias (Underfitting -  $d$  nhỏ):**
  - $J_{train}$  cao (mô hình không khớp tốt dữ liệu huấn luyện).
  - $J_{cv}$  cao (xấp xỉ  $J_{train}$ ).
  - Đặc điểm: Cả hai lỗi đều cao và gần nhau.
- **High Variance (Overfitting -  $d$  lớn):**
  - $J_{train}$  thấp (mô hình khớp rất tốt dữ liệu huấn luyện).
  - $J_{cv}$  cao (thất bại trên dữ liệu mới).
  - Đặc điểm:  $J_{cv} \gg J_{train}$  (Khoảng cách giữa hai đường lớn).

## 8.5 Regularization và Bias/Variance

Tham số điều chuẩn  $\lambda$  ảnh hưởng trực tiếp đến Bias và Variance.

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2 \quad (8.3)$$

- **Large  $\lambda$  (Quá lớn):** Phạt nặng các tham số  $\theta$ , ép chúng về gần 0. Mô hình trở thành đường thẳng (hoặc hằng số). → **High Bias (Underfitting)**.
- **Small  $\lambda$  (Quá nhỏ hoặc bằng 0):** Không có điều chuẩn, mô hình quá phức tạp. → **High Variance (Overfitting)**.
- **Intermediate  $\lambda$  (Vừa phải):** Cân bằng tốt, cho kết quả "Just right".

## 8.6 Learning Curves (Đường cong học tập)

Learning Curves là đồ thị biểu diễn lỗi ( $J_{train}$  và  $J_{cv}$ ) theo số lượng mẫu huấn luyện  $m$  (training set size).

### Bias/variance

Training error:  $J_{train}(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$

Cross validation error:  $J_{cv}(\theta) = \frac{1}{2m_{cv}} \sum_{i=1}^{m_{cv}} (h_{\theta}(x_{cv}^{(i)}) - y_{cv}^{(i)})^2$  (or  $J_{test}(\theta)$ )

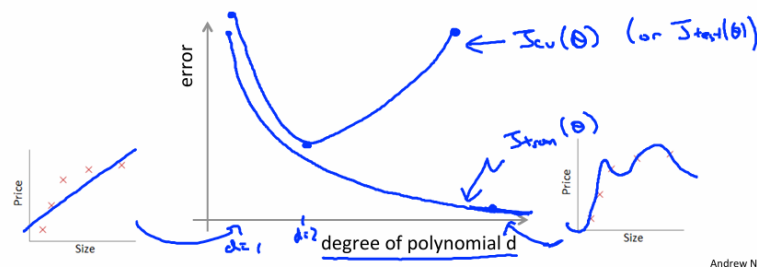


Figure 8.5: Learning Curves.

Thực tế người ta luôn muốn loss càng thấp càng tốt và validation loss và training loss càng gần nhau càng tốt.

→ thực tế mỗi lần tính training error thì phải plot ra 1 lần

### 8.6.1 Trường hợp High Bias (Underfitting)

Tình huống underfit

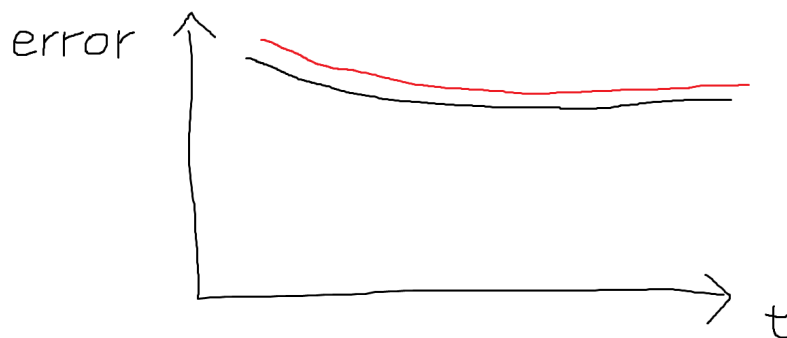


Figure 8.6: Underfit

Mô hình quá đơn giản (ví dụ đường thẳng cho dữ liệu cong).

- Khi  $m$  tăng,  $J_{train}$  tăng dần (khó khớp hết dữ liệu).
- $J_{cv}$  giảm nhưng hội tụ ở mức cao.
- **Kết luận:** Nếu mô hình bị High Bias, việc thêm dữ liệu huấn luyện **không** giúp ích nhiều.

### 8.6.2 Trường hợp High Variance (Overfitting)

Tình huống gặp nhiều nhất: training error gặp cực tiểu rất nhanh, sau đó loss nó càng ngày càng xuống  $\rightarrow$  overfit



Figure 8.7: Overfit

Mô hình quá phức tạp, khớp quá kỹ tập huấn luyện.

- $J_{train}$  rất thấp.
- $J_{cv}$  cao hơn nhiều so với  $J_{train}$  (có khoảng cách lớn - gap).
- Khi  $m$  tăng,  $J_{cv}$  có xu hướng giảm xuống và  $J_{train}$  tăng lên, khoảng cách thu hẹp lại.
- **Kết luận:** Nếu mô hình bị High Variance, việc thêm dữ liệu huấn luyện **có khả năng giúp ích**.

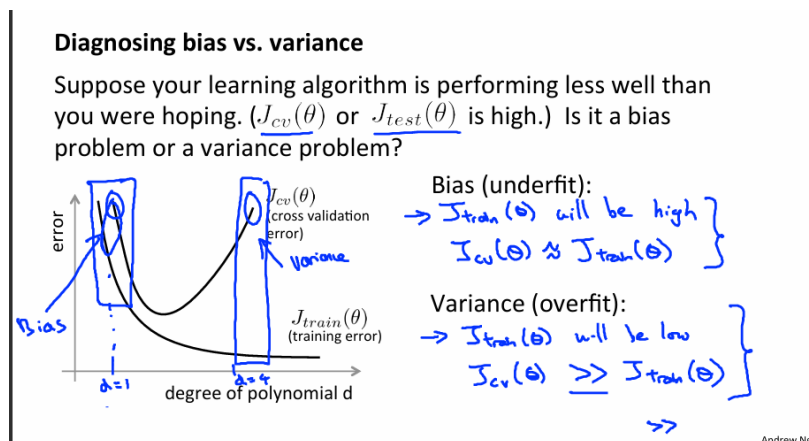


Figure 8.8: Diagnosing bias vs variance.



## 8.7 Tổng kết giải pháp (Deciding What to Do Next)

Quay lại các giải pháp ban đầu, ta có thể ánh xạ chúng vào từng vấn đề cụ thể:

| <b>Giải pháp</b>                                        | <b>Khắc phục vấn đề</b> |
|---------------------------------------------------------|-------------------------|
| Get more training examples (Thêm dữ liệu)               | <b>High Variance</b>    |
| Try smaller sets of features (Giảm đặc trưng)           | <b>High Variance</b>    |
| Try getting additional features (Thêm đặc trưng)        | <b>High Bias</b>        |
| Try adding polynomial features (Thêm đặc trưng bậc cao) | <b>High Bias</b>        |
| Try decreasing $\lambda$ (Giảm $\lambda$ )              | <b>High Bias</b>        |
| Try increasing $\lambda$ (Tăng $\lambda$ )              | <b>High Variance</b>    |

Table 8.1: Các giải pháp khắc phục dựa trên chẩn đoán mô hình

**Chỉ dùng bảng này là đủ để cải thiện model chưa ?**

- Chưa, cách này còn quá đơn giản.
- Đa phần chỉ là tune hyperparameters.
- Có nhiều trường hợp đặc biệt, cần phân tích rõ hơn mới có thể tìm ra giải pháp.

## Chapter 9

# Machine Learning System Design (Thiết kế hệ thống học máy)

### 9.1 Building a Spam Classifier (Xây dựng bộ phân loại thư rác)

#### 9.1.1 Problem Description (Mô tả bài toán)

Xét bài toán phân loại email là Spam (thư rác) hay Non-spam (thư thường). Đây là một bài toán **Supervised Learning** (Học có giám sát) điển hình.

- $y = 1$ : Spam.
- $y = 0$ : Non-spam.
- $x$ : Các **Features** (Đặc trưng) của email.

#### 9.1.2 Feature Representation (Biểu diễn đặc trưng)

Để biểu diễn một email thành vector đặc trưng  $x$ , ta thường chọn một danh sách các từ khóa (ví dụ: 100 từ) thường xuất hiện để phân biệt spam/non-spam (như: *deal*, *buy*, *discount*, *andrew*, *now*...).

Vector  $x \in R^{100}$  được xác định như sau:

$$x_j = \begin{cases} 1 & \text{nếu từ thứ } j \text{ xuất hiện trong email} \\ 0 & \text{nếu ngược lại} \end{cases} \quad (9.1)$$

**Lưu ý thực tế:** Thay vì chọn thủ công 100 từ, người ta thường lấy danh sách các từ xuất hiện nhiều nhất ( $n$  từ, từ 10,000 đến 50,000) trong tập dữ liệu huấn luyện.

#### 9.1.3 Debugging a Learning Algorithm (Gỡ lỗi thuật toán học)

Giả sử bạn đã triển khai thuật toán **Regularized Linear Regression** (Hồi quy tuyến tính có điều chuẩn) để dự đoán giá nhà, nhưng kết quả dự đoán trên dữ liệu mới có sai số quá lớn. Bạn nên làm gì?

Dưới đây là bảng tổng hợp các phương pháp và vấn đề mà chúng giải quyết:

| Phương pháp (Action)                                    | Khắc phục vấn đề (Fixes)           |
|---------------------------------------------------------|------------------------------------|
| Get more training examples (Thêm dữ liệu)               | <b>High Variance</b> (Overfitting) |
| Try smaller sets of features (Giảm đặc trưng)           | <b>High Variance</b> (Overfitting) |
| Try getting additional features (Thêm đặc trưng)        | <b>High Bias</b> (Underfitting)    |
| Try adding polynomial features (Thêm đặc trưng bậc cao) | <b>High Bias</b> (Underfitting)    |
| Try decreasing $\lambda$ (Giảm tham số $\lambda$ )      | <b>High Bias</b> (Underfitting)    |
| Try increasing $\lambda$ (Tăng tham số $\lambda$ )      | <b>High Variance</b> (Overfitting) |

Table 9.1: Các giải pháp khắc phục lỗi mô hình

Tuy nhiên, bảng trên khá đơn giản. Trong thực tế, có nhiều trường hợp đặc biệt (corner cases). Thay vì thử sai ngẫu nhiên, ta nên phân tích các mẫu bị lỗi (*error analysis*) để tìm nguyên nhân cụ thể.

## 9.2 Error Analysis (Phân tích lỗi)

Quy trình khuyến nghị để giải quyết một bài toán học máy:

1. Bắt đầu với một thuật toán đơn giản, cài đặt nhanh chóng (ví dụ trong 24 giờ).
2. Vẽ **Learning Curves** (Đường cong học tập) để quyết định xem có cần thêm dữ liệu hay thêm đặc trưng không.
3. Thực hiện **Error Analysis**: Kiểm tra thủ công các ví dụ (trong tập **Cross Validation**) mà thuật toán dự đoán sai.

### 9.2.1 Ví dụ thực tế

Giả sử  $m_{cv} = 500$  và thuật toán phân loại sai 100 email. Ta kiểm tra thủ công 100 email này và phân loại chúng dựa trên:

- **Loại email**: Pharma (Thuốc), Replica/Fake (Hàng giả), Steal passwords (Đánh cắp mật khẩu), Khác.
- **Dấu hiệu (Cues)**: Những đặc điểm nào có thể giúp thuật toán phân loại đúng?

Kết quả phân tích:

- Pharma: 12
- Replica: 4
- Steal passwords: 53
- Khác: 31

→ Lỗi chủ yếu đến từ loại "Steal passwords". Ta có thể tập trung cải thiện việc nhận diện loại này

ví dụ: Tìm xem loại này có điểm gì chung, ví dụ hay xuất hiện 1 chuỗi hay từ nào đó  
 => preprocessing (tiền xử lý, loại bỏ stop words) + rule-based + xét threshold -> không cần thay đổi weights.

## 9.3 Error Metrics for Skewed Classes (Độ đo lỗi cho lớp lệch)

### 9.3.1 Vấn đề với Accuracy (Độ chính xác)

Xét bài toán phân loại ung thư (**Cancer Classification**):  $y = 1$  (ung thư),  $y = 0$  (lành tính). Giả sử mô hình đạt 1% lỗi trên tập test (99% Accuracy). Tuy nhiên, thực tế chỉ có 0.5% bệnh nhân bị ung thư. Đây là trường hợp **Skewed Classes** (Lớp dữ liệu bị lệch).

Nếu ta viết một hàm đơn giản luôn dự đoán  $y = 0$  (luôn lành tính):

```
function predictCancer(x):  
    return 0
```

Hàm này sẽ đạt độ chính xác 99.5% (chỉ sai 0.5%). Rõ ràng **Accuracy** không phải là thước đo tốt trong trường hợp này.

## 9.4 Precision and Recall (Độ chính xác và Độ phủ)

Để đánh giá tốt hơn, ta sử dụng **Precision** và **Recall**. Ta xây dựng **Confusion Matrix** (Ma trận nhầm lẫn):

|              | Actual: 1           | Actual: 0           |
|--------------|---------------------|---------------------|
| Predicted: 1 | True Positive (TP)  | False Positive (FP) |
| Predicted: 0 | False Negative (FN) | True Negative (TN)  |

### 9.4.1 Định nghĩa

- **Precision (Độ chính xác):** Trong số các bệnh nhân được dự đoán ung thư ( $y = 1$ ), bao nhiêu phần trăm thực sự bị ung thư?

$$\text{Precision} = \frac{\text{True Positives}}{\text{Total Predicted Positive}} = \frac{TP}{TP + FP} \quad (9.2)$$

- **Recall (Độ phủ/Độ nhạy):** Trong số tất cả bệnh nhân thực sự bị ung thư, bao nhiêu phần trăm chúng ta phát hiện được?

$$\text{Recall} = \frac{\text{True Positives}}{\text{Total Actual Positive}} = \frac{TP}{TP + FN} \quad (9.3)$$

Nếu mô hình luôn dự đoán  $y = 0$ , thì  $TP = 0 \rightarrow \text{Recall} = 0$ , ta biết ngay mô hình này vô dụng.

## 9.5 Trading off Precision and Recall (Đánh đổi giữa Precision và Recall)

Đối với Logistic Regression, ta dự đoán  $y = 1$  nếu  $h_{\theta}(x) \geq \text{threshold}$  (ngưỡng).

- **Tăng ngưỡng (ví dụ 0.7, 0.9):**
  - Chỉ dự đoán ung thư khi rất tự tin.

– Kết quả: **High Precision** (ít báo động giả), **Low Recall** (bỏ sót bệnh nhân).

- **Giảm ngưỡng (ví dụ 0.3, 0.1):**

– Tránh bỏ sót các ca ung thư (tránh False Negatives).

– Kết quả: **High Recall** (phát hiện được nhiều), **Low Precision** (nhiều báo động giả).

## 9.6 F1 Score (Điểm F1)

Làm thế nào để so sánh các thuật toán với các cặp số Precision/Recall khác nhau?

Nếu dùng trung bình cộng  $\frac{P+R}{2}$ , ta có thể bị lừa bởi các thuật toán cực đoan (ví dụ Recall = 1 nhưng Precision cực thấp).

Ta sử dụng **F1 Score** (Trung bình điều hòa của P và R):

$$F_1 \text{ Score} = 2 \frac{P \cdot R}{P + R} \quad (9.4)$$

- Nếu  $P = 0$  hoặc  $R = 0 \rightarrow F_1 = 0$ .

- Nếu  $P = 1$  và  $R = 1 \rightarrow F_1 = 1$ .

### 9.6.1 Tổng quát: $F_\beta$ Score

Công thức tổng quát cho phép điều chỉnh trọng số giữa Precision và Recall:

$$F_\beta \text{ Score} = (1 + \beta^2) \frac{P \cdot R}{(\beta^2 \cdot P) + R} \quad (9.5)$$

- $\beta = 1$ : Cân bằng (F1 Score).

- $\beta = 0.5$ : Ưu tiên Precision.

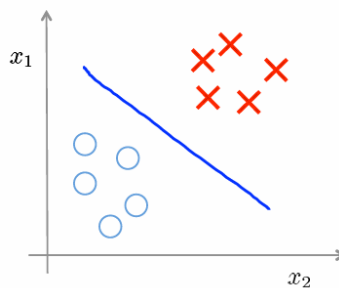
- $\beta = 2$ : Ưu tiên Recall.

## Chapter 10

# Clustering (Phân cụm)

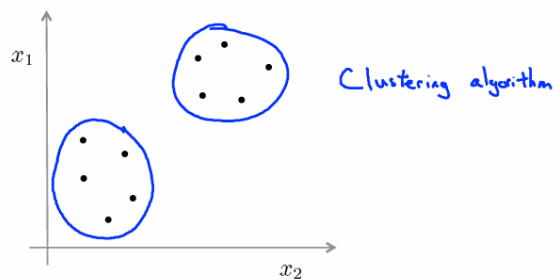
### 10.1 Giới thiệu về Clustering

#### Supervised learning



Training set:  $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), (x^{(3)}, y^{(3)}), \dots, (x^{(m)}, y^{(m)})\}$  ←

#### Unsupervised learning



Training set:  $\{\underline{x^{(1)}}, \underline{x^{(2)}}, \underline{x^{(3)}}, \dots, \underline{x^{(m)}}\}$  ←

Figure 10.1: Supervised Learning vs Unsupervised Learning.

Clustering là một họ thuật toán lớn thuộc nhóm **Unsupervised Learning** (Học không giám sát). Khác với Supervised Learning (có nhãn  $y$ ), Unsupervised Learning làm việc với tập dữ liệu huấn luyện không có nhãn:

$$\{x^{(1)}, x^{(2)}, \dots, x^{(m)}\}$$

Mục tiêu của Clustering là tìm kiếm các mẫu ẩn (hidden patterns) hoặc cấu trúc trong dữ liệu để nhóm các điểm dữ liệu tương đồng lại với nhau.

### 10.1.1 Các ứng dụng của Clustering

- **Market Segmentation (Phân khúc thị trường):** Phân nhóm khách hàng dựa trên dữ liệu nhân khẩu học, hành vi mua sắm để có chiến lược kinh doanh phù hợp -> đây là một trong các bài toán kinh điển của Unsupervised Learning.
- **Social Network Analysis (Phân tích mạng xã hội):** Tìm các cộng đồng hoặc nhóm người dùng có liên kết chặt chẽ.
- **Organize Computing Clusters:** Tối ưu hóa việc sắp xếp các cụm máy tính.
- **Astronomical Data Analysis:** Phân tích dữ liệu thiên văn.
- **Anomaly Detection (Phát hiện bất thường):** Khi dữ liệu có rất ít hoặc không có nhãn bất thường, Clustering có thể giúp phát hiện các điểm dữ liệu sai khác so với phần còn lại.

## 10.2 Thuật toán K-means

K-means là thuật toán phân cụm đơn giản và phổ biến nhất. Thuật toán hoạt động lặp đi lặp lại để chia dữ liệu thành  $K$  cụm.

### 10.2.1 Quy trình thuật toán

**Input:**

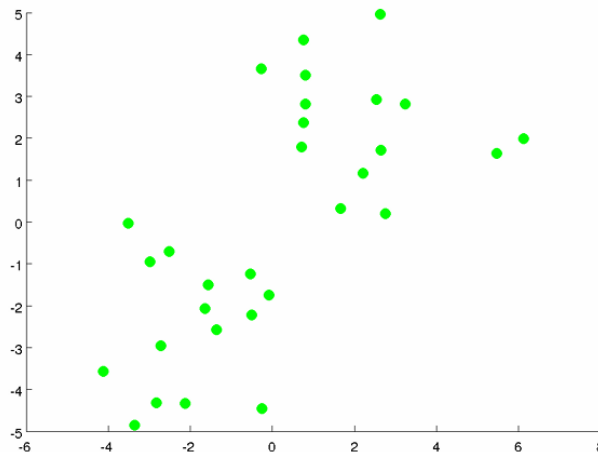


Figure 10.2: K-means input.

- $K$ : Số lượng cụm mong muốn.
- Tập dữ liệu huấn luyện  $\{x^{(1)}, x^{(2)}, \dots, x^{(m)}\}$  (với  $x^{(i)} \in R^n$ ).

### Các bước thực hiện:

1. **Initialization (Khởi tạo tâm cụm):** Với mỗi cụm sẽ có một centroids, là vị trí chính giữa của cụm. Chọn ngẫu nhiên  $K$  điểm làm tâm cụm ban đầu (cluster centroids), hoặc tốt hơn thì dựa trên phân bố của dữ liệu  $\rightarrow$  khởi tạo được  $K$  cluster centroids  $\mu_1, \mu_2, \dots, \mu_K$ .

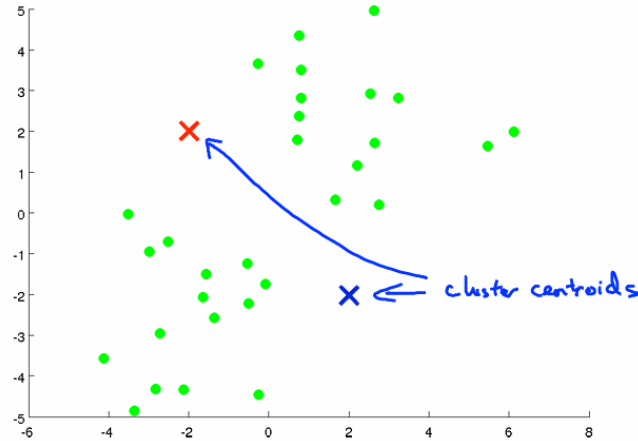


Figure 10.3: K-means khởi tạo tâm cụm.

### 2. Loop (Lặp lại cho đến khi hội tụ):

- **Cluster Assignment Step (Gán cụm):** Duyệt qua từng điểm dữ liệu  $x^{(i)}$ , tính khoảng cách tới tất cả các tâm cụm  $\mu_k$ . Gán  $x^{(i)}$  vào cụm có tâm gần nhất.

$$c^{(i)} := \arg \min_k ||x^{(i)} - \mu_k||^2$$

Trong đó  $c^{(i)}$  là chỉ số cụm (từ 1 đến  $K$ ) của điểm  $x^{(i)}$ .

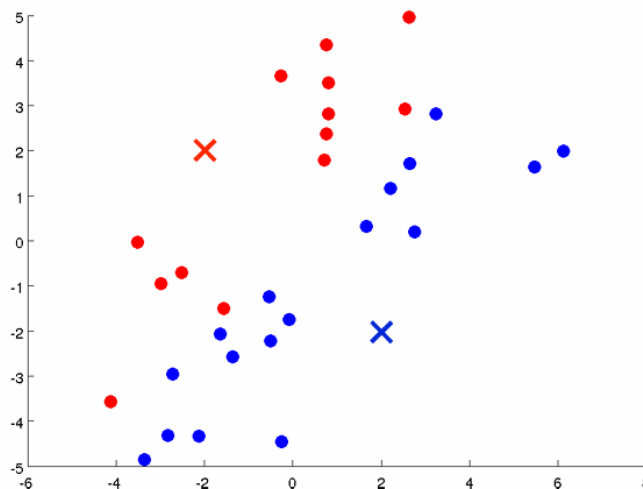


Figure 10.4: K-means phân cụm cho các điểm dữ liệu.



- **Move Centroids Step (Cập nhật tâm cụm):** Với mỗi cụm  $k$ , tính trung bình (mean) của tất cả các điểm dữ liệu đã được gán vào cụm đó và cập nhật  $\mu_k$  thành giá trị trung bình mới.

$$\mu_k := \frac{1}{|C_k|} \sum_{i \in C_k} x^{(i)}$$

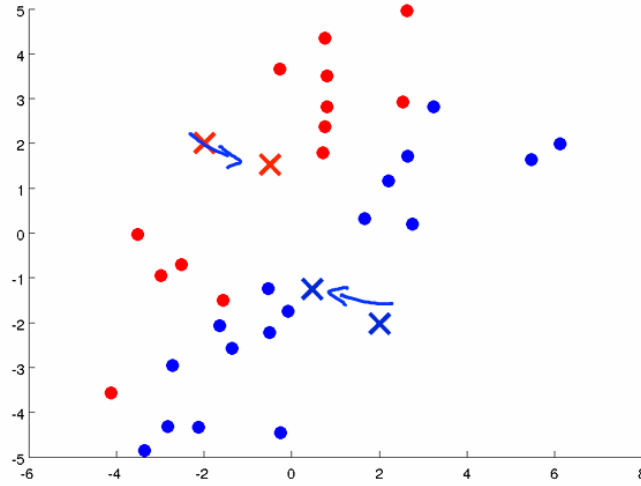


Figure 10.5: K-means cập nhật tâm cụm.

Nếu tâm cụm vẫn còn dịch chuyển thì tiếp tục vòng lặp (quay về Cluster Assignment Step)

Thuật toán dừng lại khi các tâm cụm không còn thay đổi (hội tụ).

### 10.3 Hàm Mục tiêu (Optimization Objective)

K-means thực chất là quá trình tối ưu hóa một hàm chi phí, gọi là **Distortion function** (Hàm độ méo dạng).

Ký hiệu:

- $c^{(i)}$ : Chỉ số cụm của điểm dữ liệu  $x^{(i)}$ .

$$c^{(i)} := \arg \min_k \|x^{(i)} - \mu_k\|^2$$

- $\mu_{c^{(i)}}$ : Tâm cụm tương ứng với điểm  $x^{(i)}$  (tâm của cụm chứa điểm  $x^{(i)}$ ).

Hàm tối ưu  $J$ :

$$J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K) = \frac{1}{m} \sum_{i=1}^m \|x^{(i)} - \mu_{c^{(i)}}\|^2 \quad (10.1)$$

Mục tiêu của K-means là tìm các  $c^{(i)}$  và  $\mu_k$  để cực tiểu hóa  $J$ :

$$\min_{c, \mu} J(c, \mu)$$

## 10.4 Chứng minh hàm tối ưu:

Tại sao Cluster Assignment Step lại tương đương việc tìm điểm dữ liệu  $i$  thuộc cụm nào từ  $c^{(1)}$  đến  $c^{(m)}$  để làm sao minimize được  $J$  ?

$$J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K) = \frac{1}{m} \sum_{i=1}^m \|x^{(i)} - \mu_{c(i)}\|^2 \quad (10.2)$$

Điều kiện để tối ưu hàm  $J$  là để tối ưu  $m$  cái  $c^{(i)}$ , chứ không phải tối ưu mỗi 1 điểm  $c$ .

→ phải chứng minh được việc tối ưu hàm  $J$  đồng thời các  $c_i$  thì = tối ưu độc lập lần lượt các  $c$

→ phải chứng minh được đẳng thức sau:

$$\min_{c^i} J(c^{(1)}, \dots, c^{(m)}, \mu_1, \dots, \mu_K) = \min_{c^i} \left( \frac{1}{m} \sum_{i=1}^m \|x^{(i)} - \mu_{c(i)}\|^2 \right) = \frac{1}{m} \sum_{i=1}^m \min_{c^i} \|x^{(i)} - \mu_{c(i)}\|^2 \quad (10.3)$$

Đẳng thức này đúng khi loss  $J$  cho các điểm  $i$  độc lập với nhau, tức là  $\|x^{(i)} - \mu_{c(i)}\|^2$  độc lập với nhau.

-> Chúng độc lập với nhau khi cập nhật đồng thời và cập nhật độc lập  $c(i)$  là như nhau

-> Hay là công thức sau độc lập với mỗi  $i$

$$c^{(i)} = \arg \min_{k \in \{1, \dots, K\}} \|x^{(i)} - \mu_k\|^2$$

-> Điều này là hiển nhiên.

=> **Cluster Assignment Step** chính là cố định  $\mu$  và tối ưu  $c^{(i)}$  để giảm  $J$ .

Chứng minh tương tự => **Move Centroids Step** chính là cố định  $c^{(i)}$  và tối ưu  $\mu$  để giảm  $J$ .

Từ đó suy ra:

- **Cluster Assignment Step** chính là cố định  $\mu$  và tối ưu  $c^{(i)}$  để giảm  $J$ .
- **Move Centroids Step** chính là cố định  $c^{(i)}$  và tối ưu  $\mu$  để giảm  $J$ .

## 10.5 Khởi tạo ngẫu nhiên (Random Initialization)

Kết quả của K-means có thể phụ thuộc vào việc khởi tạo tâm cụm ban đầu và có nguy cơ rơi vào **Local Optima** (Cực trị địa phương), dẫn đến kết quả phân cụm không tốt.

**Giải pháp:** Thực hiện K-means nhiều lần (ví dụ: 50-100 lần) với các khởi tạo ngẫu nhiên khác nhau.

1. For  $i = 1$  to 100:
2. Khởi tạo ngẫu nhiên  $K$  tâm cụm.
3. Chạy K-means để nhận được  $c^{(i)}$  và  $\mu_k$ .
4. Tính giá trị hàm chi phí  $J$ .
5. Cuối cùng, chọn kết quả có giá trị  $J$  nhỏ nhất.

Trên thực tế người ta sẽ không dùng khởi tạo random mà khởi tạo có phân tích dữ liệu hơn.

## 10.6 Lựa chọn số cụm $K$ (Choosing the Value of $K$ )

Việc chọn  $K$  thường không dễ dàng và đôi khi mang tính chủ quan hoặc phụ thuộc vào bài toán cụ thể.

### 10.6.1 Phương pháp Elbow (Khuỷu tay)

Vẽ đồ thị giá trị hàm chi phí  $J$  theo số lượng cụm  $K$ .

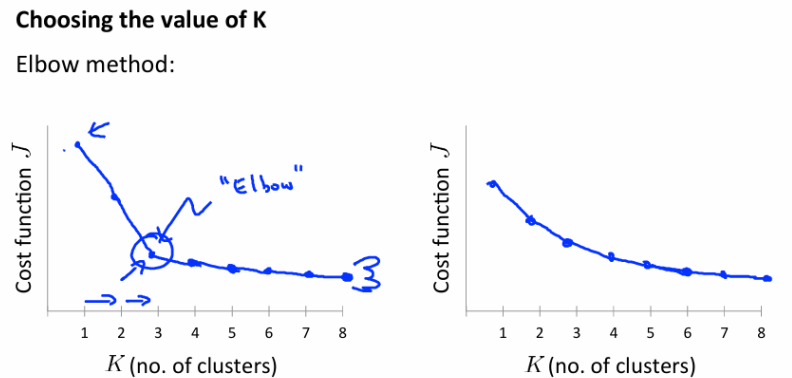


Figure 10.6: Elbow method

- Khi  $K$  tăng,  $J$  sẽ giảm.
- Ta tìm điểm "khuỷu tay" (elbow point) - nơi mà tốc độ giảm của  $J$  bắt đầu chậm lại đáng kể. Đó có thể là giá trị  $K$  hợp lý.

*Lưu ý:* Đôi khi đồ thị giảm đều và không xuất hiện điểm khuỷu tay rõ ràng, khi đó phương pháp này không hiệu quả.

-> khi đó cần phân tích dữ liệu và downstream task để chọn  $k$  phù hợp.

### 10.6.2 Dựa trên mục đích sử dụng (Downstream Purpose)

Chọn  $K$  dựa trên nhu cầu thực tế của bài toán kinh doanh.

**Ví dụ T-shirt sizing:**

### Choosing the value of K

Sometimes, you're running K-means to get clusters to use for some later/downstream purpose. Evaluate K-means based on a metric for how well it performs for that later purpose.

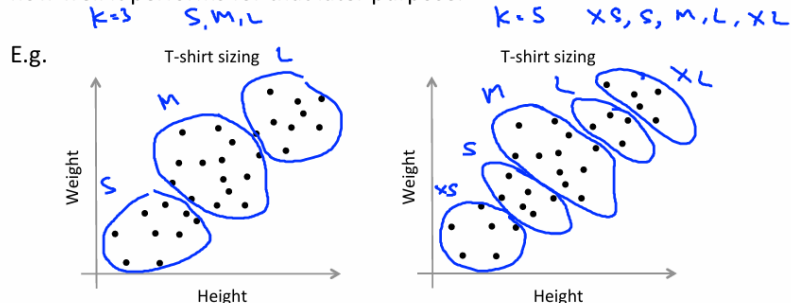


Figure 10.7: Chọn k cho K-Means.

- Nếu muốn làm 3 cỡ áo (S, M, L) → Chọn  $K = 3$ .
- Nếu muốn làm 5 cỡ áo (XS, S, M, L, XL) → Chọn  $K = 5$ .

**Kết luận:** Với các dạng phân bố phức tạp hơn (ví dụ: hình cánh hoa, lồng vào nhau), cần xem xét các thuật toán phân cụm khác.

### 10.6.3 Khi nào thì sử dụng K-Means ?

K-means hoạt động tốt nhất với các cụm dữ liệu có dạng hình cầu (spherical) vì tâm của dữ liệu hình cầu là tâm của hình cầu đó → không cần cập nhật tâm cụm nhiều.

**Khi nào thì người ta chọn làm k-means ?**

- Phân tích kĩ dữ liệu, xem dữ liệu có dạng gì, có phù hợp với k-means không
- Nó phân bố nhiều hay 1 dạng cầu
- Nếu 1 dạng cầu thường k phù hợp lắm → thành 1 tâm → người ta muốn nhiều dạng cầu
- Hoặc là nó có 1 dạng cầu, nhưng kiểu hình hơi cánh hoa, thì phù hợp k-means
- Xác định k là phần nan giải của k-means
- Có thể sử dụng các thuật toán phân cụm khác

# Chapter 11

## Dimensionality Reduction (Giảm chiều dữ liệu)

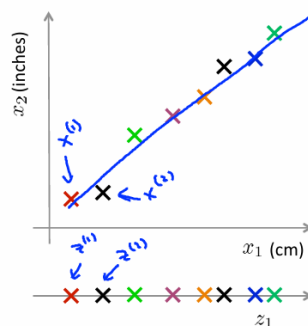
### 11.1 Giới thiệu về Dimensionality Reduction

#### 11.1.1 Động lực (Motivation)

Giảm chiều dữ liệu (Dimensionality Reduction) là kỹ thuật quan trọng trong Machine Learning với hai mục đích chính:

1. Data Compression (Nén dữ liệu):

#### Data Compression



Reduce data from  
2D to 1D

$$\begin{aligned} x^{(1)} \in \mathbb{R}^2 &\rightarrow z^{(1)} \in \mathbb{R} \\ x^{(2)} \in \mathbb{R}^2 &\rightarrow z^{(2)} \in \mathbb{R} \\ &\vdots \\ x^{(m)} \in \mathbb{R}^2 &\rightarrow z^{(m)} \in \mathbb{R} \end{aligned}$$

#### Data Compression

10000  $\rightarrow$  1000

Reduce data from 3D to 2D

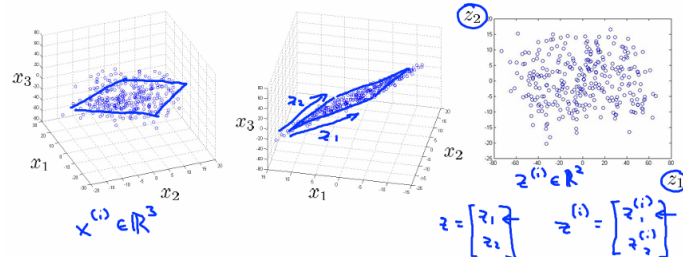


Figure 11.1: Minh họa Data Compression.

- Giảm không gian lưu trữ (bộ nhớ/ổ đĩa).
- Tăng tốc độ thuật toán học máy (*Supervised learning speedup*).
- Loại bỏ các đặc trưng dư thừa (redundant features) hoặc có tương quan cao (ví dụ: độ dài tính bằng cm và inches).

## 2. Data Visualization (Trực quan hóa dữ liệu):

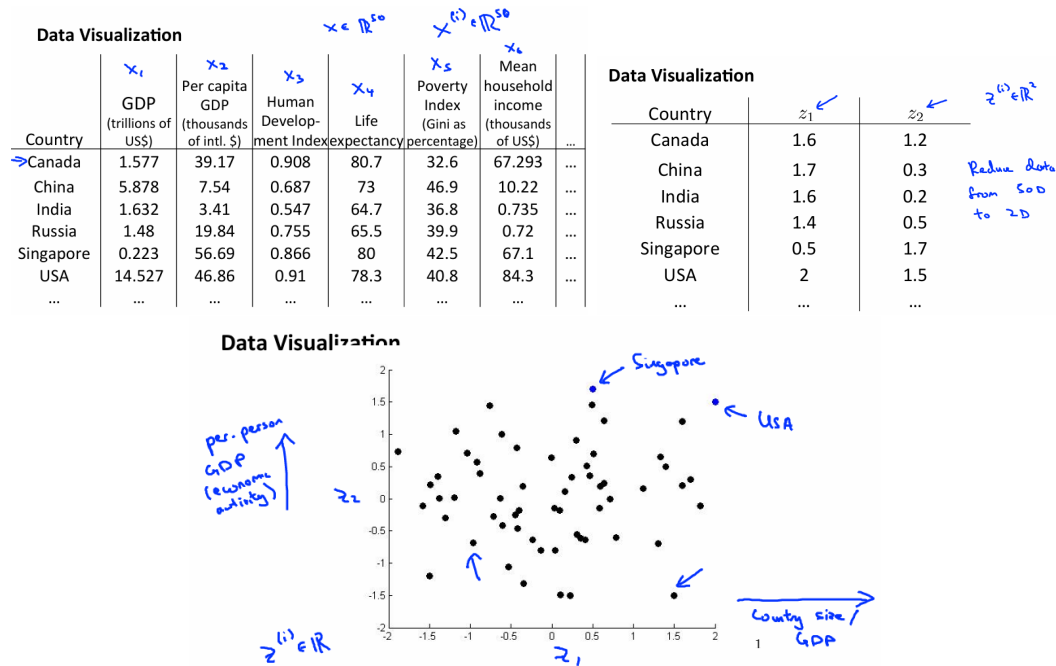


Figure 11.2: Minh họa Data Visualization.

- Dữ liệu nhiều chiều (ví dụ 50D, 1000D) rất khó hình dung.
- Giảm xuống 2D hoặc 3D giúp ta vẽ đồ thị và hiểu rõ hơn về cấu trúc dữ liệu.

### 11.1.2 Ví dụ minh họa

**Từ 2D xuống 1D:** Xét các điểm dữ liệu nằm rải rác gần một đường thẳng. Ta có thể chiếu chúng xuống đường thẳng đó để chuyển từ vector  $x^{(i)} \in \mathbb{R}^2$  sang một số thực  $z^{(i)} \in \mathbb{R}$ .

**Từ 3D xuống 2D:** Chiếu các điểm dữ liệu trong không gian 3D xuống một mặt phẳng 2D phù hợp nhất.

## 11.2 Principal Component Analysis (PCA)

### 11.2.1 Định nghĩa bài toán

PCA (Phân tích thành phần chính) là thuật toán giảm chiều dữ liệu phổ biến nhất.

Mục tiêu của PCA: Tìm một không gian con (đường thẳng, mặt phẳng, siêu phẳng...) có số chiều thấp hơn sao cho khi chiếu dữ liệu lên đó, tổng bình phương khoảng cách từ các điểm dữ liệu đến hình chiếu của nó (**Projection Error**) là nhỏ nhất.

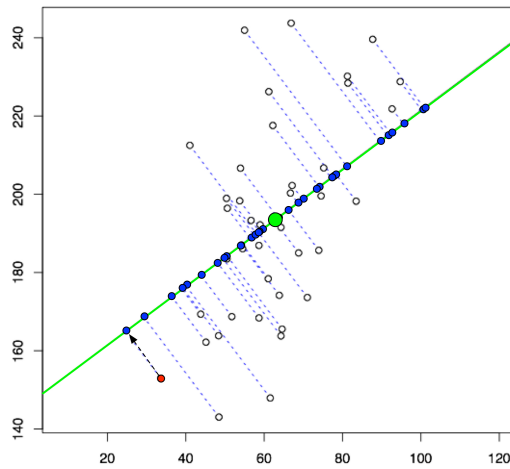
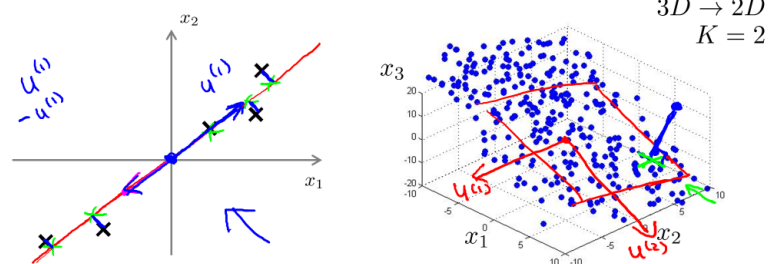


Figure 11.3: Minh họa ý tưởng của thuật toán PCA 2 chiều  $\rightarrow$  1 chiều.

Giảm  $n \rightarrow k$  chiều thì tìm  $k$  vector sao cho khi chiếu các điểm dữ liệu xuống hyperplane thì khoảng cách chiếu là nhỏ nhất (minimize projection error).

#### Principal Component Analysis (PCA) problem formulation



Reduce from 2-dimension to 1-dimension: Find a direction (a vector  $u^{(1)} \in \mathbb{R}^n$ ) onto which to project the data so as to minimize the projection error.

Reduce from  $n$ -dimension to  $k$ -dimension: Find  $k$  vectors  $u^{(1)}, u^{(2)}, \dots, u^{(k)}$  onto which to project the data, so as to minimize the projection error.

Andrew Ng

Figure 11.4: Minh họa ý tưởng của thuật toán PCA.

### Sự khác biệt giữa PCA và Linear Regression:

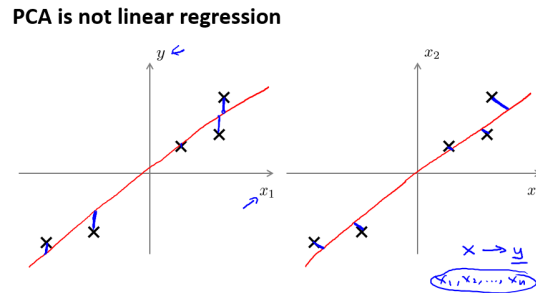


Figure 11.5: Minh họa sự khác biệt giữa PCA và Linear Regression.

- **Linear Regression:** Tìm đường thẳng để cực tiểu hóa sai số dự đoán theo phương dọc (vuông góc với trục hoành). Cố gắng dự đoán giá trị  $y$  từ  $x$ .
- **PCA:** Tìm đường thẳng để cực tiểu hóa khoảng cách vuông góc từ điểm dữ liệu đến đường thẳng đó. Không có phân biệt biến  $x$  và  $y$  (tất cả đều là features).

Tại sao khi giảm  $n$  chiều dữ liệu xuống  $k$  chiều dữ liệu, chúng ta lại cần tìm  $k$  vectors sao cho khi chiếu xuống, khoảng cách hình chiếu (projection error) là nhỏ nhất ?

- Giảm chiều khi các chiều có lượng thông tin đóng góp nhưng không nhiều.
- Yếu tố tiên quyết khi giảm chiều dữ liệu là giảm mất mát thông tin của dữ liệu  $\rightarrow$  dữ liệu ban đầu phải không thay đổi nhiều (ít dịch chuyển nhất).
- Tương đương với việc tìm vector để khoảng cách hình chiếu là nhỏ nhất.

PCA giảm chiều dữ liệu theo hướng mà variance của dữ liệu lớn nhất.

#### 11.2.2 Các bước thực hiện PCA

**Bước 1: Data Preprocessing (Tiền xử lý dữ liệu)** Trước khi chạy PCA, bắt buộc phải thực hiện chuẩn hóa dữ liệu:

- **Mean Normalization:** Đặt trung bình của mỗi feature về 0.

$$\mu_j = \frac{1}{m} \sum_{i=1}^m x_j^{(i)}$$

$$x_j^{(i)} \leftarrow x_j^{(i)} - \mu_j$$

- **Feature Scaling:** Nếu các feature có độ lớn khác nhau (ví dụ: diện tích nhà vs số phòng ngủ), cần chia cho độ lệch chuẩn hoặc khoảng giá trị để chúng có cùng tỉ lệ.

$$x_j^{(i)} \leftarrow \frac{(x_j^{(i)} - \mu_j)}{s_j}$$



**Bước 2: Tính Covariance Matrix (Ma trận hiệp phương sai)** Tính ma trận  $\Sigma$  (Sigma):

$$\Sigma = \frac{1}{m} \sum_{i=1}^m (x^{(i)})(x^{(i)})^T = \frac{1}{m} X^T X \quad (11.1)$$

**Bước 3: Tính Eigenvectors và Eigenvalues** Sử dụng phân tích SVD (Singular Value Decomposition) hoặc 'eig' để tìm các vector riêng và trị riêng của ma trận  $\Sigma$ :

$$[U, S, V] = \text{svd}(\Sigma)$$

Trong đó  $U$  là ma trận các vector riêng (Principal Components).

**Bước 4: Chọn K Eigenvectors ứng với K Eigenvalues lớn nhất** Để giảm từ  $n$  chiều xuống  $k$  chiều, ta chọn  $k$  cột đầu tiên của ma trận  $U$  (gọi là  $U_{\text{reduce}}$ ).

**Bước 5: Chiếu dữ liệu** Tính vector mới  $z^{(i)}$  trong không gian  $k$  chiều:

$$z^{(i)} = U_{\text{reduce}}^T x^{(i)} \quad (11.2)$$

## 11.3 Bản chất PCA

1. Tại sao trước khi áp dụng PCA, chúng ta cần bắt buộc làm mean normalization ? nếu không thì sao ?

- Việc normalization để chuẩn hóa giúp variance nó không phụ thuộc nhiều vào giá trị lớn (1 triệu VND và 10 độ thì 1 triệu » 10 nhưng độ chênh lệch thực tế giữa 2 đơn vị khác nhau là khác nhau).
- Cái mà chúng ta quan tâm chỉ là độ biến thiên dữ liệu → nếu chiếu dữ liệu theo chiều có độ biến thiên lớn nhất thì sẽ giữ được nhiều thông tin nhất (ma trận hiệp phương sai chứa đầy đủ độ biến thiên của dữ liệu).
- Nếu không mean normalization, thì khi PCA với mong muốn tìm chiều có độ biến thiên nhiều nhất thì khả năng rất cao là 1 trong các vector riêng sẽ hướng dữ liệu về gốc (mất đi 1 chiều để giữ thông tin) vì ma trận hiệp phương sai sẽ có dạng như hình.

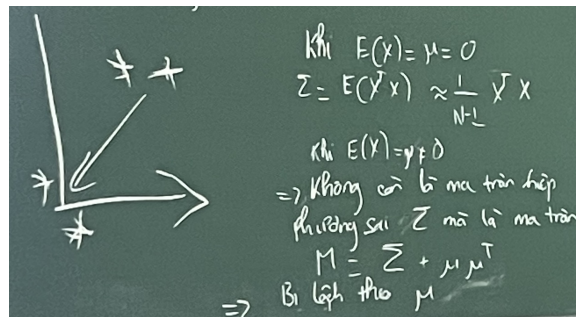
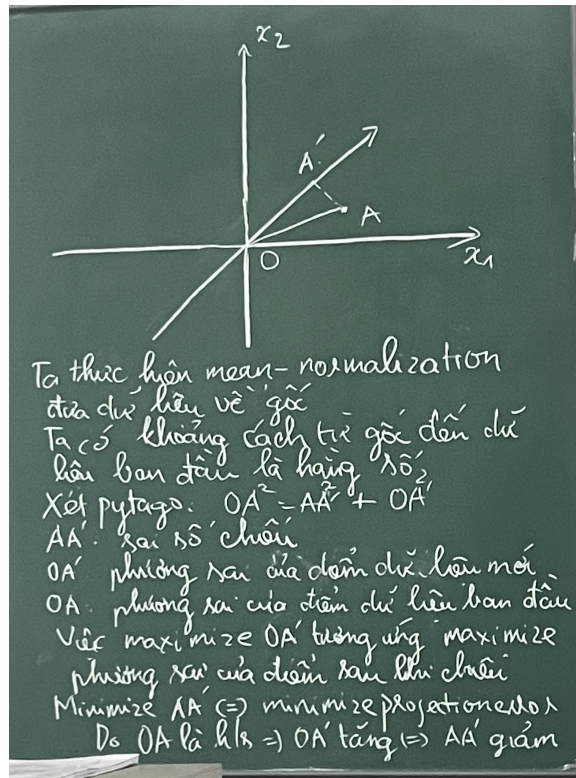


Figure 11.6: Minh họa PCA nếu không Normalization.

2. tại sao việc maximize variance  $\Leftrightarrow$  minimize projection error ?



$$\text{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)]$$

An alternative expression of  $\text{Cov}(X, Y)$

$$\begin{aligned} \text{Cov}(X, Y) &= E[\{X - E(X)\}\{Y - E(Y)\}] \\ &= E[X\{Y - E(Y)\} - E(X)\{Y - E(Y)\}] \\ &= E[XY - XE(Y)] = E(XY) - E(X)E(Y) \end{aligned}$$

Figure 11.7: Maximize variance vs minimize projection error.

Variance + projection error là 1 hằng số  $\rightarrow$  muốn giảm projection error thì phải tăng variance

## 11.4 Lựa chọn số chiều K (Choosing K)

Làm sao để chọn  $k$  phù hợp? Thông thường, ta chọn  $k$  nhỏ nhất sao cho giữ lại được phần lớn phương sai (variance) của dữ liệu (ví dụ: 99% variance retained).

Công thức kiểm tra:

$$\frac{\frac{1}{m} \sum_{i=1}^m \|x^{(i)} - x_{approx}\|^2}{\frac{1}{m} \sum_{i=1}^m \|x^{(i)}\|^2} \leq 0.01$$

(Tỉ số là sai số chiều trung bình, mẫu số là tổng phương sai dữ liệu).

## 11.5 Lời khuyên khi áp dụng PCA

### 11.5.1 Khi nào nên dùng PCA?

- Để nén dữ liệu nhằm giảm bộ nhớ hoặc tăng tốc thuật toán học máy.
- Để trực quan hóa dữ liệu (chọn  $k = 2$  hoặc  $k = 3$ ).

### 11.5.2 Sai lầm thường gặp (Bad use of PCA)

**KHÔNG dùng PCA để chống Overfitting:** Mặc dù PCA giảm số lượng features (giống như feature selection), nhưng nó không quan tâm đến nhãn  $y$ . Nó chỉ dựa trên phương sai của  $x$ . Do đó, nó có thể loại bỏ các thông tin quan trọng giúp phân biệt các lớp. → Để giải quyết Overfitting, hãy dùng **Regularization** thay vì PCA.

**Lưu ý quy trình:** Chỉ tính toán các tham số PCA (ma trận  $U_{reduce}$ ) trên tập **Training Set**, sau đó áp dụng cùng tham số đó để biến đổi tập Cross Validation và Test Set.

## Chapter 12

# Anomaly Detection (Phát hiện bất thường)

### 12.1 Giới thiệu về Anomaly Detection

Anomaly Detection là một bài toán quan trọng trong Machine Learning, được ứng dụng rộng rãi trong nhiều lĩnh vực thực tế như phát hiện gian lận, giám sát hệ thống, và sản xuất công nghiệp.

#### 12.1.1 Ví dụ minh họa: Động cơ máy bay

Giả sử ta có tập dữ liệu về các động cơ máy bay với các đặc trưng:

- $x_1$ : Nhiệt lượng tỏa ra (heat generated).
- $x_2$ : Cường độ rung (vibration intensity).

Tập dữ liệu:  $\{x^{(1)}, x^{(2)}, \dots, x^{(m)}\}$ . Khi có một động cơ mới  $x_{test}$ , ta cần xác định xem nó có bất thường (anomaly) hay không.

#### Anomaly detection example

Aircraft engine features:

- $x_1$  = heat generated
- $x_2$  = vibration intensity
- ...

Dataset:  $\{x^{(1)}, x^{(2)}, \dots, x^{(m)}\}$

New engine:  $x_{test}$

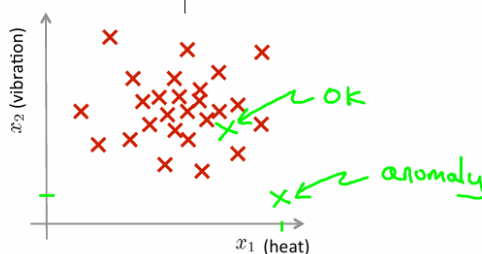


Figure 12.1: Ví dụ về Anomaly Detextion.

### 12.1.2 Phương pháp Density Estimation (Ước lượng mật độ)

#### Density estimation

→ Dataset:  $\{x^{(1)}, x^{(2)}, \dots, x^{(m)}\}$

→ Is  $x_{test}$  anomalous?

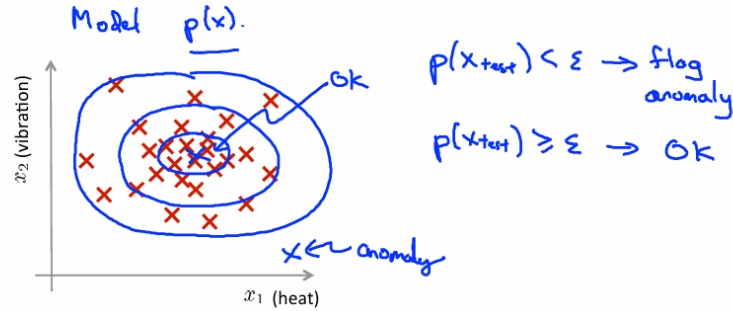


Figure 12.2: Density estimation cho bài toán Anomaly Detection.

Mô hình hóa  $p(x)$  (xác suất xuất hiện của  $x$ ) dựa trên tập dữ liệu đã có.

- Nếu  $p(x_{test}) < \epsilon$ : Đánh dấu là bất thường (Anomaly).
- Nếu  $p(x_{test}) \geq \epsilon$ : Đánh dấu là bình thường (OK).

*Lưu ý về khái niệm:* Trong miền liên tục, xác suất tại một điểm cụ thể xấp xỉ bằng 0. Do đó, ta sử dụng khái niệm **Density** (mật độ xác suất) thay vì Probability, Density của một điểm sẽ là Propability của các điểm xung quanh nó (tích tích phân). Những điểm bất thường thường có mật độ phân bố thấp (hiếm khi xảy ra).

## 12.2 Các ứng dụng phổ biến

### 1. Fraud Detection (Phát hiện gian lận):

- $x^{(i)}$ : Các đặc trưng hoạt động của người dùng  $i$ .
- Xây dựng mô hình  $p(x)$  từ dữ liệu người dùng bình thường.
- Xác định các hành vi bất thường khi  $p(x) < \epsilon$ .

### 2. Manufacturing (Sản xuất): Kiểm tra sản phẩm lỗi.

### 3. Monitoring Data Centers (Giám sát trung tâm dữ liệu):

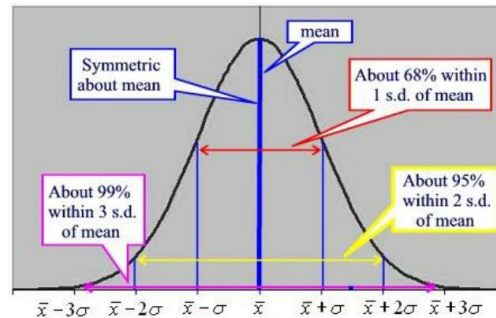
- $x_1$ : Sử dụng bộ nhớ.
- $x_2$ : Số lượng truy cập đĩa/giây.
- $x_3$ : Tải CPU.
- $x_4$ : Tải mạng.

## 12.3 Gaussian (Normal) Distribution

### 12.3.1 Hàm mật độ xác suất (PDF)

#### Normal (Gaussian) distribution

- Normal density function  $f_x(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right]$



- What does the figure tell us about the values of the CDF?



Figure 12.3: Normal Gaussian Distribution.

( $f(x)$  ở đây là công thức đã được tính ra từ phép tích phân  
Phân phối chuẩn Gaussian với trung bình  $\mu$  và phương sai  $\sigma^2$  được ký hiệu là  $\mathcal{N}(\mu, \sigma^2)$ .  
Công thức hàm mật độ:

$$p(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right) \quad (12.1)$$

- $\mu$  (Mean): Xác định vị trí trung tâm của phân phối.
- $\sigma$  (Standard Deviation - Độ lệch chuẩn): Xác định độ rộng (độ phân tán) của phân phối.

### 12.3.2 Gaussian Distribution Example)

- Toàn bộ phân phối = 1.
- Standard deviation (độ lệch chuẩn) =  $\sqrt{\text{variance}}$
- Standard deviation ảnh hưởng đến độ thưa của phân bố (khoảng cách các điểm dữ liệu và mean của nó).
- Khi  $\sigma$  giảm thì chiều cao phải tăng lên là do nó luôn phải giữ cho diện tích = 1.

### Gaussian distribution example

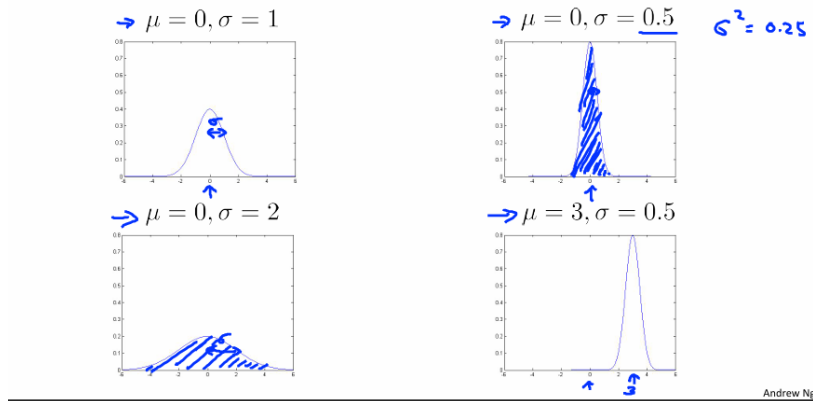


Figure 12.4: Gaussian Distribution Example.

### 12.3.3 Ước lượng tham số (Parameter Estimation)

Để biểu diễn phân bố của 1 tập dữ liệu, ta cần ước lượng các tham số như  $\mu, \sigma$

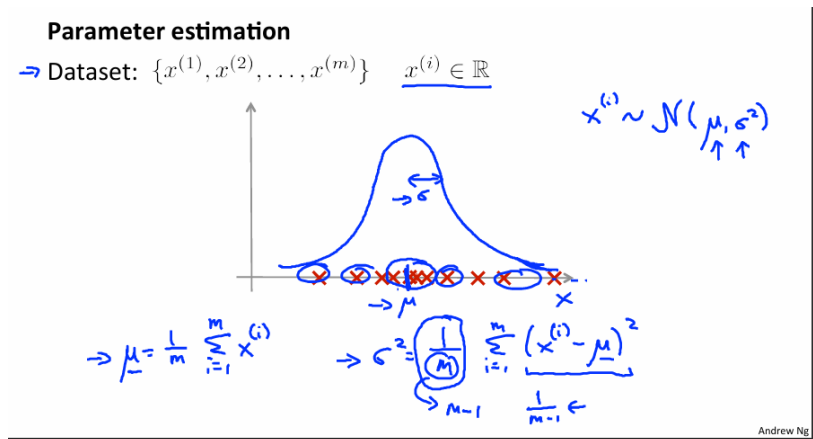


Figure 12.5: Parameter Estimation.

- $\mu$  ở đây là mean của tập mẫu.
- $\sigma = \sqrt{\text{variance}}$
- variance = trung bình khoảng cách giữa các điểm dữ liệu  $\rightarrow$  mean.

Với tập dữ liệu  $\{x^{(1)}, \dots, x^{(m)}\}$ , ta ước lượng các tham số như sau:

$$\mu = \frac{1}{m} \sum_{i=1}^m x^{(i)} \quad (12.2)$$

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu)^2 \quad (12.3)$$

Công thức trên là công thức ước lượng cho cả biến ngẫu nhiên (ở đây đang ước lượng theo công thức tính trung bình mẫu, phương sai mẫu)

## 12.4 Thuật toán Anomaly Detection

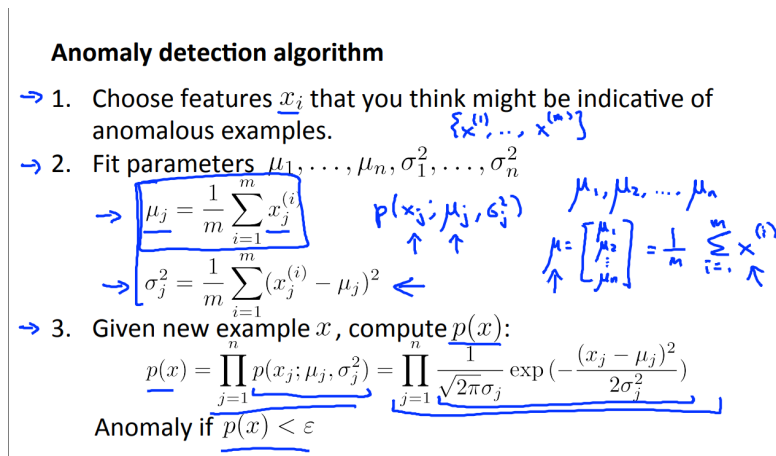


Figure 12.6: Anomaly Detection Algorithm.

Giả sử dữ liệu có  $n$  **đặc trưng độc lập**, ta mô hình hóa  $p(x)$  là tích của các xác suất riêng lẻ:

$$p(x) = p(x_1; \mu_1, \sigma_1^2) \times p(x_2; \mu_2, \sigma_2^2) \times \dots \times p(x_n; \mu_n, \sigma_n^2) = \prod_{j=1}^n p(x_j; \mu_j, \sigma_j^2) \quad (12.4)$$

### Quy trình thực hiện:

1. Chọn các đặc trưng  $x_i$  quan trọng có khả năng chỉ báo bất thường.
2. Tính toán các tham số  $\mu_1, \dots, \mu_n$  và  $\sigma_1^2, \dots, \sigma_n^2$  từ tập dữ liệu.
3. Với một mẫu mới  $x$ , tính  $p(x)$  theo công thức tích phân phối chuẩn.
4. Nếu  $p(x) < \epsilon$ , kết luận là bất thường.

Nhưng trong thực tế thì các feature gần như không thể hoàn toàn độc lập được với nhau. -> người ta thường dùng multivariate gaussian distribution.

## 12.5 Anomaly Detection vs Supervised Learning

Tại sao không dùng Supervised Learning cho các bài toán như phát hiện thư rác (Spam)?

## 12.6 Xử lý đặc trưng (Feature Engineering)

### 12.6.1 Non-Gaussian Features

Nên chọn các feature nào để dự đoán ?

- Visualize lên, xem feature nào phân phối gauss
- Thực tế sẽ có nhiều phân phối  $\neq$  gauss, có thể là long tail distribution (đuôi dài)



| Anomaly Detection                                                                                                                  | Supervised Learning                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Số lượng mẫu dương ( $y = 1$ , bất thường) rất nhỏ (0-20 mẫu). Số lượng mẫu âm ( $y = 0$ , bình thường) rất lớn.                   | Số lượng mẫu dương và mẫu âm đều lớn.                                                                                                                    |
| Có nhiều loại bất thường khác nhau. Khó để học được đặc trưng chung từ số ít mẫu dương.                                            | Đủ mẫu dương để thuật toán học được đặc trưng của lớp dương.                                                                                             |
| Các bất thường trong tương lai có thể hoàn toàn khác với những gì đã thấy.                                                         | Các mẫu dương tương lai có khả năng giống với tập huấn luyện.                                                                                            |
| Rất ít điểm bất thường trong dataset, thậm chí nó còn chưa xuất hiện (tức là đang hoạt động tốt, vẫn phải tìm ra điểm bất thường). | Có các mẫu negative rồi, mình chỉ cần học có giám sát $\rightarrow$ ngay cả khi không biết bất thường ở điểm nào, thì cũng mong muốn phân loại được spam |
| <i>Ví dụ:</i> Gian lận tài chính, lỗi sản xuất.                                                                                    | <i>Ví dụ:</i> Phân loại Spam, dự báo thời tiết, ung thư.                                                                                                 |

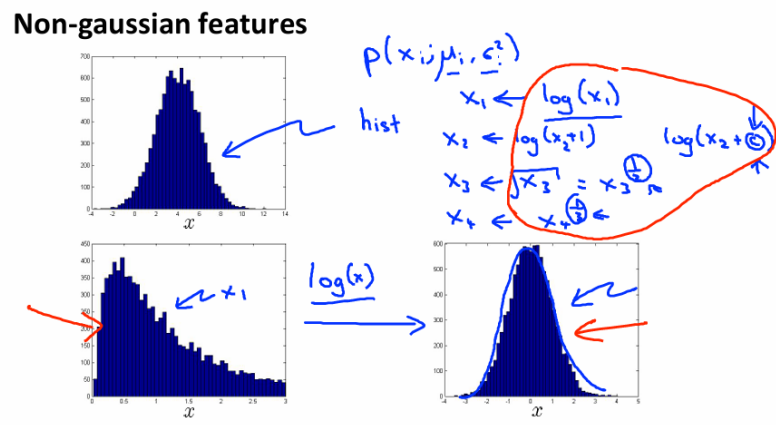


Figure 12.7: Non-gaussian features.

- Cách 1: vớt feature này đi
- Hoặc nếu thấy nó ảnh hưởng output  $\rightarrow$  có thể đưa qua log transform, dùng biến thể log, dùng hàm mũ, ...

Nếu biểu đồ histogram của một đặc trưng không có dạng hình chuông (Gaussian), ta nên chuyển đổi nó về dạng Gaussian bằng các hàm như  $\log(x)$ ,  $\log(x + c)$ ,  $\sqrt{x}$ , hoặc  $x^{1/3}$ .

### 12.6.2 Error Analysis

Nếu thuật toán thất bại trong việc phát hiện một mẫu bất thường, hãy phân tích mẫu đó để tìm ra đặc trưng mới giúp phân biệt nó với các mẫu bình thường. Ví dụ: Tạo đặc trưng mới  $x_3 = \frac{\text{CPU Load}}{\text{Memory Use}}$  để phát hiện máy tính bị kẹt vòng lặp vô hạn.

## 12.7 Multivariate Gaussian Distribution (Phân phối chuẩn đa biến)

### 12.7.1 Hạn chế của mô hình gốc

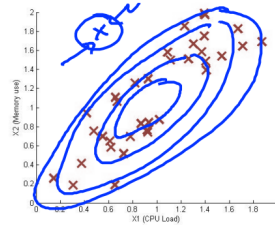
Mô hình gốc (tích các  $p(x_j)$ ) giả định các đặc trưng độc lập với nhau. Điều này có thể dẫn đến việc bỏ sót các bất thường có sự tương quan giữa các đặc trưng (ví dụ: CPU thấp nhưng Memory cao bất thường) bởi vì trong thực tế thì các đặc trưng gần như không thể nào hoàn toàn độc lập với nhau.

### 12.7.2 Mô hình đa biến

#### Anomaly detection with the multivariate Gaussian

1. Fit model  $p(x)$  by setting

$$\begin{cases} \mu = \frac{1}{m} \sum_{i=1}^m x^{(i)} \\ \Sigma = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu)(x^{(i)} - \mu)^T \end{cases}$$



2. Given a new example  $x$ , compute

$$p(x) = \frac{1}{(2\pi)^{\frac{n}{2}} |\Sigma|^{\frac{1}{2}}} \exp \left( -\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right)$$

Flag an anomaly if  $p(x) < \varepsilon$

Figure 12.8: Anomaly Detect with the multivariate Gaussian.

Sử dụng phân phối chuẩn đa biến để mô hình hóa sự tương quan:

$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left( -\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right) \quad (12.5)$$

Trong đó:

- $\mu \in R^n$ : Vector trung bình.
- $\Sigma \in R^{n \times n}$ : Ma trận hiệp phương sai (Covariance Matrix).

### 12.7.3 So sánh mô hình

- **Original Model:** Tính toán đơn giản, hoạt động tốt ngay cả khi  $m$  (số mẫu) nhỏ. Tuy nhiên, cần tạo thủ công các đặc trưng kết hợp nếu có sự tương quan.
- **Multivariate Model:** Tự động nắm bắt sự tương quan giữa các đặc trưng. Tuy nhiên, chi phí tính toán cao (tính nghịch đảo ma trận  $\Sigma^{-1}$ ) và yêu cầu số lượng mẫu lớn ( $m > n$ , tốt nhất là  $m \geq 10n$ ).

## Chapter 13

# Recommender Systems (Hệ thống gợi ý)

### 13.1 Problem Definition (Định nghĩa bài toán)

#### 13.1.1 Ví dụ: Dự đoán đánh giá phim (Predicting movie ratings)

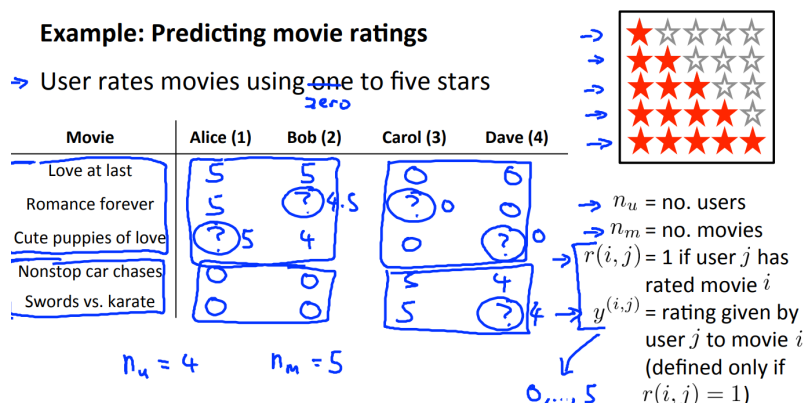


Figure 13.1: Example predict movie ratings.

Giả sử ta có dữ liệu về việc người dùng đánh giá các bộ phim theo thang điểm từ 0 đến 5 sao.

- $n_u$ : Số lượng người dùng (users).
- $n_m$ : Số lượng phim (movies).
- $r(i, j) = 1$ : Nếu người dùng  $j$  đã đánh giá phim  $i$ .
- $y^{(i, j)}$ : Điểm đánh giá của người dùng  $j$  cho phim  $i$  (chỉ xác định nếu  $r(i, j) = 1$ ).

**Mục tiêu:** Dự đoán các giá trị bị thiếu (dấu "?") trong bảng đánh giá để gợi ý phim mới cho người dùng.

## 13.2 Content-based Recommendations (Gợi ý dựa trên nội dung)

Phương pháp này dựa vào thông tin mô tả (content) của sản phẩm (có thể được cung cấp bởi nhà sản xuất, nhà phê bình,..). Ví dụ, mỗi bộ phim có các đặc trưng (features) như: mức độ lãng mạn ( $x_1$ ), mức độ hành động ( $x_2$ ), v.v.

### 13.2.1 Mô hình hóa

Với mỗi người dùng  $j$ , ta học một vector tham số  $\theta^{(j)} \in R^{n+1}$  thể hiện sở thích của họ. Dự đoán đánh giá của người dùng  $j$  cho phim  $i$  bằng:

$$(\theta^{(j)})^T x^{(i)} \quad (13.1)$$

Trong đó  $x^{(i)}$  là vector đặc trưng của phim  $i$ .

### 13.2.2 Hàm tối ưu (Optimization Objective)

Để học tham số  $\theta^{(j)}$  cho người dùng  $j$ , ta cực minimize MSE loss (kèm theo regularization term):

$$\min_{\theta^{(j)}} \frac{1}{2} \sum_{i:r(i,j)=1} ((\theta^{(j)})^T x^{(i)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{k=1}^n (\theta_k^{(j)})^2 \quad (13.2)$$

Tổng hợp cho tất cả người dùng:

$$\min_{\theta^{(1)}, \dots, \theta^{(n_u)}} \frac{1}{2} \sum_{j=1}^{n_u} \sum_{i:r(i,j)=1} ((\theta^{(j)})^T x^{(i)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (\theta_k^{(j)})^2 \quad (13.3)$$

**Vấn đề thực tế:** Việc thu thập đầy đủ và chính xác các đặc trưng (features) của phim (như độ lãng mạn, hành động) là rất khó khăn, tốn kém và mang tính chủ quan.

## 13.3 Collaborative Filtering (Lọc cộng tác)

Đây là phương pháp khắc phục nhược điểm của Content-based. Thay vì cần đặc trưng phim có sẵn, ta có thể **tự học** các đặc trưng này dựa trên đánh giá của cộng đồng người dùng.

### 13.3.1 Ý tưởng cốt lõi

- Nếu ta có  $\theta$  (sở thích người dùng), ta có thể học  $x$  (đặc trưng phim).
- Nếu ta có  $x$  (đặc trưng phim), ta có thể học  $\theta$  (sở thích người dùng).
- **Collaborative Filtering:** Thực hiện lặp đi lặp lại hoặc đồng thời việc học cả  $x$  và  $\theta$ .

### 13.3.2 Hàm tối ưu đồng thời (Simultaneous Optimization)

Hàm chi phí kết hợp cả sai số dự đoán và điều chuẩn cho cả  $x$  và  $\theta$ :

$$J(x^{(1)}, \dots, x^{(n_m)}, \theta^{(1)}, \dots, \theta^{(n_u)}) = \frac{1}{2} \sum_{(i,j):r(i,j)=1} ((\theta^{(j)})^T x^{(i)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^n (x_k^{(i)})^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (\theta_k^{(j)})^2 \quad (13.4)$$

Mục tiêu:

$$\min_{x^{(1)}, \dots, x^{(n_m)}, \theta^{(1)}, \dots, \theta^{(n_u)}} J(\dots) \quad (13.5)$$

### 13.3.3 Thuật toán (Collaborative Filtering Algorithm)

1. **Khởi tạo:** Gán giá trị ngẫu nhiên nhỏ cho  $x^{(1)}, \dots, x^{(n_m)}$  và  $\theta^{(1)}, \dots, \theta^{(n_u)}$ .
2. **Tối ưu hóa:** Sử dụng Gradient Descent (hoặc thuật toán khác) để cực tiểu hóa  $J$ .

- Cập nhật  $x^{(i)}$ :

$$x_k^{(i)} := x_k^{(i)} - \alpha \left( \sum_{j:r(i,j)=1} ((\theta^{(j)})^T x^{(i)} - y^{(i,j)}) \theta_k^{(j)} + \lambda x_k^{(i)} \right)$$

- Cập nhật  $\theta^{(j)}$ :

$$\theta_k^{(j)} := \theta_k^{(j)} - \alpha \left( \sum_{i:r(i,j)=1} ((\theta^{(j)})^T x^{(i)} - y^{(i,j)}) x_k^{(i)} + \lambda \theta_k^{(j)} \right)$$

3. **Dự đoán:** Với một người dùng có tham số  $\theta$  và một bộ phim có đặc trưng  $x$ , điểm dự đoán là  $\theta^T x$ .

## 13.4 Low Rank Matrix Factorization (Phân rã ma trận hạng thấp)

Collaborative Filtering còn được gọi là **Low Rank Matrix Factorization**. Ma trận dự đoán  $Y_{pred}$  được tính bằng tích của hai ma trận:

$$Y_{pred} = X\Theta^T \quad (13.6)$$

Trong đó:

- $X$ : Ma trận các vector đặc trưng phim (kích thước  $n_m \times n$ ).
- $\Theta$ : Ma trận các vector tham số người dùng (kích thước  $n_u \times n$ ).

**Ý nghĩa:** Phương pháp này cố gắng phân tách ma trận đánh giá ban đầu (thường rất lớn và thưa) thành tích của hai ma trận con có hạng thấp (low rank). Điều này giúp:

- Giảm chiều dữ liệu cần lưu trữ và tính toán.
- Nắm bắt được các đặc trưng ẩn (latent features) quan trọng nhất.
- Ví dụ: Thay vì lưu ma trận  $5 \times 4$  (20 phần tử), ta lưu 2 ma trận con  $5 \times 2$  và  $4 \times 2$  (tổng 18 phần tử), khi dữ liệu lớn hiệu quả nén sẽ rất đáng kể (chưa tính đến việc chỉ lưu trữ các cột độc lập tuyến tính, hàng của ma trận sẽ nhỏ đi nhiều).

## Chapter 14

# Support Vector Machines (SVM)

### 14.1 Optimization Objective (Mục tiêu tối ưu hóa)

#### 14.1.1 Từ Logistic Regression đến SVM

Để hiểu SVM, ta bắt đầu xem xét lại hàm chi phí của Logistic Regression. Với một mẫu dữ liệu  $(x, y)$ , hàm mất mát là:

$$-(y \log h_{\theta}(x) + (1 - y) \log(1 - h_{\theta}(x)))$$

Trong đó  $h_{\theta}(x) = \frac{1}{1+e^{-\theta^T x}}$ .

Logistic Regression chỉ chọn ra Decision Boundary phân loại đúng cho dữ liệu, không phải là Decision Boundary hợp lý nhất -> Nó sẽ phân loại sai nhiều trường hợp

Ý tưởng của SVM là tìm ra Decision Boundary sao cho phân tách tốt nhất, tách bạch nhất giữa các class.

Trong SVM, chúng ta thay thế hàm  $-\log(\dots)$  bằng một cost function mới gọi là **Hinge Loss** (Hàm gãy khúc), ký hiệu là  $\text{cost}_1(z)$  và  $\text{cost}_0(z)$ .

- **Nếu  $y = 1$  (muốn  $\theta^T x \gg 0$ ):** Hàm  $\text{cost}_1(z)$  bằng 0 khi  $z \geq 1$ . Nếu  $z < 1$ , cost tăng tuyến tính.
- **Nếu  $y = 0$  (muốn  $\theta^T x \ll 0$ ):** Hàm  $\text{cost}_0(z)$  bằng 0 khi  $z \leq -1$ . Nếu  $z > -1$ , cost tăng tuyến tính.

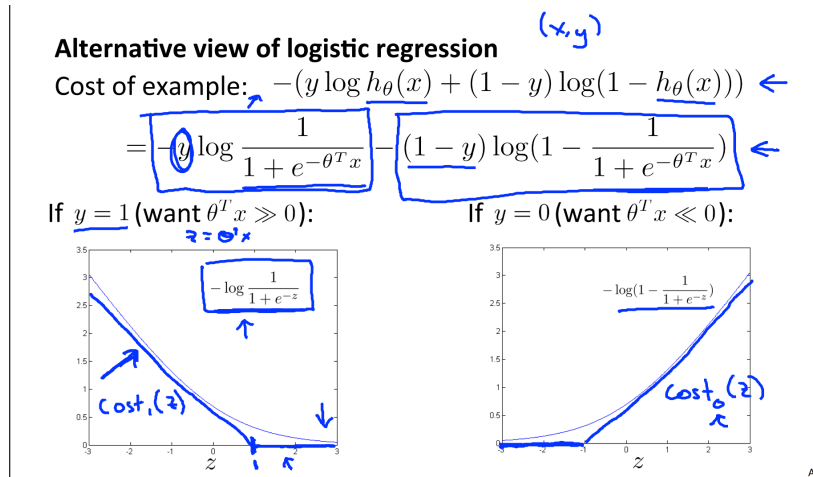


Figure 14.1: SVM's Cost function idea.

### 14.1.2 Hàm chi phí của SVM

Công thức tổng quát của SVM (bỏ qua hằng số  $\frac{1}{m}$  và thay thế  $\lambda$  bằng  $C$ ):

$$J(\theta) = C \sum_{i=1}^m \left[ y^{(i)} \text{cost}_1(\theta^T x^{(i)}) + (1 - y^{(i)}) \text{cost}_0(\theta^T x^{(i)}) \right] + \frac{1}{2} \sum_{j=1}^n \theta_j^2 \quad (14.1)$$

**Phân tích tham số  $C$ :**

- $C$  đóng vai trò tương tự như  $\frac{1}{\lambda}$ .
- **Nếu  $C$  rất lớn:** Tương đương  $\lambda \approx 0$  (Không điều chuẩn). SVM sẽ cố gắng phân loại đúng mọi điểm dữ liệu, dễ bị ảnh hưởng bởi điểm nhiễu (outliers) → **High Variance (Overfitting)**.
- **Nếu  $C$  nhỏ:** Tương đương  $\lambda$  lớn. SVM chấp nhận một số lỗi sai để đổi lấy lề (margin) rộng hơn → **High Bias (Underfitting)**.

## 14.2 Large Margin Intuition (Trực giác về Lề lớn)

Tại sao SVM còn được gọi là **\*\*Large Margin Classifier\*\***?

### 14.2.1 Điều kiện an toàn

Khác với Logistic Regression chỉ cần  $\theta^T x \geq 0$  để dự đoán  $y = 1$ , SVM yêu cầu điều kiện khắt khe hơn để  $\text{Cost} = 0$ :

- Nếu  $y = 1$ , ta muốn  $\theta^T x \geq 1$  (không phải chỉ  $\geq 0$ ).
- Nếu  $y = 0$ , ta muốn  $\theta^T x \leq -1$  (không phải chỉ  $< 0$ ).

Điều này tạo ra một "vùng an toàn" hoặc khoảng đệm giữa các lớp.

### 14.2.2 Decision Boundary (Ranh giới quyết định)

Khi  $C$  rất lớn, SVM sẽ tìm đường phân cách sao cho khoảng cách ngắn nhất từ đường đó đến các điểm dữ liệu (gọi là Margin) là lớn nhất.

#### SVM Decision Boundary: Linearly separable case

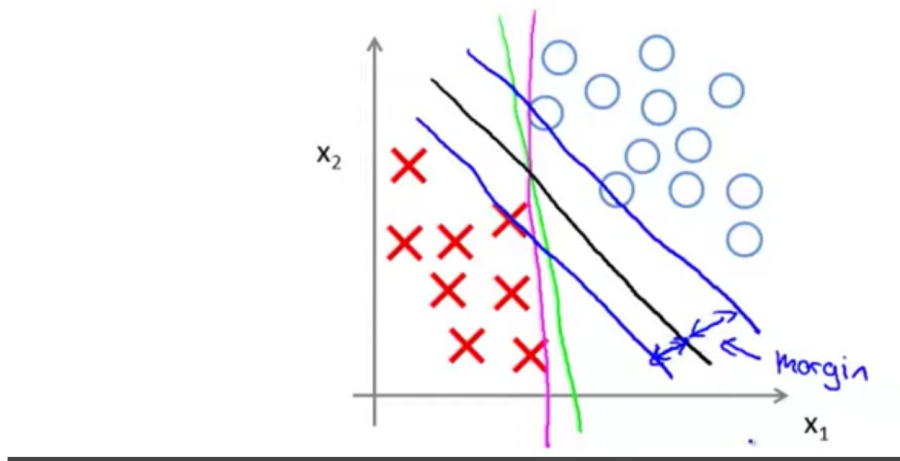


Figure 14.2: SVM tìm kiếm Hyperplane sao cho khoảng cách đến các Support Vectors là lớn nhất.

Lúc training thì sẽ train theo Support Vectors, còn inference vẫn dùng Decision Boundary.

**Xử lý nhiễu (Outliers):** Nếu có một điểm dữ liệu nhiễu nằm xen vào giữa, một  $C$  quá lớn sẽ khiến Decision Boundary bị lệch hẳn đi để bao lấy điểm đó (Margin nhỏ lại). Nếu giảm  $C$ , SVM sẽ bỏ qua điểm nhiễu đó để giữ được Margin lớn chung cho tổng thể.

## 14.3 Mathematics Behind Large Margin Classification (Toán học đằng sau)

### 14.3.1 Inner Product

Phép toán đằng sau đó bản chất là inner product. Các bạn có thể tìm hiểu kỹ hơn toán học đằng sau SVM.

Xét  $u, v \in \mathbb{R}^2$ :  $u^T v = p \cdot \|u\|$ , trong đó:

- $\|u\|$ : Độ dài của vector  $u$ .
- $p$ : Độ dài hình chiếu của vector  $v$  lên vector  $u$ .

### 14.3.2 Tối ưu hóa Margin

Hàm mục tiêu của SVM là cực tiểu hóa  $\frac{1}{2}\|\theta\|^2$ . Ta có:  $\theta^T x^{(i)} = p^{(i)} \cdot \|\theta\|$ .

- Để  $\theta^T x^{(i)} \geq 1$ , nếu  $\|\theta\|$  nhỏ (do cực tiểu hóa), thì  $p^{(i)}$  (hình chiếu của  $x$  lên  $\theta$ ) phải lớn.



- $p^{(i)}$  lớn đồng nghĩa với việc các điểm dữ liệu  $x^{(i)}$  nằm xa vector pháp tuyến  $\theta$  (tức là nằm xa đường phân cách).
- $\rightarrow$  Tạo ra **Large Margin**.

## 14.4 Kernels (Hạt nhân)

Khi dữ liệu không thể phân tách tuyến tính (Non-linear), ta cần tạo ra các đặc trưng phức tạp hơn.

### 14.4.1 Ý tưởng Landmarks (Điểm mốc)

Thay vì tạo đa thức bậc cao  $(x_1^2, x_1x_2, \dots)$ , ta chọn các điểm mốc  $l^{(1)}, l^{(2)}, \dots$  trong không gian. Với mỗi điểm dữ liệu  $x$ , ta tính độ tương đồng (**Similarity**) với các landmarks này.

### 14.4.2 Gaussian Kernel (RBF Kernel)

Công thức tính độ tương đồng:

$$f_i = \text{similarity}(x, l^{(i)}) = \exp\left(-\frac{\|x - l^{(i)}\|^2}{2\sigma^2}\right) \quad (14.2)$$

**Phân tích:**

- Nếu  $x \approx l^{(i)}$ : Tử số  $\approx 0 \Rightarrow f_i \approx e^0 = 1$  (Rất giống).
- Nếu  $x$  xa  $l^{(i)}$ : Tử số lớn  $\Rightarrow f_i \approx e^{-\infty} = 0$  (Không giống).

Trong thực tế, ta chọn  $l^{(i)} = x^{(i)}$  (Mỗi điểm training là một landmark). Vậy nếu có  $m$  mẫu, ta sẽ có  $m$  features mới  $f \in R^m$ .

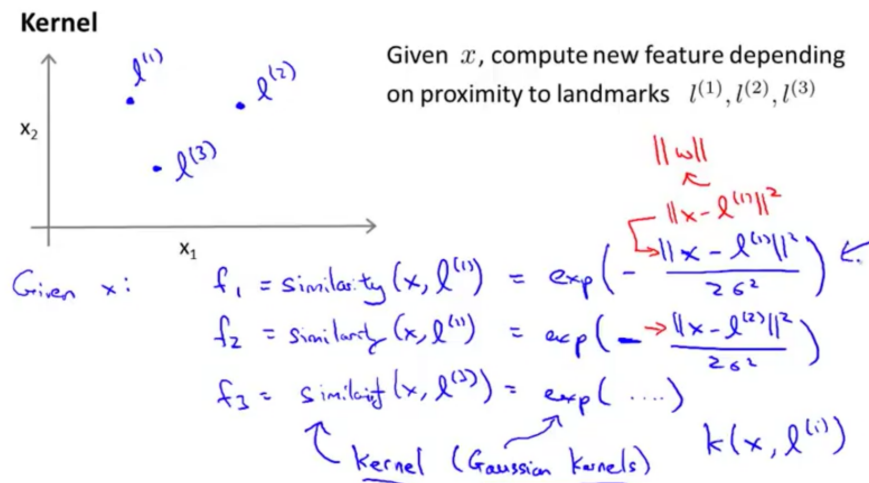


Figure 14.3: Ví dụ cách tính feature bằng Gaussian kernel.

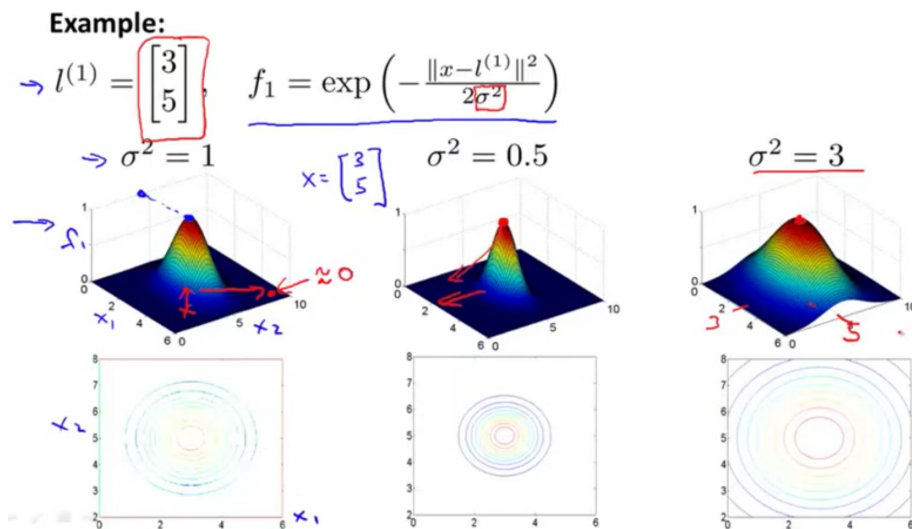


Figure 14.4: Minh họa ảnh hưởng của variance.

#### 14.4.3 Ảnh hưởng của $\sigma^2$

- $\sigma^2$  **lớn**: Đỉnh chuông Gaussian rộng và thoải. Feature  $f_i$  biến thiên chậm.  $\rightarrow$  High Bias (Underfitting).
- $\sigma^2$  **nhỏ**: Đỉnh chuông nhọn và dốc. Feature  $f_i$  giảm cực nhanh khi rời xa landmark.  $\rightarrow$  High Variance (Overfitting).

#### 14.4.4 Feature Scaling cho SVM

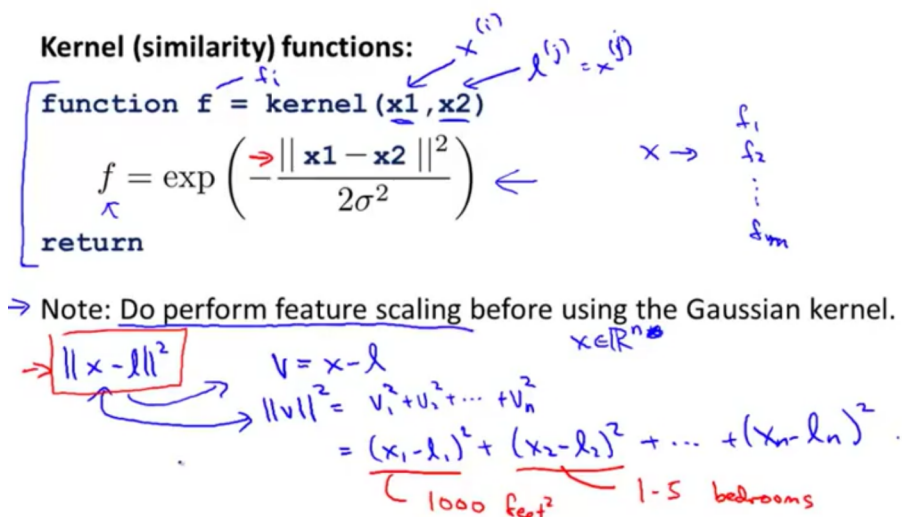


Figure 14.5: SVM Feature Scaling example.

- Ở đây đang bỏ qua  $x_0$ , coi  $x \in \mathbb{R}^n$ .

- khi feat lớn, còn lại nhỏ, thì  $\|x - l\|^2$  gần như hoàn toàn bị chi phối bởi feat, các phần khác như bedroom sẽ bị bỏ qua.  
→ cần phải "do perform feature scaling before using the Gaussian kernel" (ví dụ có thể dùng Standardization)

## 14.5 SVM trong thực tế (Using an SVM)

Không nên tự code SVM từ đầu (trừ khi để học). Hãy dùng các thư viện tối ưu cao như `libsvm`, `linear-svm` (trong `sklearn`).

### 14.5.1 Các bước thực hiện

1. Chọn tham số  $C$ .
2. Chọn Kernel:
  - **No Kernel (Linear Kernel):** Dùng khi số lượng features  $n$  lớn, số mẫu  $m$  nhỏ.
  - **Gaussian Kernel:** Dùng khi  $n$  nhỏ,  $m$  trung bình.
  - **Bắt buộc phải Feature Scaling** trước khi dùng.

### 14.5.2 So sánh Logistic Regression và SVM

Giả sử  $n$  là số features,  $m$  là số mẫu.

| Trường hợp           | Ví dụ                                     | Khuyến dùng                                                                                                             |
|----------------------|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| $n \geq m$           | $n = 10k, m = 10..1k$<br>(Genomics, Text) | <b>Logistic Regression</b> hoặc <b>SVM Linear</b> . (Không đủ dữ liệu để học Kernel phức tạp).                          |
| $n$ nhỏ, $m$ vừa     | $n = 1..1k, m = 10..10k$                  | <b>SVM Gaussian Kernel</b> . (Hoạt động cực tốt).                                                                       |
| $n$ nhỏ, $m$ rất lớn | $n = 1..1k, m = 50k+$                     | Tạo thêm features thủ công, rồi dùng <b>Logistic Regression</b> hoặc <b>SVM Linear</b> . (Gaussian Kernel sẽ rất chậm). |

Table 14.1: Hướng dẫn lựa chọn thuật toán

**Lưu ý:** Neural Network hoạt động tốt trong tất cả các trường hợp trên nhưng có thể chậm hơn và khó huấn luyện hơn (Local optima). SVM (với hàm lỗi) đảm bảo luôn tìm được Global Optimum.

Nếu muốn thì mình hoàn toàn có thể dùng Kernel cho các thuật toán Clustering khác như Logistic Regression, nhưng các trick tính toán của Kernel chỉ hỗ trợ tốt cho SVM, không khái quát hóa tốt cho các thuật toán khác (ví dụ nếu dùng cho Logistic Regression thì có thể sẽ tính rất chậm.)

## Appendix A

# Appendix: Summary & Cheat Sheet (Phụ lục và Tóm tắt)

### A.1 Bảng chọn lựa thuật toán (Algorithm Selection Guide)

Khi đối mặt với một bài toán thực tế, câu hỏi đầu tiên luôn là: "Nên dùng thuật toán nào?". Dưới đây là bảng tóm tắt dựa trên tính chất dữ liệu và mục tiêu bài toán.

### A.2 Bảng ký hiệu toán học (Mathematical Notation)

Để tránh nhầm lẫn trong quá trình đọc lại các công thức, dưới đây là quy ước ký hiệu chung cho toàn bộ tài liệu:

| Ký hiệu              | Ý nghĩa                                                                  |
|----------------------|--------------------------------------------------------------------------|
| $m$                  | Số lượng mẫu dữ liệu huấn luyện (training examples).                     |
| $n$                  | Số lượng đặc trưng (features) (không tính bias unit $x_0$ ).             |
| $x^{(i)}$            | Vector đầu vào (input features) của mẫu thứ $i$ .                        |
| $y^{(i)}$            | Giá trị đầu ra thực tế (output/label) của mẫu thứ $i$ .                  |
| $(x^{(i)}, y^{(i)})$ | Một mẫu huấn luyện (training example).                                   |
| $h_{\theta}(x)$      | Hàm giả thuyết (Hypothesis function).                                    |
| $\theta$ (hoặc $W$ ) | Các tham số (parameters/weights) của mô hình.                            |
| $J(\theta)$          | Hàm chi phí (Cost function/Loss function).                               |
| $\alpha$             | Tốc độ học (Learning rate).                                              |
| $\lambda$            | Tham số điều chuẩn (Regularization parameter).                           |
| $L$                  | Tổng số lớp (layers) trong Neural Network.                               |
| $s_l$                | Số lượng đơn vị (units) trong lớp $l$ .                                  |
| $\Sigma$             | Ma trận hiệp phương sai (Covariance Matrix) trong PCA/Anomaly Detection. |
| $\mu$                | Vector trung bình (Mean vector).                                         |

### A.3 Các bước xây dựng hệ thống ML (Pipeline Checklist)

Khi bắt đầu một dự án mới, hãy tuân theo quy trình sau để tiết kiệm thời gian:

1. **Khởi tạo nhanh:** Xây dựng một thuật toán đơn giản nhất có thể (quick and dirty) để chạy được ngay.

| Loại bài toán                                   | Dữ liệu đầu ra ( $y$ )                         | Thuật toán gợi ý                                                                      | Lưu ý                                               |
|-------------------------------------------------|------------------------------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------|
| <b>Regression</b> (Hồi quy)                     | Liên tục (Continuous)<br>Vd: Giá nhà, nhiệt độ | <b>Linear Regression</b><br>(Có thể thêm Polynomial)                                  | Dùng Regularization nếu bị Overfitting.             |
| <b>Classification</b> (Phân loại)               | Rời rạc (Discrete)<br>Vd: 0/1, Spam/Non-spam   | <b>Logistic Regression</b> (đơn giản)<br><b>Neural Networks</b> (phức tạp, phi tuyến) | Neural Net mạnh mẽ với dữ liệu lớn (ảnh, âm thanh). |
| <b>Clustering</b> (Phân cụm)                    | Không có nhãn (Unlabeled)                      | <b>K-Means</b>                                                                        | Cần chọn $K$ hợp lý (Elbow method).                 |
| <b>Anomaly Detection</b> (Phát hiện bất thường) | Không có nhãn (hoặc rất ít nhãn dương)         | <b>Gaussian Distribution</b> (Density Estimation)                                     | Dùng khi số lượng $y = 1$ rất nhỏ (0-20 mẫu).       |
| <b>Recommender</b> (Gợi ý)                      | Ma trận đánh giá thưa (Sparse Matrix)          | <b>Collaborative Filtering</b> (Low Rank Matrix Factorization)                        | Tự học đặc trưng ( $x$ ) và sở thích ( $\theta$ ).  |
| <b>Data Compression</b> (Nén/Trực quan hóa)     | Dữ liệu nhiều chiều ( $n$ lớn)                 | <b>PCA</b> (Principal Component Analysis)                                             | Dùng dùng PCA để chống Overfitting.                 |

Table A.1: Hướng dẫn chọn thuật toán Machine Learning cơ bản

2. **Chia dữ liệu:** Chia dataset thành 3 phần: Training (60%) - Cross Validation (20%) - Test (20%).
3. **Vẽ Learning Curves:** Kiểm tra xem mô hình đang bị High Bias (cần thêm features, tăng độ phức tạp) hay High Variance (cần thêm data, regularization).
4. **Phân tích lỗi (Error Analysis):** Kiểm tra thủ công các mẫu bị dự đoán sai trong tập Cross Validation để tìm quy luật.
5. **Đánh giá:** Sử dụng Precision/Recall hoặc F1-Score nếu dữ liệu bị lệch (skewed classes), không chỉ dựa vào Accuracy.

## A.4 Thư viện Python tham khảo

Các thuật toán lý thuyết trên được hiện thực hóa qua các thư viện phổ biến sau:

- **NumPy & Pandas:** Xử lý ma trận, vector và đọc dữ liệu ( $X, y$ ).
- **Matplotlib & Seaborn:** Vẽ đồ thị (Learning curves, Data visualization).

- **Scikit-Learn (sklearn):**
  - `sklearn.linear_model`: Linear/Logistic Regression.
  - `sklearn.cluster.KMeans`: K-Means.
  - `sklearn.decomposition.PCA`: PCA.
  - `sklearn.svm`: Support Vector Machines (thường dùng thay thế Neural Net cho bài toán nhỏ).
  - `sklearn.model_selection`: Train/Test split, Cross-validation.
- **TensorFlow / PyTorch:** Dùng cho Neural Networks phức tạp (Deep Learning).